Cong *et al. Cybersecurity* (2026) 9:126
https://doi.org/10.1186/s42400-025-00462-6

Cybersecurity

RESEARCH Open Access

Dualmatrix: conquering zkSNARK for large matrix multiplication

Mingshu Cong[1*], Tsz Hon Yuen[2] and Siu Ming Yiu[1]

Abstract

We present `DualMatrix`, a zkSNARK solution for large-scale matrix multiplication. Classical zkSNARK protocols typically underperform in data analytic contexts, hampered by the large size of datasets and the superlinear nature of matrix multiplication. `DualMatrix` excels in its scalability. The prover time of `DualMatrix` scales linearly with respect to the number of non-zero elements in the input matrices. For $n \times n$ matrix multiplication with N non-zero elements across three input matrices, `DualMatrix` employs a structured reference string (SRS) of size $O(n)$, and achieves RAM usage of $O(N+n)$, transcript size of $O(\log n)$, prover time of $O(N+n)$, and verifier time of $O(\log n)$. The prover time, notably at $O(N+n)$ and surpassing all existing protocols, includes $O(N+n)$ field multiplications and $O(n)$ exponentiations and pairings within bilinear groups. These efficiencies make `DualMatrix` effective for linear algebra on large matrices common in real-world applications. We evaluated `DualMatrix` with $2^{15} \times 2^{15}$ input matrices each containing $1G$ non-zero integers, which necessitate $32T$ integer multiplications in naive matrix multiplication. `DualMatrix` recorded prover and verifier times of 150.84s and 0.56s, respectively. When applied to $1M \times 1M$ sparse matrices each containing $1G$ non-zero integers, it demonstrated prover and verifier times of 1, 384.45s and 0.67s. Our approach outperforms current zkSNARK solutions by successfully handling the large matrix multiplication task in experiment. We extend matrix operations from field matrices to group matrices, formalizing group matrix algebra. This mathematical advancement brings notable symmetries beneficial for high-dimensional elliptic curve cryptography. By leveraging the bilinear properties of our group matrix algebra in the context of the two-tier commitment scheme, `DualMatrix` achieves efficiency gains over previous matrix multiplication arguments. To accomplish this, we extend and enhance Bulletproofs to construct an inner product argument featuring a transparent setup and logarithmic verifier time.

Keywords ZkSNARK, Sparse Matrix Multiplication, Large Matrix, Pairing Group

Introduction

Succinct non-interactive arguments of knowledge (SNARKs), particularly zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs), find extensive applications in blockchain, cryptocurrency, and verifiable computation. However, their application is limited in areas like data analysis and machine learning where large matrices are involved. The bottleneck is the slow prover time due to matrix operations, particularly matrix multiplication. Because of the inherent superlinear time complexity of matrix multiplication, general-purpose zkSNARKs lead to superlinear prover times when applied to tasks involving large matrix multiplication.

The schoolbook method for $n \times n$ matrix multiplication requires n^3 multiplications. The most efficient known algorithms have time complexities of $O(n^{2.37})$ for dense matrices (Vassilevska Williams 2012) and $O(N^{0.7}n^{1.2} + n^{2+o(1)})$ for sparse matrices (Yuster and

*Correspondence:
Mingshu Cong
mcong@connect.hku.hk
[1] Department of Computer Science, The University of Hong Kong, Pokfulam Road, Pokfulam, Hong Kong
[2] Faculty of Information Technology, Monash University, Exhibition Walk, Melbourne, Australia

Zwick 2005), where N represents the count of non-zero elements in the matrices. Consequently, applying general-purpose zkSNARKs to matrix multiplication leads to large circuits and slow prover times. Existing zkSNARK protocols for matrix multiplication are typically characterized by $O(n^3)$ prover time. For instance, the Libra protocol (Xie et al. 2019) reports prover times of 0.79s and 50s for 64×64 and 256×256 matrices. Its prover time is expected to escalate to 0.89h for $1,024 \times 1,024$ matrices, although the precise concrete prover time is lacking due to the memory overflow issues incurred. Thaler's protocol (Thaler 2013), originally designed for public matrices and not succinct, achieves a prover and verifier time complexity of $O(N \log n)$ when handling sparse matrices. Eliminating the logarithmic term from the prover time complexity is non-trivial. These problems underscore the challenges in developing scalable zkSNARK protocols for matrix multiplication.

In practical scenarios, large matrices are common. For instance, major cryptocurrency exchanges support over 2^8 cryptocurrencies and more than 2^{20} users. Recording the cryptocurrency holdings of each user necessitates a $2^{20} \times 2^8$ matrix. Computing the account balance (in USD) of each user at each minute of the day—totaling over 2^{10} time points—requires multiplying a $2^{20} \times 2^8$ matrix with a $2^8 \times 2^{10}$ matrix. Additionally, consider an iPhone 15 photo with 48 million pixels, equating to a matrix size exceeding $2^{22} \times 2^{23}$. Verifiable computation to extract features from such an image would demand a SNARK protocol capable of handling matrix multiplication at this magnitude. Moreover, large AI models such as the Llama2-70B model (Touvron et al. 2023) perform matrix multiplications with dimensions up to $5,120 \times 32,000$. To enable verifiable computation for such large models, we need an efficient SNARK protocol for large matrix multiplication.

Fully Succinct zkSNARK The goal of this paper is to develop a zkSNARK protocol for integer matrix multiplication with prover time linear to the *number of non-zero elements* in the input matrices rather than the number of multiplication gates involved. As defined in Gabizon et al. (2019), a fully succinct zkSNARK protocol is characterized by:

- *Setup time.* The setup time is linear relative to the witness size.
- *Prover time.* The prover time is linear relative to the witness size.
- *Transcript size.* The transcript size is logarithmic relative to the witness size.
- *Verifier time.* The verifier time is logarithmic relative to the witness size.

In the context of matrix multiplication, we define the witness size as N, the total number of non-zero elements in the input matrices. This choice is motivated by the $O(N)$ space requirement for efficient storage of the input matrices. This differs from M, the circuit size or the number of multiplication gates used in general-purpose zkSNARKs, and n^2, which counts all elements in the matrices, including zeros. In this paper, we assume $n \le N \le n^2$. With this witness size, existing zkSNARK solutions with $O(n^2)$ prover time lose full succinctness when applied to matrix multiplication tasks when $N = o(n^2)$.

Main result

`DualMatrix` is a specialized zkSNARK protocol tailored for input matrices $\boldsymbol{a} \in \mathbb{Z}_p^{n\times n}, \boldsymbol{b} \in \mathbb{Z}_p^{n\times n}$, and $\boldsymbol{c} \in \mathbb{Z}_p^{n\times n}$, where $\boldsymbol{c} = \boldsymbol{ab}$. These matrices are securely committed using the two-tier commitment scheme (Groth 2011), and the commitments are denoted by $C_a, C_b, C_c \in \mathbb{G}_T$. In our notation, bold letters (e.g., $\boldsymbol{c}$) represent matrices, while subscripted smaller letters (e.g., c_{ij}) denote scalar elements within the matrices. We use the symbol $\oplus$ for point addition in the elliptic curve groups $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$ of prime order p, with a pairing operation $e : \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T$. We interchangeably use *exponentiation* and scalar multiplication for group operations. We utilize public base vectors $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$. The two-tier commitment to $\boldsymbol{c} = \{c_{ij}\} \in \mathbb{Z}_p^{n\times n}$ is given by:

$$C_c := \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{c} \,|\, \vec{\boldsymbol{H}} \rangle := \bigoplus_{i=1}^{n} \bigoplus_{j=1}^{n} c_{ij}\, e(G_i, H_j) \in \mathbb{G}_T \tag{1}$$

We refer to $\langle \cdot | \cdot | \cdot \rangle$ as the *Dirac notation*, representing the projection of $\boldsymbol{c}$ onto $\vec{\boldsymbol{G}}$ and $\vec{\boldsymbol{H}}$. Analogous definitions apply for $C_a = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle$ and $C_b = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{b} \,|\, \vec{\boldsymbol{H}} \rangle$. Our matrix commitment scheme exhibits the *homomorphic* property, which is valuable for many real-world applications.

`DualMatrix` is designed for the following relation $\mathcal{R}_{\mathsf{dualMatrix}}$:

$$\left\{ \begin{pmatrix} C_c, C_a, C_b \in \mathbb{G}_T, \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \\ \vec{\boldsymbol{H}} \in \mathbb{G}_2^n, \\ \tilde{U} \in \mathbb{G}_T \end{pmatrix} : \begin{pmatrix} \boldsymbol{c} \in \mathbb{Z}_p^{n\times n}, \\ \boldsymbol{a} \in \mathbb{Z}_p^{n\times n}, \\ \boldsymbol{b} \in \mathbb{Z}_p^{n\times n}, \\ \tilde{c}, \tilde{a}, \tilde{b} \in \mathbb{Z}_p \end{pmatrix} \middle| \begin{pmatrix} \boldsymbol{c} = \boldsymbol{ab} \\ \wedge\, C_c = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{c} \,|\, \vec{\boldsymbol{H}} \rangle \oplus \tilde{c}\tilde{U} \\ \wedge\, C_a = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle \oplus \tilde{a}\tilde{U} \\ \wedge\, C_b = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{b} \,|\, \vec{\boldsymbol{H}} \rangle \oplus \tilde{b}\tilde{U} \end{pmatrix} \right\}. \tag{2}$$

We restrict our discussion to square matrices; the generalization to non-square matrices is straightforward. For $n \times n$ matrix multiplication with N non-zero elements in total, `DualMatrix` achieves prover time of $O(N + n)$, verifier time of $O(\log n)$, and a transcript size of $O(\log n)$, with the SRS size being $O(n)$. By caching intermediate results during matrix commitment, the prover time is significantly reduced to $2N$ field multiplications and $O(n)$

group operations. Throughout this paper, group operations refer to both exponentiations and pairings within bilinear groups.

Table 1 compares `DualMatrix` with existing protocols. Timings in the table are measured in field and group operations for fair comparison. An exponentiation takes $O(\lambda)$ field multiplications, where λ represents the security parameter, and an optimal Ate pairing takes 12 exponentiations and 4 field multiplications. Note that while Thaler's protocol (Thaler 2013), LegoSNARK (Campanelli et al. 2019), and QuickSilver (Yang et al. 2021) offer $O(n^2)$ prover time, their linear verifier times limit their applicability as zkSNARKs.

Technical contributions

Strictly linear zkSNARK for sparse matrix multiplication

For $n \times n$ sparse matrix multiplication with N non-zero elements, existing zkSNARK protocols exhibit, at best, quasilinear complexity relative to N, on the order of $O(N \log n)$. This quasilinear term becomes significant for large matrices. This complexity arises from the use of Bulletproof on n^2-vectors, as in zkMatrix (Cong et al. 2024), or a sumcheck protocol on n^2-term polynomials, as in Thaler's protocol (Thaler 2013). A key issue is that folding an n^2-vector with N non-zero elements can result in a vector with more than $N/2$ non-zero elements. Consequently, the time complexity could escalate to $O(N \log n)$ for the $2 \log n$ iterations. To address this issue, we substitute some of the most costly computations in Bulletproof with matrix projections conducted before the Bulletproof iteration, which maintain strict linearity relative to N.

Accelerating Bulletproofs by leveraging the rank-1 property of two-tier commitments.

The two-tier commitment corresponds to a Pedersen vector commitment on a rank-1 $\mathbb{G}_T$ matrix, described as follows:

$$\langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle = \mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top}).$$

Here, $\otimes$ is the tensor product defined for group vectors, and $\mathsf{vec}(\cdot)$ represents the vectorization of matrices. $\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top}$ is a rank-1 matrix in $\mathbb{G}_T$ because it is the tensor product of two vectors. The tensor product between group vectors and the inner product between field vectors and group vectors are formally defined in Sect. 3.2.

A key contribution of this work is to exploit the rank-1 property of the $\mathbb{G}_T$ matrix, which significantly reduces the prover's workload in Bulletproof for an n^2-vector from $O(n^2)$ group operations to $O(n)$ group operations and pairings.

Fewer group operations

Group operations are much more expensive than field operations. Our method requires only $O(n)$ group operations and pairings in prover time, compared to the $O(n^2)$ group operations required by Bulletproof and zkMatrix. By exploiting the rank-1 property and incorporating it into each line of the Bulletproof iteration, we obtain an inner-product argument on two n^2-vectors requiring

Table 1 Comparison of Zero-Knowledge Proofs for Matrix Multiplication

Protocol	SRS Size	RAM Usage	Communi-cation	Timing	
				Prover	Verifier
Thaler's Thaler (2013)	NA	$O(n^2)$	$O(\log n)$	$O(N \log n)\mathbb{F}$	$O(N \log n)\mathbb{F}$
QuickSilver Yang et al. (2021)	NA	$O(n^2)$	$O(n^2)$	$O(n^2)\mathbb{F}$	$O(n^2)\mathbb{F}$
LegoSNARK Campanelli et al. (2019)	$O(n^2)$	$O(n^2)$	$O(\log n)$	$O(n^2)\mathbb{E}$	$O(n^2)\mathbb{F}$ $+O(\log n)\mathbb{E}$
Pinocchio Parno et al. (2016)	$O(n^3)$	$O(n^3)$	$O(1)$	$O(n^3)\mathbb{E}$	$O(1)\mathbb{P}$
Libra Xie et al. (2019)	$O(n^3)$	$O(n^3)$	$O(\log n)$	$O(n^3)\mathbb{E}$	$O(\log n)\mathbb{E}$
Bulletproof Bünz et al. (2018)	$O(n^2)$	$O(n^2)$	$O(\log n)$	$O(n^2)\mathbb{E}$	$O(n^2)\mathbb{E}$
zkMatrix Cong et al. (2024)	$O(n^2)$	$O(n^2)$	$O(\log n)$	$O(n^2)\mathbb{E}$	$O(\log n)\mathbb{E}$
`DualMatrix`	$O(n)$	$O(N+n)$	$O(\log n)$	$O(N+n)\mathbb{F}$ $+O(n)(\mathbb{E}+\mathbb{P})$	$O(\log n)\mathbb{E}$

Metrics in the table pertain to $n \times n$ matrix multiplications over $\mathbb{Z}_p$, with N representing the total non-zero elements across three input matrices. For dense matrices, $N = 3n^2$. We assume $n \le N \le n^2$ and $N = o(n^2)$. The timing columns only show the dominating factor(s). $\mathbb{F}$, $\mathbb{E}$, and $\mathbb{P}$ represent the number of field multiplications, scalar multiplications in pairing groups, and pairings, respectively. An optimal Ate pairing takes 12 exponentiations and 4 field multiplications. The sizes of the SRS, RAM usage, and communication are measured in units of field element size. The sizes of group elements are constant multiples of the size of a field element. The times required for committing to the input matrices and computing the unverified matrix multiplication are excluded from the prover time. zkMatrix proposes a matrix multiplication argument based on an improved version of Bulletproofs. The metrics shown in the row "Bulletproof" correspond to the performance of this matrix multiplication argument using the original, unimproved Bulletproofs protocol

only $O(n)$ group operations and pairings, improving upon the $O(n^2)$ group operations needed by Bulletproofs.

Logarithmic verifier time for Bulletproofs with transparent setup and reduced SRS size

Bulletproofs require linear verifier time and thus does not qualify as a zkSNARK. Its subsequent enhancements (Bünz et al. 2021; Cong et al. 2024) addressed this issue but required a one-shot trusted setup. In contrast, state-of-the-art zkSNARKs typically feature transparent setups. We meet this standard by transforming Bulletproofs into a zkSNARK with logarithmic verifier time under a transparent setup. Table 1 also demonstrates our improvements in verifier time compared to Bulletproofs. Specifically, for n^2-inner product arguments, our SRS size is $O(n)$, representing a square-root improvement over the $O(n^2)$ SRS size of the original Bulletproof.

Adding zero-knowledge with logarithmic additional cost

Efficiently adding zero-knowledge to Bulletproofs is non-trivial. Previously, the most notable achievement in the literature managed to achieve only almost zero-knowledge for Bulletproofs (Hoffmann et al. 2019). In contrast, our technique attains perfect special honest-verifier zero-knowledge (SHVZK), which is a more standard definition of zero-knowledge, with just a logarithmic increase in the prover's workload.

Scalable implementation

`DualMatrix` is the first zkSNARK implementation capable of handling matrix multiplication with up to 2^{30} non-zero integers. For $2^{15} \times 2^{15}$ dense matrices, `DualMatrix` achieved prover and verifier times of 150.84s and 0.56s, respectively. In the case of $2^{20} \times 2^{20}$ sparse matrices, prover and verifier times were 1, 384.45s and 0.67s, respectively. Notably, both general-purpose zkSNARKs and zkMatrix struggle with these tasks due to slow prover times and potential memory overflow.

Related work

Bulletproofs and zkMatrix

Bulletproofs (Bünz et al. 2018) and its subsequent enhancements and generalizations (Bünz et al. 2021; Cong et al. 2024; Gentry et al. 2022; Yuen et al. 2021), especially zkMatrix (Cong et al. 2024), build the foundation for our work. Bulletproof-style proofs generally achieve linear prover time. A recapitulation of Bulletproofs is available in Appendix B. We are also inspired by Lee (2021), which utilizes more pairings to achieve sub-linear prover time for matrix-related tasks.

Other specialized protocols for matrix multiplication

Groth (2009) proposed zero-knowledge proofs for linear algebra, yet did not address matrix multiplication. Because the matrix multiplication computation can be represented by a layered circuit, zero-knowledge protocols designed for such circuits, like LegoSNARK (Campanelli et al. 2019) and Libra (Xie et al. 2019), often use the matrix multiplication task to benchmark their performance. Interactive protocols, such as QuickSilver (Yang et al. 2021) and Mystique (Weng et al. 2021), attain $O(n^2)$ prover time at the expense of $O(n^2)$ communication.

Thaler's protocol (Thaler 2013) is a renowned specialized protocol for matrix multiplication. It transforms the matrix multiplication problem into a sumcheck problem on multilinear polynomials. This protocol has found applications in privacy-preserving machine learning tasks, such as Ghodsi and Gu (2017); Liu et al. (2021). However, for sparse matrices, the prover time of Thaler's protocol scales quasi-linearly, rather than strictly linearly, with the number of non-zero elements.

General-purpose zkSNARKs

General-purpose zkSNARKs typically convert every computation into an arithmetic circuit and then into an R1CS (Bootle et al. 2016) problem, with the number of constraints M being proportional to the circuit size or the count of multiplication gates. Early-generation zkSNARKs (Ames et al. 2017; Ben-Sasson et al. 2019; Bünz et al. 2020; Chiesa et al. 2020, 2020; Gabizon et al. 2019; Gennaro et al. 2013; Groth 2016; Maller 2019; Parno et al. 2016; Zhang et al. 2020) require $O(M \log M)$ field operations, largely due to the fast Fourier transform (FFT) operations involved. Recent FFT-free zkSNARKs, such as Libra (Xie et al. 2019), Quarks (Setty and Lee 2020), Gemini (Bootle et al. 2022), Spartan (Setty 2020), Brakedown (Golovnev et al. 2023), and Hyperplonk (Chen et al. 2023), achieve linear prover time relative to M. While these solutions are very efficient for discrete tasks like SHA-256 hash verification, they falter with matrix multiplication, as the number of constraints scales superlinearly with respect to N.

Advantages of DualMatrix

`DualMatrix` offers significant advantages over previous approaches: (1) It is the first strictly linear zkSNARK for sparse matrix multiplication; (2) it requires only $O(n)$ group operations and pairings in prover time; (3) it

transforms Bulletproofs into a zkSNARK protocol with a sublinear SRS under a transparent setup; (4) it adds zero-knowledge to Bulletproof-style proofs with only logarithmic additional workload; and (5) it scales effectively to large matrices.

Potential applications

Verifiable large language model

Verifiable machine learning enables a user to verify the correctness of the output of a neural network from an untrusted server without having to redo the entire computation. Given that neural networks often involve high-dimensional matrix multiplications, developing efficient zkSNAKRs for matrix multiplication becomes crucial. Initial efforts for verifiable machine learning typically employ non-SNARK interactive protocols (Ghodsi and Gu 2017; Weng et al. 2021; Xu et al. 2019). These works are often limited to small neural networks, such as ResNet-101 in Weng et al. (2021). As large language models become increasingly prevalent, it is essential to assess whether verifiable computation can scale to accommodate such models. For instance, the Llama2-70B model employs a word embedding dimension of 5, 120 and supports 32, 000 context window, leading to a $5,120 \times 32,000$ matrix (Touvron et al. 2023; Xu et al. 2024). `DualMatrix` offers a promising avenue for generating zkSNARK proofs for these large-scale matrix operations.

Privacy-preserving statistics on sensitive dataset

There is often a necessity to publish reliable statistical analyses on sensitive datasets like medical records while preserving their confidentiality.

Consider a health institute performing statistical analysis to evaluate the efficacy of a medical treatment. This analysis might take into account sensitive factors such as patient age and initial symptom severity. Linear regression (LR) is a fundamental statistical method that estimates the coefficients $\boldsymbol{\beta} \in \mathbb{R}^{m \times n}$ in the statistical equation $\vec{\boldsymbol{y}} = \boldsymbol{\beta}\vec{\boldsymbol{x}} + \vec{\boldsymbol{\epsilon}}$, where $\vec{\boldsymbol{y}} \in \mathbb{R}^m$ represents indicators of efficacy of the treatment, $\vec{\boldsymbol{x}} \in \mathbb{R}^n$ denotes the dependent factors, and $\vec{\boldsymbol{\epsilon}} \in \mathbb{R}^m$ indicates the statistical noise. l observation points of vectors $\vec{\boldsymbol{y}}$ and $\vec{\boldsymbol{x}}$ form the matrices $\boldsymbol{Y} \in \mathbb{R}^{l \times m}$ and $\boldsymbol{X} \in \mathbb{R}^{l \times n}$. Linear regression determines the optimal estimation of $\boldsymbol{\beta}$ as:

$$\boldsymbol{\beta} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1}(\boldsymbol{X}^\top \boldsymbol{Y}).$$

`DualMatrix` is capable of generating zkSNARK proofs to support the correctness of such LR estimations.

High-level idea

Background

Following the approach of zkMatrix Cong et al. (2024, Sect.5.1), matrix multiplication can be verified through the following equivalence:

$$\{\boldsymbol{c} = \boldsymbol{ab} \in \mathbb{Z}_p^{n \times n}\} \Leftrightarrow \{\forall \vec{\boldsymbol{l}} \in \mathbb{Z}_p^n, \vec{\boldsymbol{r}} \in \mathbb{Z}_p^n, (\vec{\boldsymbol{l}}^\top \boldsymbol{c}\vec{\boldsymbol{r}}) = (\vec{\boldsymbol{l}}^\top \boldsymbol{a})(\boldsymbol{b}\vec{\boldsymbol{r}})\}. \tag{3}$$

Here, $\vec{\boldsymbol{l}} \in \mathbb{Z}_p^n$ and $\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$ are random challenge vectors. We call $\vec{\boldsymbol{l}}^\top \boldsymbol{c}\vec{\boldsymbol{r}}$ the *scalar projection* of $\boldsymbol{c}$ onto the left row vector $\vec{\boldsymbol{l}}$ and the right column vector $\vec{\boldsymbol{r}}$. We define $\vec{\boldsymbol{l}}^\top \boldsymbol{a}$ the *left projection* of $\boldsymbol{a}$ onto the left row vector $\vec{\boldsymbol{l}}$, and $\boldsymbol{b}\vec{\boldsymbol{r}}$ the *right projection* of $\boldsymbol{b}$ onto the right column vector $\vec{\boldsymbol{r}}$. $(\vec{\boldsymbol{l}}^\top \boldsymbol{a})(\boldsymbol{b}\vec{\boldsymbol{r}})$ is the inner product of two vectors.

These inner products can be verified using Bulletproofs. However, directly invoking Bulletproofs is slow and does not scale well with large matrices. Therefore, we leverage sparsity and the rank-1 property of two-tier commitments to enhance the efficiency of Bulletproofs, both asymptotically and concretely.

Caching first-tier commitments

We can use either column-major or row-major order to compute the two-tier commitment. Using the column-major approach, Pedersen vector commitments for every column of $\boldsymbol{a}$ are computed using the base vector $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$, resulting in *first-tier commitments* $A_{G1}, \cdots, A_{Gn} \in \mathbb{G}_1$. Alternatively, the row-major approach commits to each row of $\boldsymbol{a}$ onto $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$, resulting in $A_{H1}, \cdots, A_{Hn} \in \mathbb{G}_2$. Both approaches lead to the identical two-tier commitment $C_a \in \mathbb{G}_T$:

$$C_a = \bigoplus\nolimits_{i=1}^{n} e(A_{Gi}, H_i) = \bigoplus\nolimits_{i=1}^{n} e(G_i, A_{Hi}) \in \mathbb{G}_T.$$

For efficiency, these first-tier commitments are cached and later reused in proving matrix multiplication.

Rank-1 property for accelerating Bulletproof prover

Two-tier commitment corresponds to Pedersen vector commitment using a rank-1 $\mathbb{G}_T$ matrix. Consider the two-tier commitment in $\mathbb{G}_T$ (Groth 2011) to a matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n \times n}$ onto base vectors $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$. This commitment is equal to the inner product between the vectorization of $\boldsymbol{a}$ and that of $\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^\top$, denoted as $\mathsf{vec}(\boldsymbol{a})$ and $\mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^\top)$, respectively. Specifically:

$$C_a := \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle = \mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^\top) \in \mathbb{G}_T,$$

where $\bullet$ represents the inner product, and $\otimes$ represents the tensor product, which is generalized from the integer

vector tensor product by using pairings as the multiplication operation between single group elements.

In the above, $\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top}$ is a rank-1 matrix in $\mathbb{G}_T$. When invoking a Bulletproof on a n^2-vector, this rank-1 property helps reduce $O(n^2)$ group operations to $O(n)$ group operations and pairings. For instance, in the first iteration of Bulletproofs, the prover computes:

$$\mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})_L \oplus [x_{(1)}^{-1} \cdot \mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})_R] \equiv (\vec{\boldsymbol{G}}_L \oplus x_{(1)}^{-1} \vec{\boldsymbol{G}}_R) \otimes \vec{\boldsymbol{H}}^{\top},$$

where $x_{(1)} \in \mathbb{Z}_p^*$ is the public-coin challenge in Bulletproofs. Here, we swap the order of the folding and the tensor product. While the left-hand-side (LHS) of above equation requires $O(n^2)$ group operations, the right-hand-side (RHS) only needs $O(n)$ group operations and pairings.

This rank-1 property also leads to a shorter SRS, as n elements in $\mathbb{G}_1$ and n elements in $\mathbb{G}_2$ are enough to derive n^2 elements in $\mathbb{G}_T$.

Leveraging sparsity

When dealing with sparse vectors, the prover's complexity in Bulletproofs is not strictly linear. In the most extreme case, when applying Bulletproofs to an n^2-vector with N non-zero elements, during the jth iteration of the $2\log n$ iterations, the vector might contain up to $\min(n^2/2^j, N)$ non-zero elements. As a result, the prover's time complexity is on the order of $O(N \log n)$.

To address this issue, we have the prover compute the transcript elements without explicitly folding $\mathsf{vec}(\boldsymbol{a})$. Specifically, consider the following equation during the first iteration of the Bulletproof protocol:

$$\mathsf{vec}(\boldsymbol{a})_L \bullet \mathsf{vec}(\vec{\boldsymbol{l}} \otimes \vec{\boldsymbol{r}})_R \equiv (\boldsymbol{a}\vec{\boldsymbol{r}})_L \bullet \vec{\boldsymbol{l}}_R,$$

where $\vec{\boldsymbol{l}} \in \mathbb{Z}_p^n$ and $\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$ are public field vectors. By computing the matrix projection $\boldsymbol{a}\vec{\boldsymbol{r}}$ before starting the Bulletproof iterations, we can effectively leverage sparsity. Computing this matrix projection requires only N field operations and is thus strictly linear. With the matrix projection computed beforehand, the RHS of the above equation necessitates only $O(n)$ operations. Therefore, the overall time complexity is reduced to $O(N + n)$.

Utilizing established security of Bulletproofs

For the security proof of the four subprotocols, we leverage the established security of Bulletproofs. Instead of demonstrating security from scratch, we apply a set of transformations to align our relations in $\mathbb{G}_T$ with those of Bulletproofs in $\mathbb{G}$. We use the following paradigm for security proofs of the subprotocols:

1. *Equivalence of relations*:
   relations in this paper
   $$\xleftarrow{\text{transformation}}$$
   relations of Bulletproofs;
2. *Equivalence of transcripts*:
   transcript elements in $\mathbb{G}_T$ generated by subprotocols of `DualMatrix`
   = transcript elements in $\mathbb{G}_T$ of Bulletproofs post-transformation
   $$\xleftarrow{\text{transformation}}$$
   transcript elements of Bulletproofs in $\mathbb{G}$ pre-transformation;
3. *Equivalence of verification checks*:
   verification checks in this paper
   $$\xleftarrow{\text{transformation}}$$
   verification checks of Bulletproofs.

Given that the assumptions of Bulletproofs in $\mathbb{G}_T$ are satisfied within our framework, and leveraging the fact that Bulletproofs are arguments of knowledge, the above paradigm ensures that subprotocols of `DualMatrix` also function as arguments of knowledge for the respective relations.

Notably, our subprotocols go beyond merely invoking Bulletproofs as a black box. While both our subprotocols and Bulletproofs compute the same transcript, ours are asymptotically faster than Bulletproofs, due to sparsity and the rank-1 structure inherent in the two-tier commitment scheme. The equivalence between our subprotocols and Bulletproofs are demonstrated line by line.

Logarithmic verifier time with transparent setup

Bulletproofs are characterized by linear verifier time. This inefficiency stems from the need to compute multi-scalar-multiplications like $V_G = \vec{\boldsymbol{\xi}}_G \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_1$ or $V_H = \vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_2$. Here, $\vec{\boldsymbol{\xi}}_G$ and $\vec{\boldsymbol{\xi}}_H$ are public field vectors defined by the challenges in Bulletproof. zkMatrix proposed that the prover assists the verifier in computing these group elements, and then provides proof that this value is correctly computed under a trusted setup. We adopt this core idea but facilitate a transparent setup.

To this end, we leverage a protocol that, given two committed group vectors $\vec{\boldsymbol{A}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{B}} \in \mathbb{G}_2^n$ committed to $A = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_T$ and $B = \vec{\boldsymbol{B}} \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_T$ with base vectors $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$ and $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$, respectively, provides an efficient inner-pairing-product argument with logarithmic

verifier time for $C = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}} \in \mathbb{G}_T$. This argument recursively reduces $\vec{\boldsymbol{A}}$ and $\vec{\boldsymbol{B}}$ into single group elements $\hat{A} \in \mathbb{G}_1$ and $\hat{B} \in \mathbb{G}_2$, and concludes by verifying a pairing equation $e(\hat{A}, \hat{B})$.

Given public group elements $V_G, V_H \in \mathbb{G}_T$, and public vectors $\vec{\boldsymbol{\xi}}_G, \vec{\boldsymbol{\xi}}_H \in \mathbb{Z}_p^n$, the prover can prove $V_G = \vec{\boldsymbol{\xi}}_G \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_1$ and $V_H = \vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_2$ by proving:

$$e(V_G, \hat{H}_0) = (\vec{\boldsymbol{\xi}}_G \hat{H}_0) \bullet \vec{\boldsymbol{G}}, \qquad e(\hat{G}_0, V_H) = (\vec{\boldsymbol{\xi}}_H \hat{G}_0) \bullet \vec{\boldsymbol{H}},$$

where $\hat{G}_0 \in \mathbb{G}_1$ and $\hat{H}_2 \in \mathbb{G}_2$ are public group elements.

Using folding techniques, these inner products can be iteratively reduced to verifying pairings:

$$C_G = e(\hat{A}_G, \hat{B}_G), \qquad C_H = e(\hat{A}_H, \hat{B}_H),$$

where $\hat{B}_G$ and $\hat{A}_H$ are obtained by reducing $\vec{\boldsymbol{\xi}}_G \hat{H}_0$ and $\vec{\boldsymbol{\xi}}_H \hat{G}_0$, and $\hat{A}_G$ and $\hat{B}_H$ are obtained by iteratively folding $\vec{\boldsymbol{G}}$ and $\vec{\boldsymbol{H}}$, respectively.

However, applying this protocol alone does not enhance efficiency, as calculating $\hat{A}_G$ and $\hat{B}_H$ from $\vec{\boldsymbol{G}}$ and $\vec{\boldsymbol{H}}$ remains computationally costly. We observe, however, that these group elements $\hat{A}_G$ and $\hat{B}_H$ can also be obtained from transcript elements of the inner-pairing-product arguments for two "auxiliary" inner products:

$$\Delta = \vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{H}}, \qquad O = \vec{\boldsymbol{O}} \bullet \vec{\boldsymbol{O}},$$

with base vectors $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$, $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$, and pre-computed $\Delta \in \mathbb{G}_T$. In these auxiliary protocols, the reduced base vectors that pass the verification check are uniquely determined. The reliability of the final transcript elements from these auxiliary inner-pairing-product arguments thus ensures the reliability of the reduced $\hat{A}_G$ and $\hat{B}_H$ from the original inner-pairing-product argument.

Adding zero-knowledge via hiding transcript

To equip `DualMatrix` with zero knowledge, we introduce the *hiding transcript* technique. This approach transforms each transcript element of the original protocol into a hiding commitment.

Consider a non-zero-knowledge protocol with transcript elements $P_1, P_2, \cdots, P_J \in \mathbb{G}_T$ subjected to a verification check:

$$\zeta_1 P_1 \oplus \zeta_2 P_2 \oplus \cdots \oplus \zeta_J P_J \stackrel{?}{=} O,$$

where $\zeta_1, \zeta_2, \cdots, \zeta_J \in \mathbb{Z}_p$ are public coefficients known to both parties.

To add zero-knowledge, the prover converts each P_j for $j \in [1, J]$ into a hiding commitment $\tilde{P}_j \in \mathbb{G}_T$. This is done using an additional base $\tilde{U} \in \mathbb{G}_T$ and random values $\tilde{p}_1, \tilde{p}_2, \cdots, \tilde{p}_J \stackrel{\$}{\leftarrow} \mathbb{Z}_p$:

$$\tilde{P}_1 \leftarrow P_1 \oplus \tilde{p}_1 \tilde{U}, \quad \tilde{P}_2 \leftarrow P_2 \oplus \tilde{p}_2 \tilde{U}, \quad \cdots, \quad \tilde{P}_J \leftarrow P_J \oplus \tilde{p}_J \tilde{U}.$$

Subsequently, the parties employ Schnorr's protocol to prove that $\zeta_1 \tilde{P}_1 \oplus \cdots \oplus \zeta_J \tilde{P}_J$ is a scalar multiplication of $\tilde{U}$. Given that the original protocol is an argument of knowledge, the new protocol constructed using the hiding transcript technique also qualifies as such.

It is noteworthy that by using this technique, the additional prover's workload is linear to the transcript size, i.e., on the order of $O(\log n)$.

Preliminaries

Notations

Pairing

Consider three cyclic groups $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$, each of prime order p. A pairing is defined as a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T$ such that:

$$e(aA, bB) = ab\, e(A, B),\ \forall\, a, b \in \mathbb{Z}_p,\ A \in \mathbb{G}_1, B \in \mathbb{G}_2,$$

where $\mathbb{Z}_p$ denotes the ring of integers modulo p, and $\mathbb{Z}_p^* := \mathbb{Z}_p \backslash \{0\}$. This paper utilizes the asymmetric (type-3) pairing, characterized by no efficiently computable homomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$.

Group and field elements

Group elements such as G, H, U, L, R, A, B are capitalized, while field elements like a, b, c, l, r, x, y are in lowercase. Commitments to $a, b, c, d \in \mathbb{Z}_p$ are represented as $C_a, C_b, C_c, C_d \in \mathbb{G}_T$, respectively. Blinding factors for these commitments, $\tilde{a}, \tilde{b}, \tilde{c}, \tilde{d} \stackrel{\$}{\leftarrow} \mathbb{Z}_p^*$, are denoted with the tilde superscript. We adopt the additive notation for group operations, using $\oplus$ for group addition and O for the identity element in $\mathbb{G}_T$. Summation of indexed group elements is represented as $\bigoplus_{i=start}^{end}$. *Exponentiation* refers to scalar multiplication of a group element, with the group element termed as the base of the exponentiation. We commit to an $n \times n$ matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n \times n}$ using base vectors $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$.

Group and field matrices

Matrices and vectors are represented with bold letters, with $\vec{\cdot}$ indicating column vectors and $\vec{\cdot}^{\top}$ for row vectors. Uppercase bold letters denote matrices or vectors of group elements (group matrices or vectors), e.g., $\boldsymbol{A}, \boldsymbol{B}, \vec{\boldsymbol{A}}, \vec{\boldsymbol{B}}$, while lowercase bold letters represent matrices or vectors of field elements (field matrices or vectors), e.g., $\boldsymbol{a}, \boldsymbol{b}, \vec{\boldsymbol{a}}, \vec{\boldsymbol{b}}$.

Vectorization is the process of transforming a matrix into a vector, adopting the row-major order by default. For instance, given a matrix $\boldsymbol{a} = \{a_{ij}\} \in \mathbb{Z}_p^{m\times n}$, its vectorization is expressed as: $\text{vec}(\boldsymbol{a}) := (a_{11}, a_{12}, \cdots, a_{1n}; \cdots; a_{m1}, a_{m2}, \cdots, a_{mn})$.

Given a vector $\vec{\boldsymbol{a}} \in \mathbb{Z}_p^n$, we define its left half as $\vec{\boldsymbol{a}}_L := (a_1, \cdots, a_{n/2})$ and its right half as $\vec{\boldsymbol{a}}_R := (a_{n/2+1}, \cdots, a_n)$. $(\vec{\boldsymbol{a}}_L || \vec{\boldsymbol{a}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{a}}$ represents this partitioning procedure. For a matrix $\boldsymbol{a} \in \mathbb{Z}_p^{m\times n}$, it can be divided into left and right halves denoted as $\boldsymbol{a}_L$ and $\boldsymbol{a}_R$, or into upper and lower halves represented by $\boldsymbol{a}_U$ and $\boldsymbol{a}_D$. Such partitioning applies to both field and group vectors. Following the programming language syntax, we use the notation [start − 1 : end] for slicing a vector. For instance: $\vec{\boldsymbol{a}}_{[0:n]} := (a_1, a_2, \cdots, a_n)$.

Vectorization, partition, and slicing are applicable to group matrices as well.

Matrix projection

We call the row vector $\vec{\boldsymbol{l}}^{\top}\boldsymbol{a} \in \mathbb{Z}_p^n$ the left projection of the matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n\times n}$ onto the vector $\vec{\boldsymbol{l}} \in \mathbb{Z}_p^n$. We call the column vector $\boldsymbol{a}\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$ the right projection of the matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n\times n}$ onto the vector $\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$. We call the scalar $\vec{\boldsymbol{l}}^{\top}\boldsymbol{a}\vec{\boldsymbol{r}} \in \mathbb{Z}_p$ the scalar projection of the matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n\times n}$ onto the vectors $\vec{\boldsymbol{l}} \in \mathbb{Z}_p^n$ and $\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$.

Miscellaneous

The algorithm $\mathsf{Setup}(1^\lambda)$ generates public parameters based on the security parameter λ. We denote the prover, the verifier, the extractor, and the adversary as $\mathcal{P}$, $\mathcal{V}$, $\mathcal{K}$, and $\mathcal{A}$ respectively, with $(\mathcal{P} \rightleftharpoons \mathcal{V})$ representing the interaction between the prover and the verifier. We represent a relation as $\mathcal{R}$ =\{ Public Input : Witness | Relation \}, and a language as $\mathcal{L}$ =\{ Input | Statement(Input)\}. Inputs of algorithms are separated using commas ', ' or semicolons '; ', e.g., (Commitments; Bases: Witnesses). '$\Leftarrow$' and '$\Rightarrow$' denote the transformation between notations in $\mathbb{G}_T$ of `DualMatrix` and those in $\mathbb{G}$ of Bulletproofs. Notations marked with #[bullet] and #[semiBullet] relate to the Bulletproof and semi-Bulletproof protocols, respectively, which can be found in Appendix B, where the symbols differ from those in `DualMatrix`.

Group matrix algebra

Group matrix multiplication

We define matrix multiplication and other (inner/Kronecker/Hadamard) products for group matrices by analogy with standard integer matrix multiplication and products: specifically, each scalar multiplication between integers is replaced by a bilinear pairing between a group element from $\mathbb{G}_1$ and one from $\mathbb{G}_2$. This defines a new product between a matrix $\boldsymbol{A}$ over $\mathbb{G}_1$ and a matrix $\boldsymbol{B}$ over $\mathbb{G}_2$, resulting in a matrix $\boldsymbol{AB}$ over the target group $\mathbb{G}_T$. Consider $\boldsymbol{A} = \{A_{ik}\} \in \mathbb{G}_1^{m\times l}$ and $\boldsymbol{B} = \{B_{kj}\} \in \mathbb{G}_2^{l\times n}$, their matrix multiplication $\boldsymbol{AB}$ results in an $m \times n$ matrix with the (i, j) entry defined as $\bigoplus_{k=1}^{l} e(A_{ik}, B_{kj})$. Multiplying a group element $A \in \mathbb{G}_1$ by a group matrix $\boldsymbol{B} = \{B_{kj}\} \in \mathbb{G}_1^{l\times n}$ results in a group matrix with the (k, j) entry defined as $e(A, B_{kj})$. When $\boldsymbol{B} = \{B_{ik}\} \in \mathbb{G}_2^{m\times l}$ and $\boldsymbol{A} = \{A_{kj}\} \in \mathbb{G}_1^{l\times n}$, the matrix multiplication $\boldsymbol{BA}$ is similarly defined. Matrix multiplication between field matrices and group matrices are defined similarly by replacing each scalar product with scalar multiplication between a field element and a group element. $\mathbb{G}_T$ group matrices can only be multiplied by field matrices. Therefore, in any serial composition of matrix multiplications, at most one matrix from $\mathbb{G}_1$ and one from $\mathbb{G}_2$ may appear, since their product lies in $\mathbb{G}_T$, where further group matrix multiplication is undefined.

The inner product, represented by $\bullet$, is a special case of matrix multiplication. The inner product of two group vectors $\vec{\boldsymbol{A}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{B}} \in \mathbb{G}_2^n$ is defined as:

$$\vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}} := \vec{\boldsymbol{A}}^{\top}\vec{\boldsymbol{B}} := \vec{\boldsymbol{B}}^{\top}\vec{\boldsymbol{A}} := \vec{\boldsymbol{B}} \bullet \vec{\boldsymbol{A}} := \bigoplus_{i=1}^{n} e(A_i, B_i).$$

The Kronecker product, $\otimes$, between a $\mathbb{G}_1$ matrix $\boldsymbol{A} = \{A_{i_a j_a}\} \in \mathbb{G}_1^{m_a\times n_a}$ and a $\mathbb{G}_2$ matrix $\boldsymbol{B} = \{B_{i_b j_b}\} \in \mathbb{G}_2^{m_b\times n_b}$ is a $(m_a m_b) \times (n_a n_b)$ $\mathbb{G}_T$ matrix:

$$\boldsymbol{A} \otimes \boldsymbol{B} := \{e(A_{i_a i_b}, B_{j_a j_b})\},$$
$$\text{where } i_a = i/m_b, j_a = j/n_b, i_b = i \bmod m_b, j_b = j \bmod n_b.$$

The Kronecker product of vectors is also termed *tensor product*. As a special case, the tensor product between a column vector $\vec{\boldsymbol{G}} \in \mathbb{G}_1^m$ and a row vector $\vec{\boldsymbol{H}}^{\top} \in \mathbb{G}_2^n$ is:

$$\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top} := \{e(G_i, H_j)\} \in \mathbb{G}_T^{m\times n} = \vec{\boldsymbol{G}}\vec{\boldsymbol{H}}^{\top}.$$

Similarly, we define:

$$\vec{\boldsymbol{H}} \otimes \vec{\boldsymbol{G}}^{T} := \{e(G_j, H_i)\} \in \mathbb{G}_T^{n\times m} = \vec{\boldsymbol{H}}\vec{\boldsymbol{G}}^{T}.$$

Dirac notation

Adopting the Dirac notation from quantum mechanics, we define the projection of a matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n\times n}$ onto base vectors $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$.

The *bra* operator $\langle \vec{\boldsymbol{G}} | \cdot |$ computes the left projection of $\boldsymbol{a}$ onto $\vec{\boldsymbol{G}}$ as follows:

$$\langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,| := \vec{\boldsymbol{G}}^{\top}\boldsymbol{a}.$$

The *ket* operator $|\cdot|\vec{\boldsymbol{H}}\rangle$ computes the right projection of $\boldsymbol{a}$ onto $\vec{\boldsymbol{H}}$ as follows:

$$|\boldsymbol{a}|\vec{\boldsymbol{H}}\rangle := \boldsymbol{a}\vec{\boldsymbol{H}}.$$

The *braket* operator $\langle\vec{\boldsymbol{G}}|\cdot|\vec{\boldsymbol{H}}\rangle$ computes the scalar projection in $\mathbb{G}_T$ as follows:

$$\langle\vec{\boldsymbol{G}}|\boldsymbol{a}|\vec{\boldsymbol{H}}\rangle := \vec{\boldsymbol{G}}^{\top}\boldsymbol{a}\vec{\boldsymbol{H}}, \quad \langle\vec{\boldsymbol{H}}|\boldsymbol{a}|\vec{\boldsymbol{G}}\rangle := \vec{\boldsymbol{H}}^{\top}\boldsymbol{a}\vec{\boldsymbol{G}}.$$

For scalar bases $G \in \mathbb{G}_1$ or $H \in \mathbb{G}_2$, the braket operator is similarly defined as follows:

$$\langle G|a|H\rangle := \langle H|a|G\rangle := a\,e(G,H).$$

Properties of group matrix products

In our security proofs, we utilize a collection of properties from group matrix algebra. Among these, the following are the most important. A comprehensive list of the properties used in this paper can be found in Appendix A, which also includes properties of field matrices.

Bilinear Property. For single group elements, the bilinear map is the pairing operation that satisfies:

$$e(aA,bB) = e(bA,aB) = (ab)\,e(A,B),$$
$$\forall a,b \in \mathbb{Z}_p,\ A \in \mathbb{G}_1,\ B \in \mathbb{G}_2.$$

When extended to group matrices, A and B evolve into group matrices $\boldsymbol{A} \in \mathbb{G}_1^{m\times l}$ and $\boldsymbol{B} \in \mathbb{G}_2^{l\times n}$, respectively, and the pairing operation is extended to the group matrix multiplication defined previously. An extension of the bilinear property suggests:

$$(a\boldsymbol{A})(b\boldsymbol{B}) = (b\boldsymbol{A})(a\boldsymbol{B}) = (ab)(\boldsymbol{A}\boldsymbol{B}).$$

However, the generalized bilinear property encompasses more than this simple relation. When $\boldsymbol{a}$ and $\boldsymbol{b}$ are also extended to matrices $\boldsymbol{a} \in \mathbb{Z}_p^{m_a\times l_a}$ and $\boldsymbol{b} \in \mathbb{Z}_p^{l_b\times n_b}$, we observe the following associative property of matrix multiplication, assuming dimension compatibility:

$$(\boldsymbol{a}\boldsymbol{A})(\boldsymbol{B}\boldsymbol{b}) = \boldsymbol{a}(\boldsymbol{A}\boldsymbol{B})\boldsymbol{b}, \quad (\boldsymbol{A}\boldsymbol{a})(\boldsymbol{b}\boldsymbol{B}) = \boldsymbol{A}(\boldsymbol{a}\boldsymbol{b})\boldsymbol{B}.$$

The validity of this associative property stems from the bilinear property of pairings. Thus, it can be seen as the bilinear property within the group matrix domain.

Duality of Dirac Notation. The following duality of Dirac notation provides symmetry for our group algebra:

$$\langle\vec{\boldsymbol{G}}|\boldsymbol{a}|\vec{\boldsymbol{H}}\rangle = \langle\vec{\boldsymbol{H}}|\boldsymbol{a}^{\top}|\vec{\boldsymbol{G}}\rangle = \mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{\boldsymbol{G}}\otimes\vec{\boldsymbol{H}}). \tag{4}$$

Tensor Reducing. Tensor products reduce high-dimensional inner products to lower-dimensional ones, considerably decreasing computational complexity, by leveraging the equivalence:

$$\mathsf{vec}(\vec{\boldsymbol{a}}\otimes\vec{\boldsymbol{b}}^{\top}) \bullet \mathsf{vec}(\vec{\boldsymbol{G}}\otimes\vec{\boldsymbol{H}}^{\top}) = e(\vec{\boldsymbol{a}}\bullet\vec{\boldsymbol{G}}, \vec{\boldsymbol{b}}\bullet\vec{\boldsymbol{H}}).$$

Cryptographic definitions and assumptions

Definitions

Definition 1 (*Commitment*) A non-interactive commitment scheme consists of a pair of probabilistic-polynomial-time (PPT) algorithms $(\mathsf{Setup}, \mathsf{Com})$. The setup algorithm $pp \leftarrow \mathsf{Setup}(1^{\lambda})$ generates public parameters pp for the scheme, provided with the security parameter λ. The commitment algorithm $\mathsf{Com}_{\mathsf{pp}}$ defines a function $\mathbb{M}_{\mathsf{pp}} \times \mathbb{R}_{\mathsf{pp}} \mapsto \mathbb{C}_{\mathsf{pp}}$ for the message space $\mathbb{M}_{\mathsf{pp}}$, the randomness space $\mathbb{R}_{\mathsf{pp}}$, and the commitment space $\mathbb{C}_{\mathsf{pp}}$ determined by pp. For a message $a \in \mathbb{M}_{\mathsf{pp}}$, the algorithm draws $\tilde{a} \xleftarrow{\$} \mathbb{R}_{\mathsf{pp}}$ uniformly at random, and computes the commitment $C_a \leftarrow \mathsf{Com}_{\mathsf{pp}}(a;\tilde{a})$.

For ease of notation, we write $\mathsf{Com} = \mathsf{Com}_{\mathsf{pp}}$ in the following definitions.

Definition 2 (*Homomorphic Commitment*) A homomorphic commitment scheme is a non-interactive commitment scheme such that $\mathbb{M}_{\mathsf{pp}}$, $\mathbb{R}_{\mathsf{pp}}$, and $\mathbb{C}_{\mathsf{pp}}$ are all abelian groups, and for all $a_1, a_2 \in \mathbb{M}_{\mathsf{pp}}, \tilde{a}_1, \tilde{a}_2 \in \mathbb{R}_{\mathsf{pp}}$, we have:

$$\mathsf{Com}(a_1;\tilde{a}_1) + \mathsf{Com}(a_2;\tilde{a}_2) = \mathsf{Com}(a_1+a_2;\tilde{a}_1+\tilde{a}_2).$$

Definition 3 (*Hiding Commitment*) A commitment scheme is said to be hiding if there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all PPT adversaries $\mathcal{A}$:

$$\left|\Pr\left(i'=i \;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathsf{Setup}(1^{\lambda});\\ (a_0,a_1)\in\mathbb{M}_{\mathsf{pp}}, i\xleftarrow{\$}\{0,1\}, \tilde{a}\xleftarrow{\$}\mathbb{R}_{\mathsf{pp}},\\ C_a\leftarrow\mathsf{Com}(a_i;\tilde{a});\ i'\leftarrow\mathcal{A}(\mathsf{pp},C_a)\end{array}\right)-\frac{1}{2}\right| \le \mathsf{negl}(\lambda),$$

where the probability is over $i, \tilde{a}, \mathsf{Setup}$ and $\mathcal{A}$. If $\mathsf{negl}(\lambda) = 0$ then we say the scheme is perfectly hiding.

Definition 4 (*Binding Commitment*) A commitment scheme is said to be binding if for any PPT adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\lambda)$ such that:

$$\Pr\left(\begin{array}{l}a_0\neq a_1\\ \wedge\,\mathsf{Com}(a_0;\tilde{a}_0)=\mathsf{Com}(a_1;\tilde{a}_1)\end{array}\;\middle|\;\begin{array}{l}\mathsf{pp}\leftarrow\mathsf{Setup}(1^{\lambda});\\ a_0,\tilde{a}_0,a_1,\tilde{a}_1\leftarrow\mathcal{A}(\mathsf{pp})\end{array}\right)\le\mathsf{negl}(\lambda),$$

where the probability is over Setup and $\mathcal{A}$. If $\mathsf{negl}(\lambda) = 0$, then we say the scheme is perfectly binding.

Definition 5 (Pedersen Commitment) Let $\mathbb{M}_{\mathsf{pp}} = \mathbb{R}_{\mathsf{pp}} = \mathbb{Z}_p$ and $\mathbb{C}_{\mathsf{pp}} = \mathbb{G}$ of order p. The Pedersen commitment of a message $a \in \mathbb{Z}_p$ and a randomness $\tilde{a} \in \mathbb{Z}_p$ is:

$$\mathsf{Setup} : G, \tilde{G} \xleftarrow{\$} \mathbb{G}, \quad \mathsf{Com}(a; \tilde{a}) : C_a \leftarrow aG \oplus \tilde{a}\tilde{G}.$$

We often set $\tilde{a} = 0$.

Definition 6 (*Pedersen Vector Commitment*) Let the message space $\mathbb{M}_{\mathsf{pp}} = \mathbb{Z}_p^n$, the randomness space $\mathbb{R}_{\mathsf{pp}} = \mathbb{Z}_p$, and the commitment space $\mathbb{C}_{\mathsf{pp}} = \mathbb{G}$ of order p. The Pedersen vector commitment of a vector $\vec{\boldsymbol{a}} \in \mathbb{Z}_p^n$ and a randomness $\tilde{a} \in \mathbb{Z}_p$ is:

$$\mathsf{Setup} : \vec{\boldsymbol{G}} \xleftarrow{\$} \mathbb{G}^n, \tilde{G} \xleftarrow{\$} \mathbb{G}, \quad \mathsf{Com}(\vec{\boldsymbol{a}}; \tilde{a}) : C_a \leftarrow (\vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}}) \oplus \tilde{a}\tilde{G}.$$

Definition 7 (*Argument*) The triple $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ is called an argument for a language $\mathcal{L}$ if it satisfies perfect completeness and computational soundness as defined below.

Definition 8 (*Argument of Knowledge*) The triple $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ is called an argument of knowledge for the relation $\mathcal{R}$ if it satisfies perfect completeness and knowledge soundness as defined below.

Definition 9 (*Perfect Completeness*) $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ has perfect completeness if for any non-uniform PPT adversary $\mathcal{A}$:

$$\Pr\left(\begin{array}{l} (\mathsf{srs}, u : w) \notin \mathcal{R} \\ \vee\, \mathcal{V}(\mathsf{srs}, u; \mathsf{tr}; \mathsf{rand}) = \mathsf{TRUE} \end{array} \middle| \begin{array}{l} \mathsf{srs} \leftarrow \mathsf{Setup}(1^\lambda); (u, w) \leftarrow \mathcal{A}(\mathsf{srs}); \\ \mathsf{tr} \leftarrow (\mathcal{P} \rightleftharpoons \mathcal{V})(\mathsf{srs}, u : w) \end{array} \right) = 1,$$

where $\mathsf{srs}, u, w, \mathsf{tr}$, and rand stand for the SRS, the statement, the witness, the transcript, and the randomness, respectively.

Definition 10 (*Computational Soundness* Canetti et al. (2018)) An interactive protocol $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ for the language $\mathcal{L}$ is sound with a soundness error $\kappa^{\mathsf{s}}(|u|) : \mathbb{N} \mapsto [0, 1]$ if for any PPT interactive $\mathcal{P}^*$, the probability that $\mathcal{P}^*$ outputs an accepting transcript is negligible:

$$\Pr\left(\mathcal{V}(\mathsf{pp}, u; \pi) = \mathsf{TRUE} \,\middle|\, u \notin \mathcal{L}; \pi \leftarrow (\mathcal{P}^* \rightleftharpoons \mathcal{V})(\mathsf{pp}, u) \right) \leq \kappa^{\mathsf{s}}(|u|).$$

Definition 11 (*Knowledge Soundness* Attema et al. (2021)) $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}$ is knowledge sound with error $\kappa_\lambda(|u|) : \mathbb{N} \mapsto [0, 1]$ if there exists an algorithm $\mathcal{K}$, called a knowledge extractor, with the following properties. Given input u and black-box oracle access to a (potentially dishonest) prover $\mathcal{P}^*$, the extractor $\mathcal{K}$ runs in an expected number of steps that is polynomial in $|u|$ (counting queries to $\mathcal{P}$ as a single step) and outputs a witness $(\mathsf{srs}, u : w) \in \mathcal{R}$ with probability:

$$\Pr((\mathsf{srs}, u : \mathcal{K}(u)) \in \mathcal{R}) \geq \frac{p(\mathcal{P}^*, u) - \kappa_\lambda(|u|)}{\mathsf{poly}(|u|)},$$

where $p(\mathcal{P}^*, u)$ is the probability that $\mathcal{P}^*$ generates an accepted transcript.

Previous studies have demonstrated that the witness-extended emulation property of Bulletproofs (Bünz et al. 2018) implies knowledge soundness (Attema et al. 2021, 2022; Groth 2004).

Definition 12 (*Public-Coin*) A $(2\mu + 1)$−move public-coin interactive protocol $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ is public-coin if in the $(2j)$-th move, $\mathcal{V}$ sends the j-th challenge $x_j \xleftarrow{\$} \mathbb{X}_j$. All challenges $x_1, \cdots, x_\mu$ sent from the verifier to the prover are public information and chosen uniformly at random and independently of the prover's messages.

Definition 13 (*Adaptive Fiat-Shamir Transformation*) The adaptive Fiat-Shamir transformation on a $(2\mu + 1)$−move public-coin interactive protocol: $\mathsf{FS}(\mathsf{Setup}, \mathcal{P}, \mathcal{V}) \mapsto (\mathsf{Setup}, \mathcal{P}_{\mathsf{FS}}, \mathcal{V}_{\mathsf{FS}})$ is a two-round argument of knowledge where $\mathcal{P}_{\mathsf{FS}}(\mathsf{srs}, u, w)$ runs $\mathcal{P}(\mathsf{srs}, u, w)$ but instead of asking the verifier for the challenge x_i on message a_i, the challenges are computed as:

$$x_j = \mathsf{Hash}_i(u, a_1, \cdots, a_{j-1}), \quad j = [1, \mu].$$

the output is then the transcript $\mathsf{tr} = (a_1, \cdots, a_{\mu-1}, a_\mu)$. On input of a statement u and a proof $\mathsf{tr} = (a_1, \cdots, a_{\mu-1}, a_\mu)$, $\mathcal{V}_{\mathsf{FS}}(\mathsf{srs}, u, \mathsf{tr})$ accepts if, for $x_j, j = [1, \mu]$ as above, $\mathcal{V}$ accepts the transcript $\mathsf{tr} = (a_1, \cdots, a_{\mu-1}, a_\mu)$ on input u.

Definition 14 (*Perfect Zero-Knowledge* Groth (2016)) A non-interactive argument of knowledge $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ is perfect zero-knowledge if there exists a PPT simulator $\mathcal{S}$

and a trapdoor trap such that for all stateful distinguishers $\mathcal{D}$ the following two probabilities are equal:

$$\Pr\left(\mathcal{D}(\mathsf{srs}, u; \mathsf{tr}; \mathsf{trap}) = \mathsf{TRUE} \;\middle|\; \begin{array}{l}(\mathsf{srs}, \mathsf{trap}) \leftarrow \mathsf{Setup}(1^\lambda);\\ \mathsf{tr} \leftarrow (\mathcal{P} \rightleftharpoons \mathcal{V})(\mathsf{srs}, u : w)\end{array}\right)$$
$$= \Pr\left(\mathcal{D}(\mathsf{srs}, u; \mathsf{tr}^*; \mathsf{trap}) = \mathsf{TRUE} \;\middle|\; \begin{array}{l}(\mathsf{srs}, \mathsf{trap}) \leftarrow \mathsf{Setup}(1^\lambda);\\ \mathsf{tr}^* \leftarrow \mathcal{S}(\mathsf{srs}, u; \mathsf{trap})\end{array}\right).$$

Definition 15 (*Perfect Special Honest-Verifier Zero-Knowledge*) An argument of knowledge $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ has perfect special honest-verifier zero-knowledge (SHVZK) if there exists a PPT simulator $\mathcal{S}$ such that for all pairs of interactive adversaries $\mathcal{A}$ and $\mathcal{D}$, the following two probabilities are equal:

$$\Pr\left(\begin{array}{l}(\mathsf{srs}, u, w) \in \mathcal{R}\\ \wedge\, \mathcal{D}(\mathsf{srs}, u; \mathsf{tr}; \mathsf{rand}) = \mathsf{TRUE}\end{array} \;\middle|\; \begin{array}{l}\mathsf{srs} \leftarrow \mathsf{Setup}(1^\lambda);\ (u, w, \mathsf{rand}) \leftarrow \mathcal{A}(\mathsf{srs});\\ \mathsf{tr} \leftarrow (\mathcal{P} \rightleftharpoons \mathcal{V})(\mathsf{srs}, u, w)\end{array}\right)$$
$$= \Pr\left(\begin{array}{l}(\mathsf{srs}, u, w) \in \mathcal{R}\\ \wedge\, \mathcal{D}(\mathsf{srs}, u; \mathsf{tr}^*; \mathsf{rand}) = \mathsf{TRUE}\end{array} \;\middle|\; \begin{array}{l}\mathsf{srs} \leftarrow \mathsf{Setup}(1^\lambda);\ (u, w, \mathsf{rand}) \leftarrow \mathcal{A}(\mathsf{srs});\\ \mathsf{tr}^* \leftarrow \mathcal{S}(\mathsf{srs}, u, \mathsf{rand})\end{array}\right),$$

where rand is the public-coin randomness used by the verifier.

Assumptions

Assumption 1 (*DDH*) The decisional Diffie-Hellman (DDH) problem in a group $\mathbb{G}$ of prime order p with generator $\hat{G}$ is defined as: given $(\hat{G}, a\hat{G}, b\hat{G}, c\hat{G})$, deciding whether $c = ab$. The DDH assumption states that solving the DDH problem is difficult for any PPT adversary $\mathcal{A}$.

For bilinear groups with generators $\hat{G} \in \mathbb{G}_1$ and $\hat{H} \in \mathbb{G}_2$, the following security assumption is standard.

Assumption 2 (*SXDH* Lee (2021)) The symmetric external Diffie-Hellman (SXDH) assumption holds for a bilinear group $(e, \mathbb{Z}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, \hat{G}, \hat{H})$ if the DDH assumption holds in both $(\mathbb{Z}_p, \mathbb{G}_1, \hat{G})$ and $(\mathbb{Z}_p, \mathbb{G}_2, \hat{H})$.

A DDH instance in $\mathbb{G}_1$ can be mapped to one in $\mathbb{G}_T$ by the mapping $G \mapsto e(G, \hat{H})$, so SXDH implies that DDH holds in $\mathbb{G}_T$. Specifically, the following DLOG assumption holds for $\mathbb{G}_T$:

Assumption 3 (*DLOG*) The discrete logarithm (DLOG) problem for a group $\mathbb{G}$ of prime order p is defined as: given $G_1, G_2 \in \mathbb{G}$, find $a \in \mathbb{Z}_p$ such that $G_1 = aG_2$.

Assumption 4 (*Discrete Logarithm Relation* Bünz et al. (2018)) The discrete logarithm relation assumption states that for all PPT adversaries $\mathcal{A}$ and for all $n = \mathsf{poly}(\lambda) \geq 2$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that:

$$\Pr\left(\begin{array}{l}\exists a_i \neq 0\\ \wedge \bigoplus_{i=1}^n a_i G_i = O\end{array} \;\middle|\; \begin{array}{l}(p, \mathbb{G}) \leftarrow \mathsf{Setup}(1^\lambda), G_1, \cdots, G_n \xleftarrow{\$} \mathbb{G};\\ a_1, \cdots, a_n \in \mathbb{Z}_p \leftarrow \mathcal{A}(p, \mathbb{G}, G_1, \cdots, G_n)\end{array}\right) \leq \mathsf{negl}(\lambda).$$

$\bigoplus_{i=1}^n a_i G_i = O$ is called a non-trivial discrete logarithm relation among $G_1, \cdots, G_n$. The discrete logarithm relation assumption is the only cryptographic assumption underlying Bulletproofs.

The following lemma is an implication of the SXDH assumption.

Lemma 1 (Absence of Non-Trivial Inner-Pairing Product Abe et al. (2010)) *For* $(e, \mathbb{Z}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, \hat{G}, \hat{H})$ *as above and* $n = \mathsf{poly}(\lambda)$, *given* $\vec{\boldsymbol{A}} \xleftarrow{\$} \mathbb{G}_1^n$, *no PPT adversary* $\mathcal{A}$ *can compute a non-trivial* $\vec{\boldsymbol{B}} \in \mathbb{G}_2^n$ *such that:* $\vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}} = O$. *Similarly, given* $\vec{\boldsymbol{B}} \xleftarrow{\$} \mathbb{G}_2^n$, *no PPT adversary can compute a non-trivial* $\vec{\boldsymbol{A}} \in \mathbb{G}_1^n$ *such that:* $\vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}} = O$.

This lemma corresponds to the binding property of the AFGHO commitment scheme (Abe et al. 2010), which commits a group vector $\vec{\boldsymbol{A}} \in \mathbb{G}_1/\mathbb{G}_2$ to $C_a = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{H}} / \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_T$. The security proof for this binding property is provided in the AFGHO paper (Abe et al. 2010).

Matrix commitment

Setup

The setup algorithm randomly selects $\hat{G}_0, \hat{H}_0, \hat{G}_{-1}$, along with base vectors $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$. It then computes:

$U = \hat{G}_0 \otimes \hat{H}_0, \tilde{U} = \hat{G}_{-1} \otimes \hat{H}_0$. The SRS is defined as follows:

$$\hat{G}_0, \hat{H}_0, U, \tilde{U}, \vec{\boldsymbol{G}}, \vec{\boldsymbol{H}}.$$

Absence of non-trivial logarithm relation

Lemma 2 (Absence of Non-Trivial Logarithm Relation in $\mathbb{G}_T$) *Under the SXDH assumption, no non-trivial logarithmic relation exists among the following elements of* $\mathbb{G}_T$:

$$\{\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}\} \cup \{\vec{\boldsymbol{G}} \otimes \vec{H}_0\} \cup \{\hat{G}_0 \otimes \vec{\boldsymbol{H}}\} \cup \{U\} \cup \{\tilde{U}\} \subset \mathbb{G}_T. \tag{5}$$

The proof of this lemma is provided in Appendix C.1.

Two-tier commitment

The two-tier commitment (Groth 2011) to a matrix $\boldsymbol{a} \in \mathbb{Z}_p^{m \times n}$ is computed as follows:

- **First-tier commitment:** Apply the Pedersen vector commitment to each column or row of $\boldsymbol{a}$ using the base vector $\vec{\boldsymbol{G}} \in \mathbb{G}_1^m$ or $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$. This results in $\vec{\boldsymbol{A}}_G \in \mathbb{G}_1^n$ or $\vec{\boldsymbol{A}}_H \in \mathbb{G}_2^m$, computed as:

$$\vec{\boldsymbol{A}}_G = \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_1 \quad \text{or} \quad \vec{\boldsymbol{A}}_H = \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_2.$$

- **Second-tier commitment:** Commit to $\vec{\boldsymbol{A}}_G$ or $\vec{\boldsymbol{A}}_H$ using AFGHO commitment (Abe et al. 2010), resulting in $C_a \in \mathbb{G}_T$:

$$C_a = \vec{\boldsymbol{A}}_G \bullet \vec{\boldsymbol{H}} = \vec{\boldsymbol{A}}_H \bullet \vec{\boldsymbol{G}} = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle.$$

  We can add a hiding factor $\tilde{a} \in \mathbb{Z}_p$ with the base $\tilde{U} \in \mathbb{G}_T$ into the AFGHO commitment (Abe et al. 2010) as follows:

$$C_a = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle \oplus \tilde{a}\tilde{U}.$$

Proposition 1 (Two-Tier Commitment as Pedersen Vector Commitment in $\mathbb{G}_T$) *The two-tier commitment* $C_a \in \mathbb{G}_T$ *equals the Pedersen vector commitment to* $\mathsf{vec}(\boldsymbol{a})$ *in* $\mathbb{G}_T$ *using the base vector* $\mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})$ *and* $\tilde{U}$.

Proof

Utilizing the property in Eq. (4), we derive:

$$C_a := \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle \oplus \tilde{a}\tilde{U} = (\,\mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})\,) \oplus \tilde{a}\tilde{U},$$

matching the definition of Pedersen vector commitment (Definition 6). □

Corollary 1 (Binding, Hiding, and Homomorphic Commitment) *Given the SXDH assumption, the two-tier commitment is a homomorphic commitment, both binding and hiding.*

Proof

According to Lemma 2, there is no non-trivial logarithm relation between the elements of $\mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})$ *and* $\tilde{U}$. *Because the Pedersen vector commitment is homomorphic, binding, and hiding, the two-tier commitment inherits these properties.* □

Caching first-tier commitment

In the commitment phase, the prover can cache the first-tier commitments $\vec{\boldsymbol{A}}_G \in \mathbb{G}_1^n$ or $\vec{\boldsymbol{A}}_H \in \mathbb{G}_2^m$. These caches can be reused in the prover phase of matrix multiplication.

DualMatrix

We utilize the following equivalence to verify $n \times n$ matrix multiplication:

$$\{\boldsymbol{c} = \boldsymbol{ab}\} \iff \{(\vec{\boldsymbol{l}}^{\top}\boldsymbol{c}\vec{\boldsymbol{r}}) = (\vec{\boldsymbol{l}}^{\top}\boldsymbol{a})(\boldsymbol{b}\vec{\boldsymbol{r}}),\ \forall \vec{\boldsymbol{l}} \in \mathbb{Z}_p^n, \vec{\boldsymbol{r}} \in \mathbb{Z}_p^n\}.$$

In this context, $\vec{\boldsymbol{l}}^{\top}\boldsymbol{c}\vec{\boldsymbol{r}}$ is the scalar projection of $\boldsymbol{c}$ onto the vectors $\vec{\boldsymbol{l}}$ and $\vec{\boldsymbol{r}}$. $\vec{\boldsymbol{l}}^{\top}\boldsymbol{a}$ is the left projection of $\boldsymbol{a}$ onto the vector $\vec{\boldsymbol{l}}$, and $\boldsymbol{b}\vec{\boldsymbol{r}}$ is the right projection of $\boldsymbol{b}$ onto the vector $\vec{\boldsymbol{r}}$. Subsequent subsections provide proofs for each of these relations.

Throughout this paper, $\vec{\boldsymbol{\xi}}, \vec{\boldsymbol{\xi}}^{-1} \in \mathbb{Z}_p^n$ are defined as follows: Given $\log_2 n$ challenge values $x_{(1)}, x_{(2)}, \cdots, x_{(\log_2 n)} \in \mathbb{Z}_p^*$, $\vec{\boldsymbol{\xi}}$ and $\vec{\boldsymbol{\xi}}^{-1}$ are computed as tensor products of $\log_2 n$ vectors of lengths 2:

$$\begin{aligned} \vec{\boldsymbol{\xi}} &\leftarrow (1, x_{(1)}) \otimes (1, x_{(2)}) \otimes \cdots \otimes (1, x_{(\log_2 n)}) \in \mathbb{Z}_p^n, \\ \vec{\boldsymbol{\xi}}^{-1} &\leftarrow (1, x_{(1)}^{-1}) \otimes (1, x_{(2)}^{-1}) \otimes \cdots \otimes (1, x_{(\log_2 n)}^{-1}) \in \mathbb{Z}_p^n. \end{aligned} \tag{6}$$

This procedure requires $O(n)$ field operations.

In the following, #[bullet] and #[semiBullet] refer to Algorithms 8 and 9 in Appendix B.

Scalar projection argument

We begin by presenting the scalar-projection argument for the following relation:

$$\mathcal{R}_{\text{scalar}} = \left\{ \begin{pmatrix} C \in \mathbb{G}_T; \vec{\boldsymbol{l}}, \vec{\boldsymbol{r}} \in \mathbb{Z}_p^n; \\ \hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2, \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n \end{pmatrix} : \left(\boldsymbol{a} \in \mathbb{Z}_p^{n \times n} \right) \middle| \begin{pmatrix} C = \langle\, G_0 \,|\, \vec{\boldsymbol{l}}^{\,\top} \boldsymbol{a} \vec{\boldsymbol{r}} \,|\, H_0 \,\rangle \\ \oplus \langle\, \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \,\rangle \end{pmatrix} \right\}.$$

Algorithm 1 is an argument of knowledge for $\mathcal{R}_{\text{scalar}}$. Algorithm 1 represents a column-major version in the sense that, during the matrix commitment phase, the first-tier commitment $\vec{\boldsymbol{A}}_G = \langle\, \boldsymbol{a} \,|\, \vec{\boldsymbol{G}} \,| \in \mathbb{G}_1^n$ involves committing each column of $\boldsymbol{a} \in \mathbb{Z}_p^{n \times n}$. In the proving phase, the prover reuses this cached vector $\vec{\boldsymbol{A}}_G \in \mathbb{G}_1^n$. Symmetrically, we can construct the row-major version for the same relation by utilizing the cached vector $\vec{\boldsymbol{A}}_H = |\, \vec{\boldsymbol{H}} \,|\, \boldsymbol{a} \,\rangle \in \mathbb{G}_2^n$.

Algorithm 1 is significantly faster than using Bulletproof as a black box. Notably, Lines 1 and 8 compute matrix projections before the Bulletproof iteration, leveraging sparsity to achieve $O(N + n)$ field operations. Line 3 exploits the rank-1 property of the two-tier commitment scheme, reducing the $O(n^2)$ group operations that Bulletproofs require for an n^2-vector to $O(n)$ group operations and pairings.

The following proposition shows that Algorithm 1 is a transformation of the Semi-Bulletproof protocol (Algorithm 9) in Appendix B.

Algorithm 1 Protocol of Scalar Projection Argument (Column Major)

Input: Public input: $C \in \mathbb{G}_T, \vec{\boldsymbol{l}} \in \mathbb{Z}_p^n, \vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$;
$\vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n; \hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2$;
$\mathcal{P}$'s private input: $\boldsymbol{a} \in \mathbb{Z}_p^{n \times n}$ and cache $\vec{\boldsymbol{A}}_G = \langle\, \boldsymbol{a} \,|\, \vec{\boldsymbol{G}} \,|\, \rangle \in \mathbb{G}_1^n$

Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

$\mathcal{P}$ computes: $\vec{\boldsymbol{v}} \leftarrow \vec{\boldsymbol{l}}^{\top} \boldsymbol{a} \in \mathbb{Z}_p^n$; $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{A}}_G \in \mathbb{G}_1^n$;

foreach $j \in [1, \log_2 n]$ **do**

  $\mathcal{P}$ computes:
  $(\vec{\boldsymbol{A}}_L || \vec{\boldsymbol{A}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{A}}$; $(\vec{\boldsymbol{H}}_L || \vec{\boldsymbol{H}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{H}}$;
  $(\vec{\boldsymbol{v}}_L || \vec{\boldsymbol{v}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{v}}$; $(\vec{\boldsymbol{r}}_L || \vec{\boldsymbol{r}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{r}}$;
  $L_{(j)} \leftarrow (\vec{\boldsymbol{A}}_L \bullet \vec{\boldsymbol{H}}_R) \oplus \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{v}}_L \bullet \vec{\boldsymbol{r}}_R \,|\, \hat{H}_0 \,\rangle \in \mathbb{G}_T$;
  $R_{(j)} \leftarrow (\vec{\boldsymbol{A}}_R \bullet \vec{\boldsymbol{H}}_L) \oplus \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{v}}_R \bullet \vec{\boldsymbol{r}}_L \,|\, \hat{H}_0 \,\rangle \in \mathbb{G}_T$;

  $\mathcal{P} \to \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}_T$;

  $\mathcal{V} \to \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;

  $\mathcal{P}$ computes:
  $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{A}}_L \oplus x_{(j)}^{-1} \vec{\boldsymbol{A}}_R \in \mathbb{G}_1^{n^2/2^j}$; $\vec{\boldsymbol{H}} \leftarrow \vec{\boldsymbol{H}}_L \oplus x_{(j)} \vec{\boldsymbol{H}}_R \in \mathbb{G}_2^{n^2/2^j}$;
  $\vec{\boldsymbol{v}} \leftarrow \vec{\boldsymbol{v}}_L + x_{(j)}^{-1} \vec{\boldsymbol{v}}_R \in \mathbb{Z}_p^{n^2/2^j}$; $\vec{\boldsymbol{r}} \leftarrow \vec{\boldsymbol{r}}_L + x_{(j)} \vec{\boldsymbol{r}}_R \in \mathbb{Z}_p^{n^2/2^j}$;

// Now, $\vec{\boldsymbol{A}}, \vec{\boldsymbol{v}}$, and $\vec{\boldsymbol{H}}$ have been reduced to single elements

$\mathcal{P}$ and $\mathcal{V}$ compute $\vec{\boldsymbol{\xi}}_H^{-1} \in \mathbb{Z}_p^n$ from $x_{(1)}, \cdots, x_{(\log_2 n)}$;

$\mathcal{P}$ computes: $\vec{\boldsymbol{a}} \leftarrow \boldsymbol{a} \vec{\boldsymbol{\xi}}_H^{-1} \in \mathbb{Z}_p^n$; $H \leftarrow \vec{\boldsymbol{H}} \in \mathbb{G}_1$; $r \leftarrow \vec{\boldsymbol{r}} \in \mathbb{Z}_p$;

foreach $j \in [\log_2 n + 1, 2 \log_2 n]$ **do**

  $\mathcal{P}$ computes:
  $(\vec{\boldsymbol{a}}_L || \vec{\boldsymbol{a}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{a}}$; $(\vec{\boldsymbol{G}}_L || \vec{\boldsymbol{G}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{H}}$; $(\vec{\boldsymbol{l}}_L || \vec{\boldsymbol{l}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{l}}$;
  $L_{(j)} \leftarrow e(\vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{G}}_R, H) \oplus \langle\, \hat{G}_0 \,|\, r\, \vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{l}}_R \,|\, \hat{H}_0 \,\rangle \in \mathbb{G}_T$;
  $R_{(j)} \leftarrow e(\vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{G}}_L, H) \oplus \langle\, \hat{G}_0 \,|\, r\, \vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{l}}_L \,|\, \hat{H}_0 \,\rangle \in \mathbb{G}_T$;

  $\mathcal{P} \to \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}_T$;

  $\mathcal{V} \to \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;

  $\mathcal{P}$ computes:
  $\vec{\boldsymbol{a}} \leftarrow \vec{\boldsymbol{a}}_L + x_{(j)}^{-1} \vec{\boldsymbol{a}}_R \in \mathbb{Z}_p^{n/2^j}$; $\vec{\boldsymbol{G}} \leftarrow \vec{\boldsymbol{G}}_L \oplus x_{(j)} \vec{\boldsymbol{G}}_R \in \mathbb{G}_1^{n/2^j}$;
  $\vec{\boldsymbol{l}} \leftarrow \vec{\boldsymbol{l}}_L + x_{(j)} \vec{\boldsymbol{l}}_R \in \mathbb{Z}_p^{n/2^j}$;

// Now, $\vec{\boldsymbol{a}}$ has been reduced to a scalar

$\mathcal{P} \to \mathcal{V}$: $\hat{a} \leftarrow \vec{\boldsymbol{a}} \in \mathbb{Z}_p$;

$\mathcal{V}$ computes $\vec{\boldsymbol{\xi}}_G \in \mathbb{Z}_p^n$ from $x_{(\log_2 n + 1)}, \cdots, x_{(2 \log_2 n)}$;

$\mathcal{V}$ checks: $C \oplus \bigoplus_{j=1}^{2 \log_2 n} (x_{(j)} L_{(j)} \oplus x_{(j)}^{-1} R_{(j)})$
$\stackrel{?}{=} \langle\, \hat{G}_0 \,|\, \hat{a} (\vec{\boldsymbol{\xi}}_G \bullet \vec{\boldsymbol{l}}) (\vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{r}}) \,|\, \hat{H}_0 \,\rangle \oplus \langle\, \vec{\boldsymbol{\xi}}_G \bullet \vec{\boldsymbol{G}} \,|\, \hat{a} \,|\, \vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{H}} \,\rangle$;

Proposition 2 *Given the following transformation*:

$$\#[\mathsf{semiBullet}\,]\,n \Rrightarrow n^2, \quad \#[\mathsf{semiBullet}\,]\,\vec{\boldsymbol{a}} \Rrightarrow \mathsf{vec}(\boldsymbol{a}^\top),$$
$$\#[\mathsf{semiBullet}\,]\,\vec{\boldsymbol{G}} \Rrightarrow \mathsf{vec}([\vec{\boldsymbol{H}} \otimes \vec{\boldsymbol{G}}^\top] \oplus [(\vec{\boldsymbol{r}}\hat{H}_0) \otimes (\vec{\boldsymbol{l}}\hat{G}_0)^\top]),$$

then:

- $\mathcal{R}_{\mathsf{semiBullet}}$ in Eq. (B13) is transformed to $\mathcal{R}_{scalar}$;
- Algorithm 1 generates the same transcript elements as those generated by Algorithm 9 post-transformation.
- The verification check of Algorithm 1 is identical to the verification check of Algorithm 9 post-transformation.

Consequently, Algorithm 1 is an argument of knowledge for $\mathcal{R}_{scalar}$.

Proof of Proposition 2 is provided in Appendix C.2.

Left projection and right projection argument

To prove that a vector $\vec{\boldsymbol{c}}^\top \in \mathbb{Z}_p^n$ is the left projection $\vec{\boldsymbol{l}}^\top \boldsymbol{a}$ of the matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n\times n}$ onto a row vector $\vec{\boldsymbol{l}}^\top \in \mathbb{Z}_p^n$, we present the left projection argument in Algorithm 2 for the following relation:

Similarly, Algorithm 2 is much faster than invoking Bulletproof as a black box.

Proposition 3 *Given the following transformation*:

$$\#[\mathsf{semiBullet}\,]\,n \Rrightarrow n^2, \quad \#[\mathsf{semiBullet}\,]\,\vec{\boldsymbol{a}} \Rrightarrow \mathsf{vec}(\boldsymbol{a}),$$
$$\#[\mathsf{semiBullet}\,]\,\vec{\boldsymbol{G}} \Rrightarrow \mathsf{vec}([\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^\top] \oplus [(\vec{\boldsymbol{l}}\hat{G}_0) \otimes \vec{\boldsymbol{H}}^\top]),$$

then:

- $\mathcal{R}_{\mathsf{semiBullet}}$ in Eq. (B13) is transformed to $\mathcal{R}_{\mathsf{left}}$;
- Algorithm 2 generates the same transcript elements as those generated by Algorithm 9.
- The verification check of Algorithm 2 is identical to the verification check of Algorithm 9 post-transformation.

Consequently, Algorithm 2 is an argument of knowledge for $\mathcal{R}_{\mathsf{left}}$.

The proof of Proposition 3 closely mirrors that of Proposition 2 and is therefore omitted for brevity.

$$\mathcal{R}_{\mathsf{left}} = \left\{ \begin{pmatrix} C \in \mathbb{G}_T; \vec{\boldsymbol{l}} \in \mathbb{Z}_p^n; \hat{G}_0 \in \mathbb{G}_1, \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n \end{pmatrix} : \left(\boldsymbol{a} \in \mathbb{Z}_p^{n\times n} \right) \middle| \begin{pmatrix} C = \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{l}}^\top \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \,\rangle \\ \oplus \langle\, \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \,\rangle \end{pmatrix} \right\}. \tag{7}$$

Algorithm 2 Protocol of Left Projection Argument

Input: Public input: $C \in \mathbb{G}_T, \vec{\boldsymbol{l}} \in \mathbb{Z}_p^n$; $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n$; $\hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2$;
$\mathcal{P}$'s private input: $\boldsymbol{a} \in \mathbb{Z}_p^{n \times n}$ and cache $\vec{\boldsymbol{A}}_H = |\, \vec{\boldsymbol{H}} \,|\, \boldsymbol{a} \,\rangle \in \mathbb{G}_2^n$

Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

$\mathcal{P}$ computes: $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{A}}_H \in \mathbb{G}_2^n$;

foreach $j \in [1, \log_2 n]$ **do**

  $\mathcal{P}$ computes:
  $(\vec{\boldsymbol{A}}_L || \vec{\boldsymbol{A}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{A}}$; $(\vec{\boldsymbol{G}}_L || \vec{\boldsymbol{G}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{G}}$; $(\vec{\boldsymbol{l}}_L || \vec{\boldsymbol{l}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{l}}$;
  $L_{(j)} \leftarrow (\vec{\boldsymbol{G}}_R \bullet \vec{\boldsymbol{A}}_L) \oplus e(\hat{G}_0, \vec{\boldsymbol{l}}_R \bullet \vec{\boldsymbol{A}}_L) \in \mathbb{G}_T$;
  $R_{(j)} \leftarrow (\vec{\boldsymbol{G}}_L \bullet \vec{\boldsymbol{A}}_R) \oplus e(\hat{G}_0, \vec{\boldsymbol{l}}_L \bullet \vec{\boldsymbol{A}}_R) \in \mathbb{G}_T$;

  $\mathcal{P} \to \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}_T$;

  $\mathcal{V} \to \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;

  $\mathcal{P}$ computes:
  $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{A}}_L \oplus x_{(j)}^{-1} \vec{\boldsymbol{A}}_R \in \mathbb{G}_2^{n/2^j}$; $\vec{\boldsymbol{G}} \leftarrow \vec{\boldsymbol{G}}_L \oplus x_{(j)} \vec{\boldsymbol{G}}_R \in \mathbb{G}_1^{n/2^j}$;
  $\vec{\boldsymbol{l}} \leftarrow \vec{\boldsymbol{l}}_L + x_{(j)} \vec{\boldsymbol{l}}_R \in \mathbb{Z}_p^{n/2^j}$;

`// Now,` $\vec{\boldsymbol{A}}, \vec{\boldsymbol{G}}$ `, and` $\vec{\boldsymbol{l}}$ `have all been reduced to single elements`

$\mathcal{P}$ and $\mathcal{V}$ compute $\vec{\boldsymbol{\xi}}_G^{-1} \in \mathbb{Z}_p^n$ from $x_{(1)}, \cdots, x_{(\log_2 n)}$;

$\mathcal{P}$ computes:
$\vec{\boldsymbol{a}} = (\vec{\boldsymbol{\xi}}_G^{-1})^\top \boldsymbol{a} \in \mathbb{Z}_p^n$; $G \leftarrow \vec{\boldsymbol{G}} \in \mathbb{G}_1$; $l \leftarrow \vec{\boldsymbol{l}} \in \mathbb{Z}_p$; $W \leftarrow G \oplus (l \hat{G}_0) \in \mathbb{G}_1$;

foreach $j \in \mathsf{range}(\log_2 n + 1, 2 \log_2 n)$ **do**

  $\mathcal{P}$ computes:
  $(\vec{\boldsymbol{a}}_L || \vec{\boldsymbol{a}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{a}}$; $(\vec{\boldsymbol{H}}_L || \vec{\boldsymbol{H}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{H}}$;
  $L_{(j)} \leftarrow e(W, \vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{H}}_R) \in \mathbb{G}_T$; $R_{(j)} \leftarrow e(W, \vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{H}}_L) \in \mathbb{G}_T$;

  $\mathcal{P} \to \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}_T$;

  $\mathcal{V} \to \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;

  $\mathcal{P}$ computes:
  $\vec{\boldsymbol{a}} \leftarrow \vec{\boldsymbol{a}}_L + x_{(j)}^{-1} \vec{\boldsymbol{a}}_R \in \mathbb{G}_2^{n/2^j}$; $\vec{\boldsymbol{H}} \leftarrow \vec{\boldsymbol{H}}_L \oplus x_{(j)} \vec{\boldsymbol{H}}_R \in \mathbb{G}_1^{n/2^j}$;

`// Now,` $\vec{\boldsymbol{a}}$ `has been reduced to a scalar`

$\mathcal{P} \to \mathcal{V}$: $\hat{a} \leftarrow \vec{\boldsymbol{a}} \in \mathbb{Z}_p$;

$\mathcal{V}$ computes $\vec{\boldsymbol{\xi}}_H \in \mathbb{Z}_p^n$ from $x_{(\log_2 n+1)}, \cdots, x_{(2 \log_2 n)}$;

$\mathcal{V}$ checks: $C \oplus \bigoplus_{j=1}^{2 \log_2 n} (x_{(j)} L_{(j)} \oplus x_{(j)}^{-1} R_{(j)})$
$\stackrel{?}{=} \langle\, \hat{G}_0 \,|\, \hat{a} (\vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{l}}) \,|\, \vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{H}} \,\rangle \oplus \langle\, \vec{\boldsymbol{\xi}}_G \bullet \vec{\boldsymbol{G}} \,|\, \hat{a} \,|\, \vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{H}} \,\rangle$;

in a symmetric manner, we can construct the right-projection argument for the right projection of the matrix $\boldsymbol{a} \in \mathbb{Z}_p^{n \times n}$ onto a column vector $\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$. The target relation is $\mathcal{R}_{\mathsf{right}}$ as follows:

$$\mathcal{R}_{\mathsf{right}} = \left\{ \begin{pmatrix} C \in \mathbb{G}_T; \vec{\boldsymbol{r}} \in \mathbb{Z}_p^n; \hat{H}_0 \in \mathbb{G}_2, \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n \end{pmatrix} : \left(\boldsymbol{a} \in \mathbb{Z}_p^{n \times n} \right) \middle| \begin{pmatrix} C = \langle\, \vec{\boldsymbol{G}} \,|\, \boldsymbol{a}\vec{\boldsymbol{r}} \,|\, \hat{H}_0 \,\rangle \\ \oplus \langle\, \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \,\rangle \end{pmatrix} \right\}.$$

Because the right projection argument is exactly symmetric to the left projection argument, further details are omitted here for brevity.

Algorithm 3 is an argument of knowledge for this relation.

Inner product argument in $\mathbb{G}_T$

The inner-product relation, when represented in the target group, is as follows:

$$\mathcal{R}_{\mathsf{ipGt}} = \left\{ \begin{pmatrix} C \in \mathbb{G}_T; \\ \hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2, \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n \end{pmatrix} : \begin{pmatrix} \vec{\boldsymbol{a}} \in \mathbb{Z}_p^n, \\ \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n \end{pmatrix} \middle| \begin{pmatrix} C = \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{b}} \,|\, \hat{H}_0 \,\rangle \\ \oplus e(\hat{G}_0, \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{H}}) \\ \oplus e(\vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{b}}, \hat{H}_0) \end{pmatrix} \right\}. \tag{8}$$

Algorithm 3 Protocol of Inner Product Argument in $\mathbb{G}_T$

Input: Public input: $C \in \mathbb{G}_T$; $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n$; $\hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2$;
$\mathcal{P}$'s private input: $\vec{\boldsymbol{a}} \in \mathbb{Z}_p^n, \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n$

Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

foreach $j \in \mathsf{range}(1, \log_2 n)$ **do**

  $\mathcal{P}$ computes:

  $(\vec{\boldsymbol{a}}_L || \vec{\boldsymbol{a}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{a}}$; $(\vec{\boldsymbol{b}}_L || \vec{\boldsymbol{b}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{b}}$;

  $(\vec{\boldsymbol{G}}_L || \vec{\boldsymbol{G}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{G}}$; $(\vec{\boldsymbol{H}}_L || \vec{\boldsymbol{H}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{H}}$;

  $L_{(j)} \leftarrow e(\hat{G}_0, \vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{H}}_R) \oplus e(\vec{\boldsymbol{b}}_R \bullet \vec{\boldsymbol{G}}_L, \hat{H}_0) \oplus \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{b}}_R \,|\, \hat{H}_0 \,\rangle \in \mathbb{G}_T$;

  $R_{(j)} \leftarrow e(\hat{G}_0, \vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{H}}_L) \oplus e(\vec{\boldsymbol{b}}_L \bullet \vec{\boldsymbol{G}}_R, \hat{H}_0) \oplus \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{b}}_L \,|\, \hat{H}_0 \,\rangle \in \mathbb{G}_T$;

  $\mathcal{P} \to \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}_T$;

  $\mathcal{V} \to \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;

  $\mathcal{P}$ computes:

  $\vec{\boldsymbol{a}} \leftarrow \vec{\boldsymbol{a}}_L + x_{(j)}^{-1} \vec{\boldsymbol{a}}_R \in \mathbb{Z}_p^{n/2^j}$; $\vec{\boldsymbol{b}} \leftarrow \vec{\boldsymbol{b}}_L + x_{(j)} \vec{\boldsymbol{b}}_R \in \mathbb{Z}_p^{n/2^j}$;

  $\vec{\boldsymbol{G}} \leftarrow \vec{\boldsymbol{G}}_L \oplus x_{(j)} \vec{\boldsymbol{G}}_R \in \mathbb{G}^{n/2^j}$; $\vec{\boldsymbol{H}} \leftarrow \vec{\boldsymbol{H}}_L \oplus x_{(j)}^{-1} \vec{\boldsymbol{H}}_R \in \mathbb{G}^{n/2^j}$;

`// Now, both` $\vec{\boldsymbol{a}}$ `and` $\vec{\boldsymbol{b}}$ `have been reduced to scalars`

$\mathcal{P} \to \mathcal{V}$: $\hat{a} \leftarrow \vec{\boldsymbol{a}} \in \mathbb{Z}_p$; $\hat{b} \leftarrow \vec{\boldsymbol{b}} \in \mathbb{Z}_p$;

$\mathcal{V}$ computes $\vec{\boldsymbol{\xi}} \in \mathbb{Z}_p^n$ and $\vec{\boldsymbol{\xi}}^{-1} \in \mathbb{Z}_p^n$ from $x_{(1)}, \cdots, x_{(\log_2 n)}$;

$\mathcal{V}$ checks: $C \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} L_{(j)} \oplus x_{(j)}^{-1} R_{(j)})$

$\stackrel{?}{=} \langle\, \hat{G}_0 \,|\, \hat{a}\hat{b} \,|\, \hat{H}_0 \,\rangle \oplus \langle\, \hat{G}_0 \,|\, \hat{a} \,|\, \vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{H}} \,\rangle \oplus \langle\, \vec{\boldsymbol{\xi}}^{-1} \bullet \vec{\boldsymbol{G}} \,|\, \hat{b} \,|\, \hat{H}_0 \,\rangle$

Proposition 4 *Given the following transformation*:

$$\#[\text{bullet}]\, n \Rrightarrow n, \quad \#[\text{bullet}]\, \vec{\boldsymbol{a}} \Rrightarrow \vec{\boldsymbol{a}}, \quad \#[\text{bullet}]\, \vec{\boldsymbol{b}} \Rrightarrow \vec{\boldsymbol{b}},$$
$$\#[\text{bullet}]\, U \Rrightarrow \hat{G}_0 \otimes \hat{H}_0, \quad \#[\text{bullet}]\, \vec{\boldsymbol{G}} \Rrightarrow \hat{G}_0 \otimes \vec{\boldsymbol{H}}, \quad \#[\text{bullet}]\, \vec{\boldsymbol{H}} \Rrightarrow \vec{\boldsymbol{G}} \otimes \hat{H}_0,$$

then:

- $\mathcal{R}_{\mathsf{bullet}}$ in Eq. (B12) is transformed to $\mathcal{R}_{\mathsf{ipGt}}$;
- Algorithm 3 generates the same transcript elements as those generated by Algorithm 8.
- The verification check of Algorithm 8 is identical to the verification check of Algorithm 3 post-transformation.

Consequently, Algorithm 3 is an argument of knowledge for $\mathcal{R}_{\mathsf{ipGt}}$.

Proof of Proposition 4 is detailed in Appendix C.3.

Matrix multiplication argument

The matrix-multiplication argument is for the following relation:

$$\mathcal{R}_{\mathsf{matMul}} = \left\{ \begin{pmatrix} C_c, C_a, C_b \in \mathbb{G}_T; \\ \hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2 \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n \end{pmatrix} : \begin{pmatrix} \boldsymbol{c} \in \mathbb{Z}_p^{n\times n}, \\ \boldsymbol{a} \in \mathbb{Z}_p^{n\times n}, \\ \boldsymbol{b} \in \mathbb{Z}_p^{n\times n} \end{pmatrix} \middle| \begin{pmatrix} \boldsymbol{c} = \boldsymbol{ab} \\ \wedge\, C_c = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{c} \,|\, \vec{\boldsymbol{H}} \rangle \\ \wedge\, C_a = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle \\ \wedge\, C_b = \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{b} \,|\, \vec{\boldsymbol{H}} \rangle \end{pmatrix} \right\} \tag{9}$$

zkMatrix (Cong et al. 2024) provides an argument of knowledge for matrix multiplication by using four inner products, annotated as ①, ②, ③, ④ in the following equivalence:

$$\{\boldsymbol{c} = \boldsymbol{ab}\} \iff \{\forall y \in \mathbb{Z}_p,\ (\vec{\boldsymbol{y}}_L^\top \boldsymbol{c} \vec{\boldsymbol{y}}_R)^{①} = ((\vec{\boldsymbol{y}}_L^\top \boldsymbol{a})^{②} (\boldsymbol{b}\vec{\boldsymbol{y}}_R)^{③})^{④}\},$$

where $\vec{\boldsymbol{y}}_L^\top := (1, y^n, \cdots, y^{(n-1)n}) \in \mathbb{Z}_p^n$ and $\vec{\boldsymbol{y}}_R := (1, y, \cdots, y^{n-1})^\top \in \mathbb{Z}_p^n$. These vectors are symbolized by $\vec{\boldsymbol{l}}$ and $\vec{\boldsymbol{r}}$ in our main text. This improves the classical Freivalds' algorithm, which projects the matrices on a single side rather than on both sides.

The inner product ① is called the *standard polynomial* of the matrix $\boldsymbol{c}$. Specifically, for an $(n \times n)$-matrix $\boldsymbol{c}$, it is represented by a degree $(n^2 - 1)$ polynomial of y as follows:

$$\vec{\boldsymbol{y}}_L^\top \boldsymbol{c} \vec{\boldsymbol{y}}_R = \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} y^{(i-1)n+(j-1)}.$$

This can be viewed as the inner product between $\mathsf{vec}(\boldsymbol{c})$ and a public n^2-vector $(1, y, y^2, \ldots, y^{n^2-1})$, and thus can be proven by the semi-Bulletproof in Algorithm 9 (semi-inner-product argument as per (Cong et al. 2024)).

Following this approach, we can construct an argument of knowledge for the matrix multiplication relation utilizing the previous subprotocols, detailed in Algorithm 4.

Algorithm 4 Protocol of Matrix Multiplication Argument

Input: Public input: $C_c, C_a, C_b \in \mathbb{G}_T$; $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n$; $\hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2$;
$\mathcal{P}$'s private input: $\boldsymbol{c} \in \mathbb{Z}_p^{n\times n}, \boldsymbol{a} \in \mathbb{Z}_p^{n\times n}, \boldsymbol{b} \in \mathbb{Z}_p^{n\times n}$ and private caches:
$\vec{\boldsymbol{C}}_G = \langle\, \boldsymbol{c} \,|\, \vec{\boldsymbol{G}} \,|\, \in \mathbb{G}_1^n, \vec{\boldsymbol{A}}_H = |\, \vec{\boldsymbol{H}} \,|\, \boldsymbol{a} \,\rangle \in \mathbb{G}_2^n, \vec{\boldsymbol{B}}_G = \langle\, \boldsymbol{b} \,|\, \boldsymbol{G} \,|\, \in \mathbb{G}_1^n$
Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

1 $\mathcal{V} \to \mathcal{P}$: $y \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;
2 $\mathcal{P}$ and $\mathcal{V}$ compute:
$\vec{\boldsymbol{l}} \leftarrow (1, y^n, y^{2n}, \cdots, y^{(n-1)n}) \in \mathbb{Z}_p^m$; $\vec{\boldsymbol{r}} \leftarrow (1, y, y^2, \cdots, y^{n-1}) \in \mathbb{Z}_p^n$;
3 $\mathcal{P}$ computes:
$\vec{\boldsymbol{a}}_y \leftarrow \vec{\boldsymbol{l}}^\top \boldsymbol{a} \in \mathbb{Z}_p^n$; $\vec{\boldsymbol{b}}_y \leftarrow \boldsymbol{b}\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$; $d \leftarrow \vec{\boldsymbol{l}}^\top \boldsymbol{c}\vec{\boldsymbol{r}} \in \mathbb{Z}_p$;
$C_{ay} \leftarrow e(\hat{G}_0, \vec{\boldsymbol{a}}_y \bullet \vec{\boldsymbol{H}}) \in \mathbb{G}_T$; $C_{by} \leftarrow e(\vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{b}}_y, \hat{H}_0) \in \mathbb{G}_T$;
$C_d \leftarrow \langle\, \hat{G}_0 \,|\, d \,|\, \hat{H}_0 \,\rangle \in \mathbb{G}_T$;
4 $\mathcal{P} \to \mathcal{V}$: $C_d, C_{ay}, C_{by} \in \mathbb{G}_T$;
5 $\mathcal{V} \to \mathcal{P}$: $x \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;
6 $\mathcal{P}$ and $\mathcal{V}$ run scalar projection argument (column major) on input:
$(xC_d \oplus C_c; x\vec{\boldsymbol{l}}, \vec{\boldsymbol{r}}; \vec{\boldsymbol{G}}, \vec{\boldsymbol{H}}; \hat{G}_0, \hat{H}_0 : \boldsymbol{c}; \vec{\boldsymbol{C}}_H)$;
7 $\mathcal{P}$ and $\mathcal{V}$ run left projection argument on input:
$(xC_{ay} \oplus C_a; x\vec{\boldsymbol{l}}; \vec{\boldsymbol{G}}, \vec{\boldsymbol{H}}; \hat{G}_0, \hat{H}_0 : \boldsymbol{a}; \vec{\boldsymbol{A}}_H)$;
8 $\mathcal{P}$ and $\mathcal{V}$ run right projection argument on input:
$(xC_{by} \oplus C_b; x\vec{\boldsymbol{r}}; \vec{\boldsymbol{G}}, \vec{\boldsymbol{H}}; \hat{G}_0, \hat{H}_0 : \boldsymbol{b}; \vec{\boldsymbol{B}}_G)$;
9 $\mathcal{P}$ and $\mathcal{V}$ run inner-product argument in $\mathbb{G}_T$ on input:
$(C_d \oplus xC_{ay} \oplus x^{-1}C_{by}; \vec{\boldsymbol{G}}, \vec{\boldsymbol{H}}; \hat{G}_0, \hat{H}_0 : x\vec{\boldsymbol{a}}_y, x^{-1}\vec{\boldsymbol{b}}_y)$;
10 If all return TRUE, then **return** TRUE; otherwise **return** FALSE

Proposition 5 *Algorithm* 4 *is an argument of knowledge for* $\mathcal{R}_{\mathsf{matMul}}$.

Proof of Proposition 5 is provided in Appendix C.4.

Logarithmic verifier time with transparent setup

The verification check of Algorithms 1-3 involves computing $V_G = \vec{\boldsymbol{\xi}}_G \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_1$ and $V_H = \vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_2$, where $\vec{\boldsymbol{\xi}}_G, \vec{\boldsymbol{\xi}}_H \in \mathbb{Z}_p^n$ are public tensor-structured field vectors. Computing these values requires $O(n)$ exponentiations for the verifier. zkMatrix has proposed a method to achieve logarithmic verifier time by shifting this computational burden to the prover using a trusted setup. We improve upon zkMatrix by achieving logarithmic verifier time under a transparent setup.

Specifically, given public vectors $\vec{\boldsymbol{a}} \in \mathbb{Z}_p^n$, $\vec{\boldsymbol{b}} \in \mathbb{Z}_p^n$, and base vectors $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$, $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$ known to both parties, we design a proof for the following language $\mathcal{L}_{\mathsf{pip}}$:

$$\mathcal{L}_{\mathsf{pip}} = \left\{ \begin{pmatrix} V_a \in \mathbb{G}_1; \vec{\boldsymbol{a}} \in \mathbb{Z}_p^n; \vec{\boldsymbol{G}} \in \mathbb{G}_1^n; \\ V_b \in \mathbb{G}_2; \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n; \vec{\boldsymbol{H}} \in \mathbb{G}_2^n \end{pmatrix} : \begin{pmatrix} V_a = \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}} \\ \wedge\, V_b = \vec{\boldsymbol{b}} \bullet \vec{\boldsymbol{H}} \end{pmatrix} \right\}. \tag{10}$$

Using the proof for $\mathcal{L}_{\mathsf{pip}}$, by setting $\vec{a} = \vec{\boldsymbol{\xi}}_G$ and $\vec{\boldsymbol{b}} = \vec{\boldsymbol{\xi}}_H$, the prover can produce proofs for $V_G = \vec{\boldsymbol{\xi}}_G \bullet \vec{\boldsymbol{G}}$ and $V_H = \vec{\boldsymbol{\xi}}_H \bullet \vec{\boldsymbol{H}}$.

To this end, we first design a logarithmic-verifier-time inner-pairing-product argument for proving the inner product between a $\mathbb{G}_1$-vector $\vec{\boldsymbol{A}} \in \mathbb{G}_1^n$ and

a $\mathbb{G}_2$-vector $\vec{\boldsymbol{B}} \in \mathbb{G}_2^n$. These vectors are committed to $A = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_T$ and $B = \vec{\boldsymbol{B}} \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_T$, with base vectors $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$ and $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$, respectively. Specifically, we design an argument of knowledge for the following $\mathcal{R}_{\mathsf{pairingIP}}$:

$$\mathcal{R}_{\mathsf{pairingIP}} = \left\{ \begin{pmatrix} C, A, B \in \mathbb{G}_T; \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n; \vec{\boldsymbol{H}} \in \mathbb{G}_2^n \end{pmatrix} \middle| \begin{pmatrix} \vec{\boldsymbol{A}} \in \mathbb{G}_1^n; \\ \vec{\boldsymbol{B}} \in \mathbb{G}_2^n \end{pmatrix} : \begin{pmatrix} C = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}} \\ \wedge A = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{H}} \wedge B = \vec{\boldsymbol{B}} \bullet \vec{\boldsymbol{G}} \end{pmatrix} \right\},$$

To attain logarithmic verifier time for $\mathcal{R}_{\mathsf{pairingIP}}$, the verifier must pre-compute the following inner products in a one-time preprocessing phase with linear complexity:

$$\Delta = \vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{H}}, \quad \vec{\boldsymbol{G}}^{(0)} := \vec{\boldsymbol{G}}, \quad \vec{\boldsymbol{H}}^{(0)} := \vec{\boldsymbol{H}}, \quad \vec{\boldsymbol{G}}^{(j)} := \vec{\boldsymbol{G}}_L^{(j-1)}, \quad \vec{\boldsymbol{H}}^{(j)} := \vec{\boldsymbol{H}}_L^{(j-1)},$$
$$\Delta^{(j)} = \vec{\boldsymbol{G}}^{(j)} \bullet \vec{\boldsymbol{H}}^{(j)}, \quad \Delta_{GR}^{(j)} = \vec{\boldsymbol{G}}_R^{(j-1)} \bullet \vec{\boldsymbol{H}}^{(j)}, \quad \Delta_{HR}^{(j)} = \vec{\boldsymbol{H}}_R^{(j-1)} \bullet \vec{\boldsymbol{G}}^{(j)}, \quad j \in [1, n].$$

Given these precomputed values, Algorithm 5 functions as an argument of knowledge for $\mathcal{R}_{\mathsf{pairingIP}}$. Knowledge soundness of Algorithm 5 relies on the absence of non-trivial inner-pairing products when $\vec{\boldsymbol{G}}$ and $\vec{\boldsymbol{H}}$ are randomly selected from $\mathbb{G}_1^n$ and $\mathbb{G}_2^n$, respectively (Lemma 1). When $\vec{\boldsymbol{G}}$ or $\vec{\boldsymbol{H}}$ are linearly dependent (e.g., $\vec{\boldsymbol{G}} = \vec{\boldsymbol{O}}$ or $\vec{\boldsymbol{H}} = \vec{\boldsymbol{O}}$), knowledge soundness is compromised. However, computational soundness remains intact, provided the verifier ensures the reliability of the reduced group elements $\hat{A} \in \mathbb{G}_1$ and $\hat{B} \in \mathbb{G}_2$ at Line 14. These results are formalized below:

The idea of leveraging pre-computed $\mathbb{G}_T$ elements to achieve logarithmic verifier time in pairing-based inner product arguments was first introduced in the Dory polynomial commitment scheme (Lee 2021). We adopt this precomputation technique, but present a slightly different construction for the inner-pairing-product argument in Algorithm 5.

Proposition 6 *If both $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$ are randomly chosen group vectors, then Algorithm 6 serves as an argument of knowledge for $\mathcal{R}_{\mathsf{pairingIP}}$. Furthermore, even if non-trivial logarithmic relations exist among $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n$ or $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$, Algorithm 6 still achieves computational soundness as a proof for $\mathcal{R}_{\mathsf{pairingIP}}$, provided the reliability of $\hat{A}$ and $\hat{B}$ at the protocol's conclusion.*

Proof of Proposition 6 is provided in Appendix C.5.

Using this inner-pairing-product argument, we can design an efficient proof system for the public inner-product language $\mathcal{L}_{\mathsf{pip}}$ defined in Eq. (10). The inner products $V_a = \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_1$ and $V_b = \vec{\boldsymbol{b}} \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_2$ can be verified through the following inner-pairing products:

$$e(V_a, \hat{H}_0) = \vec{\boldsymbol{G}} \bullet (\vec{\boldsymbol{a}}\hat{H}_0), \quad e(\hat{G}_0, V_b) = (\vec{\boldsymbol{b}}\hat{G}_0) \bullet \vec{\boldsymbol{H}}. \tag{11}$$

As the commitments to vectors $(\vec{\boldsymbol{a}}\hat{H}_0)$ and $(\vec{\boldsymbol{b}}\hat{G}_0)$ are unknown for random base vectors, we define their bases as zero vectors, yielding: $(\vec{\boldsymbol{a}}\hat{H}_0) \bullet \vec{\boldsymbol{O}} = O$ and $(\vec{\boldsymbol{b}}\hat{G}_0) \bullet \vec{\boldsymbol{O}} = O$. However, in this scenario, the verifier must ensure that the reduced group elements $(\hat{A}, \hat{B})$ at Line 13 of Algorithm 5 are reliable.

To achieve this, we require the prover to concurrently execute two auxiliary inner-pairing-product arguments with identical random challenges, employing the linearly independent base vectors $\vec{\boldsymbol{G}}$ and $\vec{\boldsymbol{H}}$, for the following inner products:

$$\Delta = \vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{H}}, \quad O = \vec{\boldsymbol{O}} \bullet \vec{\boldsymbol{O}}. \tag{12}$$

The reliability of the reduced group elements from these auxiliary protocols ensures the correctness of the corresponding reduced group elements for the target inner products in Eq. (11). This procedure is detailed in Algorithm 6.

Formally, we have the following proposition:

Algorithm 5 Inner-Pairing-Product Argument

Input: Public input: $C, A, B \in \mathbb{G}_T$; $\vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n$;
$\mathcal{V}$'s precomputation: $\Delta \in \mathbb{G}_T, \Delta^{(j)} = \vec{\boldsymbol{G}}_{[0:n/2^j]} \bullet \vec{\boldsymbol{H}}_{[0:n/2^j]}$
$\Delta_{GR}^{(j)} = \vec{\boldsymbol{G}}_{[0:n/2^{j-1}]R} \bullet \vec{\boldsymbol{H}}_{[0:n/2^j]}, \Delta_{HR}^{(j)} = \vec{\boldsymbol{H}}_{[0:n/2^{j-1}]R} \bullet \vec{\boldsymbol{G}}_{[0:n/2^j]}$
$\mathcal{P}$'s private input: $\vec{\boldsymbol{A}} \in \mathbb{G}_1^n, \vec{\boldsymbol{B}} \in \mathbb{G}_2^n$
Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

foreach $j \in [1, \log_2 n]$ **do**
  // Iteratively proving $C = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}} \wedge A = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{H}} \wedge B = \vec{\boldsymbol{B}} \bullet \vec{\boldsymbol{G}}$
  $\mathcal{P}$ computes:
   $C_L \leftarrow \vec{\boldsymbol{A}}_L \bullet \vec{\boldsymbol{B}}_L \in \mathbb{G}_T$;
   $A_L \leftarrow \vec{\boldsymbol{A}}_L \bullet \vec{\boldsymbol{H}}_L \in \mathbb{G}_T$; $B_L \leftarrow \vec{\boldsymbol{B}}_L \bullet \vec{\boldsymbol{G}}_L \in \mathbb{G}_T$;
   $A_R \leftarrow \vec{\boldsymbol{A}}_R \bullet \vec{\boldsymbol{H}}_L \in \mathbb{G}_T$; $B_R \leftarrow \vec{\boldsymbol{B}}_R \bullet \vec{\boldsymbol{G}}_L \in \mathbb{G}_T$;
  $\mathcal{P} \rightarrow \mathcal{V} : C_L, A_L, B_L, A_R, B_R \in \mathbb{G}_T$;
  $\mathcal{V} \rightarrow \mathcal{P} : z \xleftarrow{\$} \mathbb{Z}_p^*$;
  $\mathcal{P}$ computes:
   $\vec{\boldsymbol{A}}' \leftarrow \vec{\boldsymbol{A}} \oplus z^2\vec{\boldsymbol{G}} \oplus z^4(\vec{\boldsymbol{A}}_L || \vec{\boldsymbol{O}}) \oplus z^{10}(\vec{\boldsymbol{O}} || \vec{\boldsymbol{G}}_L) \in \mathbb{G}_1^{n/2^j}$;
   $\vec{\boldsymbol{B}}' \leftarrow \vec{\boldsymbol{B}} \oplus z\vec{\boldsymbol{H}} \oplus z^4(\vec{\boldsymbol{B}}_L || \vec{\boldsymbol{O}}) \oplus z^9(\vec{\boldsymbol{O}} || \vec{\boldsymbol{H}}_L) \in \mathbb{G}_2^{n/2^j}$;
  $\mathcal{P}$ and $\mathcal{V}$ update:
   $C' \leftarrow C \oplus zA \oplus z^2B \oplus z^3\Delta \oplus z^5A_L \oplus z^6B_L \oplus z^9A_R \oplus z^{10}B_R$
   $\oplus (2z^4 + z^8)C_L \oplus (2z^{11} + z^{19})\Delta^{(j)} \in \mathbb{G}_T$;
   $A'_L \leftarrow (1 + z^4)A_L \oplus z^2\Delta^{(j)}$; $A'_R \leftarrow A_R \oplus z^2\Delta_{GR}^{(j)} \oplus z^{10}\Delta^{(j)} \in \mathbb{G}_T$;
   $B'_L \leftarrow (1 + z^4)B_L \oplus z\Delta^{(j)}$; $B'_R \leftarrow B_R \oplus z\Delta_{HR}^{(j)} \oplus z^9\Delta^{(j)} \in \mathbb{G}_T$;
  $\mathcal{P}$ computes: $L \leftarrow \vec{\boldsymbol{A}}'_L \bullet \vec{\boldsymbol{B}}'_R \in \mathbb{G}_T$; $R \leftarrow \vec{\boldsymbol{A}}'_R \bullet \vec{\boldsymbol{B}}'_L \in \mathbb{G}_T$;
  $\mathcal{P} \rightarrow \mathcal{V} : L, R \in \mathbb{G}_T$;
  $\mathcal{V} \rightarrow \mathcal{P} : x \xleftarrow{\$} \mathbb{Z}_p^*$;
  $\mathcal{P}$ computes: $\vec{\boldsymbol{A}}'' \leftarrow \vec{\boldsymbol{A}}'_L \oplus x^{-1}\vec{\boldsymbol{A}}'_R \in \mathbb{G}_1^{n/2^j}$; $\vec{\boldsymbol{B}}'' \leftarrow \vec{\boldsymbol{B}}'_L \oplus x\vec{\boldsymbol{B}}'_R \in \mathbb{G}_2^{n/2^j}$;
  $\mathcal{P}$ and $\mathcal{V}$ update:
   $C'' \leftarrow C' \oplus xL \oplus x^{-1}R \in \mathbb{G}_T$;
   $A'' \leftarrow A'_L \oplus x^{-1}A'_R \in \mathbb{G}_1$; $B'' \leftarrow B'_L \oplus xB'_R \in \mathbb{G}_2$;
  // Update values for the next iteration
  $\Delta \leftarrow \Delta^{(j)} \in \mathbb{G}_T$; $\vec{\boldsymbol{G}} \leftarrow \vec{\boldsymbol{G}}_L \in \mathbb{G}_1^{n/2^j}$; $\vec{\boldsymbol{H}} \leftarrow \vec{\boldsymbol{H}}_L \in \mathbb{G}_2^{n/2^j}$;
  $C \leftarrow C''$; $A \leftarrow A''$; $B \leftarrow B'' \in \mathbb{G}_T$; $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{A}}'' \in \mathbb{G}_1^{n/2^j}$; $\vec{\boldsymbol{B}} \leftarrow \vec{\boldsymbol{B}}'' \in \mathbb{G}_2^{n/2^j}$;
// At this point, $\vec{\boldsymbol{A}}$ and $\vec{\boldsymbol{B}}$ have been reduced to single elements
$\mathcal{P} \rightarrow \mathcal{V} : \hat{A} \leftarrow \vec{\boldsymbol{A}} \in \mathbb{G}_1$; $\hat{B} \leftarrow \vec{\boldsymbol{B}} \in \mathbb{G}_2$;
$\mathcal{V}$ checks: $C \stackrel{?}{=} e(\hat{A}, \hat{B})$; $A \stackrel{?}{=} e(\hat{A}, H_0)$; $B \stackrel{?}{=} e(G_0, \hat{B})$;

Algorithm 6 Public-Inner-Product Argument (Transparent Setup)

Input: Public input: $V_a \in \mathbb{G}_1, V_b \in \mathbb{G}_2, \vec{\boldsymbol{a}} \in \mathbb{Z}_p^n, \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n$;
$\hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2, \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n$;
$\mathcal{V}$'s Precomputation: $\Delta \in \mathbb{G}_T, \Delta^{(j)}, j \in [1, \log_2 n]$,
Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

1 $\mathcal{P}$ and $\mathcal{V}$ run four inner-pairing-product arguments (Algorithm 5) in parallel, with the same random challenges $x, z \in \mathbb{Z}_p^*$ at Lines 4 and 9 in each iteration across the four protocols, for the following inner products:
① $\Delta = \vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{H}}$, where $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{G}}, \vec{\boldsymbol{B}} \leftarrow \vec{\boldsymbol{H}}$, with bases $\vec{\boldsymbol{H}}$ and $\vec{\boldsymbol{G}}$;
② $O = \vec{\boldsymbol{O}} \bullet \vec{\boldsymbol{O}}$, where $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{O}}, \vec{\boldsymbol{B}} \leftarrow \vec{\boldsymbol{O}}$, with bases $\vec{\boldsymbol{H}}$ and $\vec{\boldsymbol{G}}$;
③ $e(V_a, \hat{H}_0) = (\vec{\boldsymbol{a}}\hat{H}_0) \bullet \vec{\boldsymbol{G}}$, where $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{G}}, \vec{\boldsymbol{B}} \leftarrow \vec{\boldsymbol{a}}\hat{H}_0$, with bases $\vec{\boldsymbol{H}}$ and $\vec{\boldsymbol{O}}$;
④ $e(\hat{G}_0, V_a) = (\vec{\boldsymbol{b}}\hat{G}_0) \bullet \vec{\boldsymbol{H}}$ where $\vec{\boldsymbol{A}} \leftarrow \vec{\boldsymbol{b}}\hat{G}_0, \vec{\boldsymbol{B}} \leftarrow \vec{\boldsymbol{H}}$, with bases $\vec{\boldsymbol{O}}$ and $\vec{\boldsymbol{G}}$;
If any of these protocols return FALSE, **return** FALSE;

2 At the end of Algorithm 5, the verifier obtains four pairs of group elements:
① $\hat{A}_①, \hat{B}_①$; ② $\hat{A}_②, \hat{B}_②$; ③ $\hat{A}_③, \hat{B}_③$; ④ $\hat{A}_④, \hat{B}_④$;
The verifier checks:
$\hat{A}_③ \stackrel{?}{=} \hat{A}_① \ominus \hat{A}_② \ \wedge \ \hat{B}_③ \stackrel{?}{=} \hat{B}_② \oplus [\ \prod_{j=1}^{\log_2 n}(1 + z_{(j)}^4 + x_{(j)} \cdot x_{a(j)})\ ]\hat{H}_0$
$\hat{B}_④ \stackrel{?}{=} \hat{B}_① \ominus \hat{B}_② \ \wedge \ \hat{A}_④ \stackrel{?}{=} \hat{A}_② \oplus [\ \prod_{j=1}^{\log_2 n}(1 + z_{(j)}^4 + x_{(j)}^{-1} \cdot x_{b(j)})\ ]\hat{G}_0$;

3 **return** TRUE if all above return TRUE;

Proposition 7 *Algorithm* 6 *is an argument for* $\mathcal{L}_{\mathsf{pip}}$.

Proof of Proposition 7 is provided in Appendix C.6.

Adding zero-knowledge

We introduce a technique termed the *hiding transcript* to add zero knowledge to our protocols. This method involves converting each transcript element into a hiding commitment using a base $\tilde{U} \in \mathbb{G}_T$. For instance, in Algorithm 3, the prover can convert $L_{(j)}, R_{(j)} \in \mathbb{G}_T$ into hiding commitments by randomly choosing $\tilde{l}_{(j)}, \tilde{r}_{(j)} \stackrel{\$}{\leftarrow} \mathbb{Z}_p$, and then send:

$$\tilde{L}_{(j)} = L_{(j)} \oplus \tilde{l}_{(j)}\tilde{U} \in \mathbb{G}_T, \quad \tilde{R}_{(j)} = R_{(j)} \oplus \tilde{r}_{(j)}\tilde{U} \in \mathbb{G}_T.$$

For field transcript elements $\hat{a}, \hat{b} \in \mathbb{Z}_p$, the prover uses the base $U = \hat{G}_0 \otimes \hat{H}_0 \in \mathbb{G}_T$ and $\tilde{U} \in \mathbb{G}_T$. They randomly select $\tilde{a}, \tilde{b} \stackrel{\$}{\leftarrow} \mathbb{Z}_p$ and send:

$$\tilde{A} = \hat{a}U \oplus \tilde{a}\tilde{U} \in \mathbb{G}_T, \quad \tilde{B} = \hat{b}U \oplus \tilde{b}\tilde{U} \in \mathbb{G}_T.$$

Both parties can then use Algorithm 7 to replace the verification check in Line 8 of Algorithm 3, achieving a zero-knowledge argument of knowledge for the following inner-product relation.

$$\mathcal{R}_{\mathsf{zkIpGt}} = \left\{ \begin{pmatrix} C \in \mathbb{G}_T; \tilde{U} \in \mathbb{G}_T; \\ \hat{G}_0 \in \mathbb{G}_1, \hat{H}_0 \in \mathbb{G}_2; \\ \vec{\boldsymbol{G}} \in \mathbb{G}_1^n, \vec{\boldsymbol{H}} \in \mathbb{G}_2^n; \end{pmatrix} : \begin{pmatrix} \vec{\boldsymbol{a}} \in \mathbb{Z}_p^n, \\ \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n, \\ \tilde{c} \in \mathbb{Z}_p \end{pmatrix} \middle| \begin{pmatrix} C = \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{b}} \,|\, \hat{H}_0 \,\rangle \\ \quad \oplus \langle\, \hat{G}_0 \,|\, \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{H}} \,\rangle \\ \quad \oplus \langle\, \vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{b}} \,|\, \hat{H}_0 \,\rangle \oplus \tilde{c}\tilde{U} \end{pmatrix} \right\}. \tag{13}$$

Algorithm 7 Zero-Knowledge Verification Check for Algorithm 3

Input: Public input: $\tilde{C}, \tilde{A}, \tilde{B}, U, \tilde{U} \in \mathbb{G}_T$; $\vec{\boldsymbol{G}}, \vec{\boldsymbol{G}}' \in \mathbb{G}_1^n$; $\vec{\boldsymbol{H}}, \vec{\boldsymbol{H}}' \in \mathbb{G}_2^n$; $\tilde{L}_{(j)}, \tilde{R}_{(j)} \in \mathbb{G}_T, j \in [1, \log_2 n]$; $x_{(1)}, \cdots, x_{(\log_2 n)} \in \mathbb{Z}_p^*$;
$\mathcal{P}$'s private input: $\tilde{c}, \tilde{l}_{(j)}, \tilde{r}_{(j)} \in \mathbb{Z}_p, j \in [1, \log_2 n]$
Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

$\mathcal{P}$ computes: $\vec{\boldsymbol{\xi}}, \vec{\boldsymbol{\xi}}^{-1} \in \mathbb{Z}_p^n$ from $x_{(1)}, \cdots, x_{(\log_2 n)} \in \mathbb{Z}_p^*$;
$V_H \leftarrow \vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{H}} \in \mathbb{G}_2$; $V_G \leftarrow \vec{\boldsymbol{\xi}}^{-1} \bullet \vec{\boldsymbol{G}} \in \mathbb{G}_1$;
$\mathcal{V} \rightarrow \mathcal{P}$: $V_H \in \mathbb{G}_2, V_G \in \mathbb{G}_1$;
$\mathcal{P}$ and $\mathcal{V}$ compute:
$C_H \leftarrow e(\hat{G}_0, V_H) \in \mathbb{G}_T$; $C_G \leftarrow e(V_G, \hat{H}_0) \in \mathbb{G}_T$;
$\mathcal{P}$ computes:
$\tilde{c}_U, \tilde{a}_V, \tilde{b}_V \xleftarrow{\$} \mathbb{Z}_p^*$; $\tilde{C}_U \leftarrow \hat{a}\hat{b}U \oplus \tilde{c}_U\tilde{U} \in \mathbb{G}_T$;
$\tilde{A}_V \leftarrow \hat{a}C_H \oplus \tilde{a}_V\tilde{U} \in \mathbb{G}_T$; $\tilde{B}_V \leftarrow \hat{b}C_G \oplus \tilde{b}_V\tilde{U} \in \mathbb{G}_T$;
$\mathcal{V} \rightarrow \mathcal{P}$: $\tilde{C}_U, \tilde{A}_V, \tilde{B}_V \in \mathbb{G}_T$;
$\mathcal{P}$ and $\mathcal{V}$ run $\mathbb{G}_2$-public-inner-product argument on input:
$(C_H; (x_{(1)}, \cdots, x_{(\log_2 n)}); \hat{G}_0, \hat{H}_0; \vec{\boldsymbol{G}}, \vec{\boldsymbol{G}}', \vec{\boldsymbol{H}}, \vec{\boldsymbol{H}}')$;
$\mathcal{P}$ and $\mathcal{V}$ run $\mathbb{G}_1$-public-inner-product argument on input:
$(C_G; (x_{(1)}^{-1}, \cdots, x_{(\log_2 n)}^{-1}); \hat{G}_0, \hat{H}_0; \vec{\boldsymbol{G}}, \vec{\boldsymbol{G}}', \vec{\boldsymbol{H}}, \vec{\boldsymbol{H}}')$;
$\mathcal{P}$ and $\mathcal{V}$ run zero-knowledge scalar-multiplication argument on input:
$(\tilde{C}_U, \tilde{A}, \tilde{B}; U, \tilde{U} : \hat{a}, \hat{b}; \tilde{c}_U, \tilde{a}, \tilde{b})$;
$\mathcal{P}$ and $\mathcal{V}$ run zero-knowledge semi-scalar-mulitplication argument on input:
$(\tilde{A}_V, \tilde{A}, C_H; U, \tilde{U} : \hat{a}; \tilde{a}_V, \tilde{a})$;
$\mathcal{P}$ and $\mathcal{V}$ run zero-knowledge semi-scalar-multiplication argument on input:
$(\tilde{B}_V, \tilde{B}, C_G; U, \tilde{U} : \hat{b}; \tilde{b}_V, \tilde{b})$;
$\mathcal{P}$ and $\mathcal{V}$ compute:
$F \leftarrow \{\tilde{C} \oplus \bigoplus_{j=1}^{\log_2 n}(x_{(j)}\tilde{L}_{(j)} \oplus x_{(j)}^{-1}\tilde{R}_{(j)})\} \ominus \{\tilde{C}_U \oplus \tilde{A}_V \oplus \tilde{B}_V\}$;
$\mathcal{P}$ computes:
$\tilde{f} \leftarrow \{\tilde{c} + \sum_{j=1}^{\log_2 n}(x_{(j)}\tilde{l}_{(j)} + x_{(j)}^{-1}\tilde{r}_{(j)})\} - \{\tilde{c}_U + \tilde{a}_V + \tilde{b}_V\}$;
$\mathcal{P}$ and $\mathcal{V}$ run Schnorr's protocol on input: $(F; \tilde{U} : \tilde{f})$

Within Algorithm 7, the zero-knowledge scalar-multiplication argument is a zero-knowledge argument for the following relation:

$$\mathcal{R}_{\text{scalar}} = \left\{ \begin{pmatrix} C_c, C_a, C_b, \\ U, \tilde{U} \in \mathbb{G}_T \end{pmatrix} : \begin{pmatrix} a, b, \tilde{c}, \\ \tilde{a}, \tilde{b} \in \mathbb{Z}_p \end{pmatrix} \middle| \begin{pmatrix} C_c = (ab)U \oplus \tilde{c}\tilde{U} \wedge \\ C_a = aU \oplus \tilde{a}\tilde{U} \wedge C_b = bU \oplus \tilde{b}\tilde{U} \end{pmatrix} \right\}. \tag{14}$$

Table 2 Benchmarking Performance on Dense Matrices of Size $1,024 \times 1,024$[a]

	`DualMatrix`	Thaler's[b]	zkMatrix[c]	Libra[d]	Spartan[e]	HyperPlonk[e]
Setup time	1.94s	NA	419.54s	26.93s	427.23ms	48.22s
SRS size	361.61KB	NA	801.11MB	2.06GB	1.44MB	3.22GB
Preprocessing[f]	23.38s	NA	17.45s	6.76s	23.47s	87.46s
Prover time	5.63s	77.54ms	33.42s	258.63s	357.49s	149.91s
Verifier time	469.50ms	942.78ms	78.87ms	150.38ms	682.00ms	25.32ms
Transcript size	131.13KB	168B	7.85KB	48.00KB	501.35KB	17.96KB

[a] All experiments were conducted on the same machine, utilizing 64 threads for parallelization. `DualMatrix`, Spartan, and HyperPlonk are implemented in Rust, whereas Thaler's protocol and Libra are implemented in C++. We exclude the time for computing the underlying matrix multiplication from the prover times

[b] Thaler's protocol is designed for public matrices, meaning the verifier also holds the input matrices $\boldsymbol{a}$, $\boldsymbol{b}$, and $\boldsymbol{c}$. Consequently, there is no need for setup or preprocessing. However, its performance metrics are not comparable with those of other protocols.

[c] The performance metrics of zkMatrix are computed from the theoretical performance reported in Cong et al. (2024) and measurements on execution times of single group and field operations on the same machine.

[d] Libra encountered a memory overflow on our machine when processing $1,024 \times 1,024$ matrices. Consequently, we report its performance on smaller 512×512 matrices.

[e] The performance metrics for Spartan and Hyperplonk are measured for a mock circuit with 2^{24} constraints, under the assumption of the ideal matrix-multiplication-to-circuit conversion. This should be considered as a lower bound for any practical implementation of the matrix multiplication circuit.

[f] For `DualMatrix` and zkMatrix, the preprocessing time refers to the time taken to commit to the input matrices. For Libra, Spartan, and HyperPlonk, it refers to the time taken to commit to the circuit-specific verification keys

The zero-knowledge semi-scalar-multiplication argument is for:

$$\mathcal{R}_{\mathsf{semiScalar}} = \left\{ \begin{pmatrix} C_c, C_a, B; \\ U, \tilde{U} \in \mathbb{G}_T \end{pmatrix} : \begin{pmatrix} a, \tilde{c}, \tilde{a} \\ \in \mathbb{Z}_p \end{pmatrix} \middle| \begin{pmatrix} C_c = aB \oplus \tilde{c}\tilde{U} \\ \wedge\, C_a = aU \oplus \tilde{a}\tilde{U} \end{pmatrix} \right\}. \tag{15}$$

The Schnorr's protocol is a zero-knowledge argument for:

$$\mathcal{R}_{\mathsf{schnorr}} = \left\{ (\tilde{F}; \tilde{U} \in \mathbb{G}_T) : (\tilde{f} \in \mathbb{Z}_p) | (\tilde{F} = \tilde{f}\tilde{U}) \right\}. \tag{16}$$

For brevity, pseudocode for these standard atomic protocols is omitted.

Proposition 8 *By converting each element in the transcript into a hiding commitment and using Algorithm* 7 *to replace the verification check in Line* 8 *of Algorithm* 3*, we establish a zero-knowledge inner product argument for the relation* $\mathcal{R}_{\mathsf{zkIpGt}}$ *in Eq.* (13).

Proof of Proposition 8 is detailed in Appendix C.7.

Through analogous methodologies, we can transform Algorithms 1, 2, and 4 to zero-knowledge protocols. For brevity, we omit these details. Consequently, we arrive at a zkSNARK protocol for $\mathcal{R}_{\mathsf{dualMatrix}}$ in Eq. (2).

A significant advantage of our method for adding zero-knowledge is its efficiency. Our technique adds SHVZK to Bulletproof with only a logarithmic increase in prover's complexity and a constant increase in the transcript size.

Evaluation

We conducted the experiment on a server equipped with 104 Intel(R) Xeon(R) Gold 5320 CPUs, each at 2.20GHz, and 503GB of RAM. To ensure that our results can be reproduced on less powerful machines, we limited the use to 64 threads. We employ the BLS12-381 curve as the pairing groups and adopt its scalar field as $\mathbb{Z}_p$.

Moderately-sized matrices

For benchmarking purposes, we evaluated the performance of `DualMatrix` on $1,024 \times 1,024$ dense matrices, comparing it against Thaler's protocol,[1] zkMatrix, Libra,[2] Spartan,[3] and HyperPlonk.[4] Our implementation, along with zkMatrix, Libra, and HyperPlonk, utilizes universal trusted setups, whereas Spartan employs a transparent setup. Among these implementations, Thaler's implementation operates with public input matrices, resulting in the

[1] https://people.cs.georgetown.edu/jthaler/Tcode.htm

[2] https://github.com/sunblaze-ucb/Libra

[3] https://github.com/microsoft/Spartan

[4] https://github.com/EspressoSystems/hyperplonk

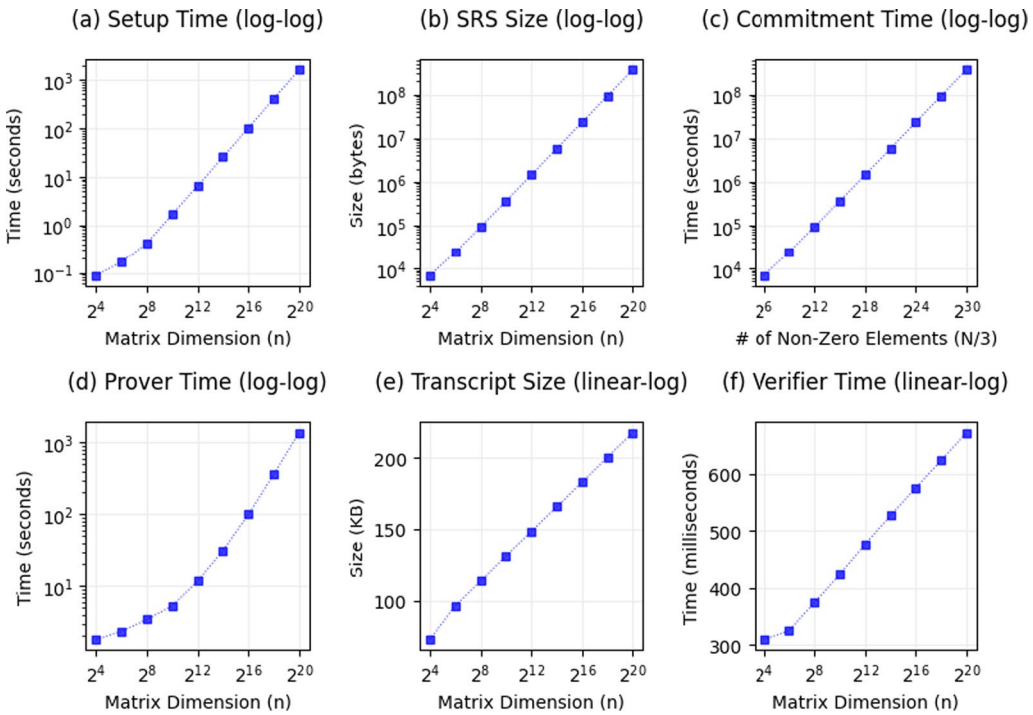

Fig. 1 Performance of `DualMatrix` for matrix multiplication tasks involving $n \times n$ sparse matrices. Each matrix contains $n^{1.5}$ non-zero integers. The cumulative count of non-zero elements across three input matrices is denoted by $N = 3n^{1.5}$. The time for matrix commitment increases linearly with the number of non-zero elements, leading to the horizontal axis of Subfigure (c) being set to $N/3$. For the other metrics the horizontal axes are set to n, reflecting its role as the determining factor

Table 3 Performance for Large Matrices Containing 1G Non-Zero Elements

	$2^{15} \times 2^{15}$ dense matrices	$2^{20} \times 2^{20}$ sparse matrices
Setup time	66.48s	1,613.11s
SRS size	11.54MB	369.10MB
Size of input data	$\boldsymbol{a}$: 25.77GB; $\boldsymbol{b}$: 25.77GB; $\boldsymbol{c}$: 34.36GB	$\boldsymbol{a}$: 25.77GB; $\boldsymbol{b}$: 25.77GB; $\boldsymbol{c}$: 34.36GB
Matrix multiplication time	17,901.42s	155.18s
Commitment time	$\boldsymbol{a}$: 5,546.88s; $\boldsymbol{b}$: 5,429.16s ; $\boldsymbol{c}$: 11,354.53s	$\boldsymbol{a}$: 5,705.66s; $\boldsymbol{b}$: 5,800.01s; $\boldsymbol{c}$: 12,439.43s
Size of cache data	$\boldsymbol{a}$: 3.67MB; $\boldsymbol{b}$: 2.10MB; $\boldsymbol{c}$: 2.10MB	$\boldsymbol{a}$: 117.44MB; $\boldsymbol{b}$: 67.11MB; $\boldsymbol{c}$: 67.11MB
Prover time	150.84s	1,384.45s
Verifier time	560.38ms	672.86ms
Transcript size	174.40KB	217.66KB

The experiments were conducted using 64 threads for parallelization. The runtimes for unverified matrix multiplication, executed on the same machine using the same parallelization, provides baselines for the prover times

fastest prover time and without the necessity for an SRS. Libra, which includes a built-in test case for matrix multiplication, encountered an overflow issue with matrices of size $1,024 \times 1,024$ on our machine. Consequently, we report its performance using smaller matrices of size 512×512. Spartan and HyperPlonk represent the state-of-the-art general-purpose zkSNARK implementations. Their performance metrics are measured on a mock circuit with 2^{24} constraints, reflecting the minimum circuit size assuming ideal matrix-multiplication-to-circuit conversion. Lacking an implementation for zkMatrix, the performance metrics of zkMatrix are computed from the theoretical cost reported in Cong et al. (2024) and the execution times of atomic group and field operations on our machine.

To prevent integer overflow from large integer multiplications, we restricted the integers in input matrices $\boldsymbol{a}$ and $\boldsymbol{b}$ to 52 bits using the i64 format. Consequently, integers in $\boldsymbol{c}$ are represented in the i128 format. The choice of 52 bits aligns with the mantissa length in the IEEE 754 standard for double-precision floating-point numbers. Notably, while the bit-length of input matrix elements marginally affects the prover and verifier times, it linearly influences the matrix commitment time.

We present the benchmarking results of $1,024 \times 1,024$ matrices in Table 2. The prover time for `DualMatrix` is 5.63s, making it 5.9x, 63.5x, and 26.6x faster than zkMatrix, Spartan, and HyperPlonk. The SRS size for `DualMatrix` is 2, 215x, 4x, and 8, 908x smaller than those of zkMatrix, Spartan, and HyperPlonk, respectively. These gaps are expected to increase with the size of the matrices. However, `DualMatrix` falls short in the transcript size and verifier time compared to zkMatrix, Libra, and HyperPlonk. For real-world matrix multiplication tasks, such a compromise for faster prover time is acceptable.

Large matrices

To reveal the capability of `DualMatrix` to handle large matrices, we conducted tests with $1G$ non-zero elements in each matrix. This corresponds to $2^{15} \times 2^{15}$ dense matrices. For sparse matrix evaluation, we used matrices of size $2^{20} \times 2^{20}$ with a sparsity ratio of $1/2^{10}$. Other zkSNARK implementations failed to handle matrices of this size. Thaler's protocol also encountered integer overflow issues with large matrices, which we believe are specific to the implementation. Unverified dense matrix multiplication of this size, using the naive method and parallelization across 64 threads, took 17, 901s.

For sparse matrices, we configured the input matrices of size $n \times n$ as follows:

$$\boldsymbol{a} = \begin{pmatrix} \boldsymbol{a}_{11} & \boldsymbol{a}_{12} & \cdots & \boldsymbol{a}_{1\sqrt{n}} \\ \boldsymbol{a}_{21} & \boldsymbol{a}_{22} & \cdots & \boldsymbol{a}_{2\sqrt{n}} \\ \vdots & \vdots & \cdots & \vdots \\ \boldsymbol{a}_{\sqrt{n}1} & \boldsymbol{a}_{\sqrt{n}2} & \cdots & \boldsymbol{a}_{\sqrt{n}\sqrt{n}} \end{pmatrix}, \boldsymbol{b} = \begin{pmatrix} \boldsymbol{b}_{11} & \boldsymbol{b}_{12} & \cdots & \boldsymbol{b}_{1\sqrt{n}} \\ \boldsymbol{b}_{21} & \boldsymbol{b}_{22} & \cdots & \boldsymbol{b}_{2\sqrt{n}} \\ \vdots & \vdots & \cdots & \vdots \\ \boldsymbol{b}_{\sqrt{n}1} & \boldsymbol{b}_{\sqrt{n}2} & \cdots & \boldsymbol{b}_{\sqrt{n}\sqrt{n}} \end{pmatrix}, \boldsymbol{c} = \boldsymbol{ab}.$$

Here, $\boldsymbol{a}_{ij}$ and $\boldsymbol{b}_{ij}$ are random diagonal matrices of size $\sqrt{n} \times \sqrt{n}$ with 52-bit integers in the i64 format. Elements of $\boldsymbol{c}$ are integers in i128 format. There are $n^{1.5}$ non-zero elements in each matrix. The total number of non-zero elements is $N = 3n^{1.5}$. The naive computation approach requires $\sqrt{n} \cdot \sqrt{n} = n$ integer multiplications for each block of $\boldsymbol{c}$. The computation of n blocks, therefore, involves n^2 integer multiplications.

In Table 3, we present the performance metrics for `DualMatrix` on these large matrices. For the dense matrix task, the prover and verifier times were 150.84s and 0.56s. For the sparse matrix task, the prover and verifier times were 1, 384.45s and 0.67s. Although the $O(N)$ field operations asymptotically dominate the prover time in theory, these $O(N)$ field multiplications require approximately 45s in our experiment for the dense matrix task and 130s for the sparse matrix task, both faster than the times required for the $O(n)$ exponentiations. As a result, the $O(n)$ exponentiations serve as the determining factor for prover times in practice. Throughout our experiments, the RAM usage remained consistently below 150GB. Out of this, 85.6GB were dedicated to storing the input matrices.

We illustrate the performance of `DualMatrix` on the sparse matrix task as n varies in Fig. 1. Among these metrics, commitment time scales linearly with N, leading us to set its horizontal axis to $N/3$. Other metrics are primarily influenced by n. This figure also highlights the scalability of our method, showing that the performance metrics align closely with theoretical predictions. The slight superlinearity observed in Subfigure (d) can be attributed to the $O(N)$ field operations that is superlinear relative to n. Both the transcript size and verifier time are logarithmic, indicating efficient scalability with respect to matrix size.

Conclusion

We have developed `DualMatrix` as a zkSNARK framework for matrix multiplication. `DualMatrix` is the first strictly linear zkSNARK protocol for matrix multiplication with respect to the number of non-zero elements of the input matrices. It features $O(n)$ SRS size, $O(N + n)$ RAM usage and prover time, and logarithmic transcript size and verifier time. By formalizing group matrix algebra and exploiting the rank-1 property of the $\mathbb{G}_T$ base matrix, we achieve significant improvements over Bulletproofs in both prover and verifier times. Our implementation of `DualMatrix` demonstrates excellent scalability, capable of handling matrices with up to 1 G non-zero elements.

Appendix A Properties of matrix operations

This paper utilizes properties of both field matrix and group matrix operations. These properties can be demonstrated through straightforward mathematical derivations.

Associative Property. For group matrices $\boldsymbol{A} \in \mathbb{G}_1^{m\times l}$ and $\boldsymbol{B} \in \mathbb{G}_2^{l\times n}$, and field matrices $\boldsymbol{a} \in \mathbb{Z}_p^{m'\times l'}$ and $\boldsymbol{b} \in \mathbb{Z}_p^{l'\times n'}$. Assuming dimension compatibility, the following associative property holds:

$$(\boldsymbol{aA})(\boldsymbol{Bb}) = \boldsymbol{a}(\boldsymbol{AB})\boldsymbol{b}, \quad (\boldsymbol{Aa})(\boldsymbol{bB}) = \boldsymbol{A}(\boldsymbol{ab})\boldsymbol{B}.$$

Distributive Property. For group matrices $\boldsymbol{A}$ and $\boldsymbol{B}$ over any of the groups $\mathbb{G}_1, \mathbb{G}_2$, or $\mathbb{G}_T$, and for field matrices $\boldsymbol{a}$ and $\boldsymbol{b}$, the following distributive property holds:

$$\begin{aligned}&\boldsymbol{a}(\boldsymbol{A}\oplus\boldsymbol{B}) = \boldsymbol{aA}\oplus\boldsymbol{aB}, \quad (\boldsymbol{A}\oplus\boldsymbol{B})\boldsymbol{b} = \boldsymbol{Ab}\oplus\boldsymbol{Bb},\\ &\boldsymbol{A}(\boldsymbol{a}+\boldsymbol{b}) = \boldsymbol{Aa}\oplus\boldsymbol{Ab}, \quad (\boldsymbol{a}+\boldsymbol{b})\boldsymbol{B} = \boldsymbol{aB}\oplus\boldsymbol{bB},\end{aligned}$$

If we also have group matrices $\boldsymbol{C}$ and $\boldsymbol{D}$ such that their products with $\boldsymbol{A}$ and $\boldsymbol{B}$ are well defined, then:

$$\boldsymbol{C}(\boldsymbol{A}\oplus\boldsymbol{B}) = \boldsymbol{CA}\oplus\boldsymbol{CB}, \quad (\boldsymbol{A}\oplus\boldsymbol{B})\boldsymbol{D} = \boldsymbol{AD}\oplus\boldsymbol{BD}.$$

Matrix Projection and Tensor Product. Given a matrix $\boldsymbol{a} \in \mathbb{Z}_p^{m\times n}$, consider the scalar projection $\vec{\boldsymbol{l}}^{\top}\boldsymbol{a}\vec{\boldsymbol{r}}$, where $\vec{\boldsymbol{l}}^{\top} \in \mathbb{Z}_p^m$ is a row vector and $\vec{\boldsymbol{r}} \in \mathbb{Z}_p^n$ is a column vector. This scalar projection can be equivalently expressed as the inner product between the vectorization of $\boldsymbol{a}$ and the vectorized tensor product $\vec{\boldsymbol{l}} \otimes \vec{\boldsymbol{r}}^{\top}$:

$$\vec{\boldsymbol{l}}^{\top}\boldsymbol{a}\vec{\boldsymbol{r}} = \mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{\boldsymbol{l}} \otimes \vec{\boldsymbol{r}}^{\top}) = \mathsf{vec}(\boldsymbol{a}^{\top}) \bullet \mathsf{vec}((\vec{\boldsymbol{l}} \otimes \vec{\boldsymbol{r}}^{\top})^{\top}). \tag{A1}$$

Dirac Notation and Tensor Product. Similarly, the braket operation applied to a matrix $\boldsymbol{a} \in \mathbb{Z}_p^{m\times n}$ can be expressed as the inner product of vectorized tensor products:

$$\begin{aligned}&\langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a} \,|\, \vec{\boldsymbol{H}} \rangle = \mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})\\ &= \mathsf{vec}(\boldsymbol{a}^{\top}) \bullet \mathsf{vec}((\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})^{\top}).\end{aligned} \tag{A2}$$

Dirac Notation on Tensor Product. Consider vectors $\vec{\boldsymbol{a}} \in \mathbb{Z}_p^m, \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n, \vec{\boldsymbol{G}} \in \mathbb{G}_1^m$, and $\vec{\boldsymbol{H}} \in \mathbb{G}_2^n$. The braket operation applied to the tensor product of $\vec{\boldsymbol{a}}$ and $\vec{\boldsymbol{b}}$ is equivalent to the product of their respective inner products:

$$\langle \vec{\boldsymbol{G}} \,|\, \vec{\boldsymbol{a}} \otimes \vec{\boldsymbol{b}}^{\top} \,|\, \vec{\boldsymbol{H}} \rangle = \vec{\boldsymbol{G}}^{\top}(\vec{\boldsymbol{a}}\vec{\boldsymbol{b}}^{\top})\vec{\boldsymbol{H}} = (\vec{\boldsymbol{a}}^{\top}\vec{\boldsymbol{G}})(\vec{\boldsymbol{b}}^{\top}\vec{\boldsymbol{H}}) = e(\vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}}, \vec{\boldsymbol{b}} \bullet \vec{\boldsymbol{H}}). \tag{A3}$$

Partition and Vectorization. Partitioning the vectorization of a matrix $\boldsymbol{a} \in \mathbb{Z}_p^{m\times n}$ corresponds to vectorizing the partitioned submatrices of $\boldsymbol{a}$:

$$\mathsf{vec}(\boldsymbol{a})_L = \begin{cases}\mathsf{vec}(\boldsymbol{a}_U), & \text{if} \quad m > 1,\\ \mathsf{vec}(\boldsymbol{a}_L), & \text{if} \quad m = 1;\end{cases} \quad \mathsf{vec}(\boldsymbol{a})_R = \begin{cases}\mathsf{vec}(\boldsymbol{a}_D), & \text{if} \quad m > 1,\\ \mathsf{vec}(\boldsymbol{a}_R), & \text{if} \quad m = 1.\end{cases} \tag{A4}$$

Similar properties hold for group matrices.

Partition and Transposition. The partition of the transposed matrix equals the transpose of the partitioned matrix.

$$\begin{aligned}&(\boldsymbol{a}_U)^{\top} = (\boldsymbol{a}^{\top})_L, \quad (\boldsymbol{a}_D)^{\top} = (\boldsymbol{a}^{\top})_R, \quad (\boldsymbol{a}_L)^{\top} = (\boldsymbol{a}^{\top})_U,\\ &(\boldsymbol{a}_R)^{\top} = (\boldsymbol{a}^{\top})_D.\end{aligned} \tag{A5}$$

Similar properties hold for group matrices.

Partition and Matrix Projection. The multiplication of a vector by a matrix, when partitioned, is equivalent to the multiplication of the vector with the corresponding partition of the matrix:

$$\begin{aligned}&(\vec{\boldsymbol{G}}^{\top}\boldsymbol{a})_L = \vec{\boldsymbol{G}}^{\top}\boldsymbol{a}_L, \ (\vec{\boldsymbol{G}}^{\top}\boldsymbol{a})_R = \vec{\boldsymbol{G}}^{\top}\boldsymbol{a}_R,\\ &(\boldsymbol{a}\vec{\boldsymbol{H}})_L = \boldsymbol{a}_U\vec{\boldsymbol{H}}, \ (\boldsymbol{a}\vec{\boldsymbol{H}})_R = \boldsymbol{a}_D\vec{\boldsymbol{H}}.\end{aligned} \tag{A6}$$

Similar properties hold for group matrices and field vectors.

Vectorization on Tensor Product. The vectorization of the tensor product between a row vector and a column vector is equal to the tensor product of two column vectors, or the transpose of the tensor product of two row vectors:

$$\mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top}) = \vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}} = (\vec{\boldsymbol{G}}^{\top} \otimes \vec{\boldsymbol{H}}^{\top})^{\top}. \tag{A7}$$

Similar properties hold for field vectors.

Transposition on Tensor Product. The transpose of the tensor product of two vectors is equivalent to the tensor product of their transposes, taken in reverse order:

$$(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})^{\top} = \vec{\boldsymbol{H}} \otimes \vec{\boldsymbol{G}}^{\top}. \tag{A8}$$

Similar properties hold for field vectors.

Partition on Tensor Product. Partitioning a tensor product is equivalent to taking the tensor product of the respective partitioned vectors. For example:

$$(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})_L = \vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}_L^{\top}, \qquad (\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top})_R = \vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}_R^{\top}. \tag{A9}$$

$$(\vec{\boldsymbol{H}} \otimes \vec{\boldsymbol{G}}^{\top})_U = \vec{\boldsymbol{H}}_L \otimes \vec{\boldsymbol{G}}^{\top}, \qquad (\vec{\boldsymbol{H}} \otimes \vec{\boldsymbol{G}}^{\top})_D = \vec{\boldsymbol{H}}_R \otimes \vec{\boldsymbol{G}}^{\top}. \tag{A10}$$

Similar properties hold for field vectors.

Kronecker Product and Matrix Multiplication. If $\boldsymbol{a}$ and $\boldsymbol{b}$ are field matrices, and $\boldsymbol{A}$ and $\boldsymbol{B}$ are group matrices, then:

$$(\boldsymbol{aA}) \otimes (\boldsymbol{bB}) = (\boldsymbol{a} \otimes \boldsymbol{b})(\boldsymbol{A} \otimes \boldsymbol{B}).$$

As a special case, when $\vec{\boldsymbol{a}}$ and $\vec{\boldsymbol{b}}$ are field vectors, we have:

$$\mathsf{vec}(\vec{\boldsymbol{a}} \otimes \vec{\boldsymbol{b}}^{\top}) \bullet \mathsf{vec}(\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}^{\top}) = e(\vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}}, \vec{\boldsymbol{b}} \bullet \vec{\boldsymbol{H}}). \tag{A11}$$

Appendix B Recapitulation of Bulletproofs

Bulletproofs (Bünz et al. 2018) offer an argument of knowledge for vectors $\vec{\boldsymbol{a}}, \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n$, achieving a communication complexity of $2\log_2 n$ group elements and 2 field elements for the following relation:

$$\mathcal{R}_{\mathsf{bullet}} = \left\{ \begin{pmatrix} C \in \mathbb{G}; U \in \mathbb{G}; \\ \vec{\boldsymbol{G}} \in \mathbb{G}^n, \vec{\boldsymbol{H}} \in \mathbb{G}^n \end{pmatrix} : \begin{pmatrix} \vec{\boldsymbol{a}} \in \mathbb{Z}_p^n, \\ \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n \end{pmatrix} \middle| \begin{pmatrix} C = (\vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{b}})U \\ \oplus \langle \vec{\boldsymbol{a}}, \vec{\boldsymbol{G}} \rangle \oplus \langle \vec{\boldsymbol{b}}, \vec{\boldsymbol{H}} \rangle \end{pmatrix} \right\}. \tag{B12}$$

In Algorithm 8, we present a modified version of Bulletproofs provided by Gentry et al. (2022), which requires fewer group operations.

Algorithm 8 Protocol of Bulletproof in Gentry et al. (2022)

Input: Public input: $C \in \mathbb{G}$; $U \in \mathbb{G}, \vec{\boldsymbol{G}}, \vec{\boldsymbol{H}} \in \mathbb{G}^n$;
$\mathcal{P}$'s private input: $\vec{\boldsymbol{a}}, \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n$

Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)

foreach $j \in [1, \log_2 n]$ **do**

  $\mathcal{P}$ computes:

  $(\vec{\boldsymbol{a}}_L || \vec{\boldsymbol{a}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{a}}$; $(\vec{\boldsymbol{b}}_L || \vec{\boldsymbol{b}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{b}}$;

  $(\vec{\boldsymbol{G}}_L || \vec{\boldsymbol{G}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{G}}$; $(\vec{\boldsymbol{H}}_L || \vec{\boldsymbol{H}}_R) \xleftarrow{\mathsf{half}} \vec{\boldsymbol{H}}$;

  $L_{(j)} \leftarrow (\vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{G}}_R) \oplus (\vec{\boldsymbol{b}}_R \bullet \vec{\boldsymbol{H}}_L) \oplus (\vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{b}}_R)U \in \mathbb{G}$;

  $R_{(j)} \leftarrow (\vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{G}}_L) \oplus (\vec{\boldsymbol{b}}_L \bullet \vec{\boldsymbol{H}}_R) \oplus (\vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{b}}_L)U \in \mathbb{G}$;

  $\mathcal{P} \to \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}$;

  $\mathcal{V} \to \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;

  $\mathcal{P}$ computes:

  $\vec{\boldsymbol{a}} \leftarrow \vec{\boldsymbol{a}}_L + x_{(j)}^{-1} \vec{\boldsymbol{a}}_R \in \mathbb{Z}_p^{n/2^j}$; $\vec{\boldsymbol{b}} \leftarrow \vec{\boldsymbol{b}}_L + x_{(j)} \vec{\boldsymbol{b}}_R \in \mathbb{Z}_p^{n/2^j}$;

  $\vec{\boldsymbol{G}} \leftarrow \vec{\boldsymbol{G}}_L \oplus x_{(j)} \vec{\boldsymbol{G}}_R \in \mathbb{G}^{n/2^j}$; $\vec{\boldsymbol{H}} \leftarrow \vec{\boldsymbol{H}}_L \oplus x_{(j)}^{-1} \vec{\boldsymbol{H}}_R \in \mathbb{G}^{n/2^j}$;

`// Now, both` $\vec{\boldsymbol{a}}$ `and` $\vec{\boldsymbol{b}}$ `have been reduced to scalars.`

$\mathcal{P} \to \mathcal{V}$: $\hat{a} \leftarrow \vec{\boldsymbol{a}} \in \mathbb{Z}_p$; $\hat{b} \leftarrow \vec{\boldsymbol{b}} \in \mathbb{Z}_p$;

$\mathcal{V}$ computes $\vec{\boldsymbol{\xi}} \in \mathbb{Z}_p^n$ and $\vec{\boldsymbol{\xi}}^{-1} \in \mathbb{Z}_p^n$ from $x_{(1)}, \cdots, x_{(\log_2 n)}$;

$\mathcal{V}$ checks: $C \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} L_{(j)} \oplus x_{(j)}^{-1} R_{(j)}) \stackrel{?}{=} \hat{a}\hat{b}U \oplus \hat{a}(\vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{G}}) \oplus \hat{b}(\vec{\boldsymbol{\xi}}^{-1} \bullet \vec{\boldsymbol{H}})$

Algorithm 9 Protocol of Semi-Bulletproof in Cong et al. (2024)

Input: Public input: $C \in \mathbb{G}, \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n$; $\vec{\boldsymbol{G}} \in \mathbb{G}^n$;
$\mathcal{P}$'s private input: $\vec{\boldsymbol{a}} \in \mathbb{Z}_p^n$
Output: TRUE ($\mathcal{V}$ accepts) or FALSE ($\mathcal{V}$ rejects)
foreach $j \in [1, \log_2 n]$ **do**
  $\mathcal{P}$ computes:
    $(\vec{\boldsymbol{a}}_L || \vec{\boldsymbol{a}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{a}}$; $(\vec{\boldsymbol{G}}_L || \vec{\boldsymbol{G}}_R) \xleftarrow{\text{half}} \vec{\boldsymbol{G}}$;
  $L_{(j)} \leftarrow (\vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{G}}_R) \oplus (\vec{\boldsymbol{a}}_L \bullet \vec{\boldsymbol{b}}_R) U \in \mathbb{G}$;
  $R_{(j)} \leftarrow (\vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{G}}_L) \oplus (\vec{\boldsymbol{a}}_R \bullet \vec{\boldsymbol{b}}_L) U \in \mathbb{G}$;
  $\mathcal{P} \to \mathcal{V}$: $L_{(j)}, R_{(j)} \in \mathbb{G}$;
  $\mathcal{V} \to \mathcal{P}$: $x_{(j)} \xleftarrow{\$} \mathsf{Coin}(\mathbb{Z}_p^*)$;
  $\mathcal{P}$ computes:
    $\vec{\boldsymbol{a}} \leftarrow \vec{\boldsymbol{a}}_L + x_{(j)}^{-1} \vec{\boldsymbol{a}}_R \in \mathbb{Z}_p^{n/2^j}$; $\vec{\boldsymbol{G}} \leftarrow \vec{\boldsymbol{G}}_L + x_{(j)} \vec{\boldsymbol{G}}_R \in \mathbb{G}^{n/2^j}$;
    $\vec{\boldsymbol{b}} \leftarrow \vec{\boldsymbol{b}}_L + x_{(j)} \vec{\boldsymbol{b}}_R \in \mathbb{Z}_p^{n/2^j}$;
// Now, $\vec{\boldsymbol{a}}$ has been reduced to scalars.
$\mathcal{P} \to \mathcal{V}$: $\hat{a} \leftarrow \vec{\boldsymbol{a}} \in \mathbb{Z}_p$;
$\mathcal{V}$ computes $\vec{\boldsymbol{\xi}} \in \mathbb{Z}_p^n$ from $x_{(1)}, \cdots, x_{(\log_2 n)}$;
$\mathcal{V}$ checks: $C \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} L_{(j)} \oplus x_{(j)}^{-1} R_{(j)}) \stackrel{?}{=} \hat{a}\hat{b}U \oplus \hat{a}(\vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{G}})$

A useful variation of Bulletproof is the semi-Bulletproof presented in Algorithm 9 for the following relation:

$$\mathcal{R}_{\mathsf{semiBullet}} = \left\{ \begin{pmatrix} C \in \mathbb{G}, \vec{\boldsymbol{b}} \in \mathbb{Z}_p^n; \\ U \in \mathbb{G}, \vec{\boldsymbol{G}} \in \mathbb{G}^n \end{pmatrix} : \left(\vec{\boldsymbol{a}} \in \mathbb{Z}_p^n \right) \middle| \; C = (\vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{b}}) U \oplus \langle \vec{\boldsymbol{a}}, \vec{\boldsymbol{G}} \rangle \right\}. \tag{B13}$$

This semi-Bulletproof protocol is essentially the same as Algorithm 10 of the full version of Yuen et al. (2021), where the security proof is provided.

Assumption for Bulletproofs. The computational knowledge soundness property of Bulletproofs relies solely on the discrete logarithm relation assumption in Assumption 4. This assumption states that there is no non-trivial logarithmic relation among the elements within the set:

$$\{U\} \cup \{\vec{\boldsymbol{G}}\} \cup \{\vec{\boldsymbol{H}}\} \subset \mathbb{G}.$$

Appendix C Security proofs

C.1 Proof of Lemma 2

Absence of Non-Trivial Logarithm Relation in $\mathbb{G}_T$. Define:

$$\vec{\boldsymbol{A}} = (\vec{\boldsymbol{G}}, \hat{G}_0, \hat{G}_{-1}).$$

If an adversary can find a non-trivial logarithm relation among the $\mathbb{G}_T$ elements, i.e. it finds $\boldsymbol{a}_{11}, \vec{\boldsymbol{a}}_{10}, \vec{\boldsymbol{a}}_{01}, a_{00}$, and a_{-10} such that:

$$\boldsymbol{a}_{11} \bullet (\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}) \oplus \vec{\boldsymbol{a}}_{10} \bullet (\vec{\boldsymbol{G}} \otimes \hat{H}_0) \oplus \vec{\boldsymbol{a}}_{01} \bullet (\hat{G}_0 \otimes \vec{\boldsymbol{H}}) \oplus a_{00} (\hat{G}_0 \otimes \hat{H}_0) \oplus a_{-10} (\hat{G}_{-1} \otimes \hat{H}_0) = O.$$

When organized in matrix form, this implies:

$$\vec{A}^{\top}\begin{pmatrix} a_{11} & \vec{a}_{01} & \vec{0} \\ \vec{a}_{10}^{\top} & a_{00} & 0 \\ a_{-10} & 0 & 0 \end{pmatrix}\begin{pmatrix} \vec{H} \\ \hat{H}_0 \\ O \end{pmatrix} = O, \quad \text{where} \quad \begin{pmatrix} a_{11} & \vec{a}_{01} & \vec{0} \\ \vec{a}_{10}^{\top} & a_{00} & 0 \\ a_{-10} & 0 & 0 \end{pmatrix}\begin{pmatrix} \vec{H} \\ \hat{H}_0 \\ O \end{pmatrix} := \vec{B}.$$

Hence, $\mathcal{A}$ finds a non-trivial $\vec{B} \in \mathbb{G}_2^n$ such that: $\vec{A} \bullet \vec{B} = O$. This constitutes an instance of Lemma 1 in Appendix 3.3.2. Consequently, the Pedersen vector commitment upholds its binding and hiding properties. □

C.2 Proof of Proposition 2

Scalar-Projection Argument.

Equivalence of Relations. We demonstrate the equivalence of $\mathcal{R}_{\text{semiBullet}}$ and $\mathcal{R}_{\text{scalar}}$ as follows:

$$\begin{aligned}
&\#[\text{semiBullet}\,]\langle \vec{a}, \vec{G}\rangle \Rrightarrow \mathsf{vec}(\boldsymbol{a}^{\top}) \bullet \mathsf{vec}([\vec{H} \otimes \vec{G}^{\top}] \oplus [(\vec{r}\hat{G}_0) \otimes (\vec{l}\hat{H}_0)^{\top}]) \\
&= [\mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}((\vec{H} \otimes \vec{G}^{\top})^{\top})] \oplus [(\mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}((\vec{r} \otimes \vec{l}^{\top})^{\top}))(\hat{G}_0 \otimes \hat{H}_0)] \text{ by Eq.(A2)} \\
&= [\mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{G} \otimes \vec{H}^{\top})] \oplus [(\mathsf{vec}(\boldsymbol{a}) \bullet \mathsf{vec}(\vec{l} \otimes \vec{r}^{\top}))(\hat{G}_0 \otimes \hat{H}_0)] \qquad \text{by Eq.(A8)} \\
&= \langle \vec{G} \,|\, \boldsymbol{a} \,|\, \vec{H} \rangle \oplus \langle \hat{G}_0 \,|\, \vec{l}^{\top} \boldsymbol{a} \vec{r} \,|\, \hat{H}_0 \rangle \qquad \text{by Eq.(A1) and(A2).}
\end{aligned}$$

Equivalence of Transcripts. Next, we align the transcript elements of Algorithm 1 with Algorithm 9 post transformation.

Define $\vec{A}_{(j)}, \vec{H}_{(j)}, \vec{v}_{(j)}$, and $\vec{r}_{(j)}$ as the values of $\vec{A}, \vec{H}, \vec{v}$, and $\vec{r}$, respectively, at the j-th iteration (Line 6) of Algorithm 1 for $j \in [1, \log_2 n]$. Define $\vec{a}_{(j)}$, $\vec{G}_{(j)}$, and $\vec{l}_{(j)}$ as the values $\vec{a}$, $\vec{G}$ and $\vec{l}$ at the j-th iteration (Line 13) for $j \in [\log_2 n + 1, 2\log_2 n]$.

We introduce the auxiliary matrices $\hat{\boldsymbol{a}}_{(j)}$ and $\hat{\boldsymbol{U}}_{(j)}$ as follows:

$$\begin{aligned}
&\hat{\boldsymbol{a}}_{(0)} \leftarrow \boldsymbol{a}^{\top}, \quad \hat{\boldsymbol{a}}_{(j+1)} \leftarrow \hat{\boldsymbol{a}}_{(j)U} + x_{(j)}^{-1}\hat{\boldsymbol{a}}_{(j)D}, \quad j \in [0, \log_2 n - 1], \\
&\qquad \hat{\boldsymbol{a}}_{(j+1)} \leftarrow \hat{\boldsymbol{a}}_{(j)L} + x_{(j)}^{-1}\hat{\boldsymbol{a}}_{(j)R}, \quad j \in [\log_2 n, 2\log_2 n - 1]; \\
&\hat{\boldsymbol{U}}_{(0)} \leftarrow ([\vec{H} \otimes \vec{G}^{\top}] \oplus [(\vec{r}\hat{H}_0) \otimes (\vec{l}\hat{G}_0)^{\top}]), \\
&\qquad \hat{\boldsymbol{U}}_{(j+1)} \leftarrow \hat{\boldsymbol{U}}_{(j)U} + x_{(j)}\hat{\boldsymbol{U}}_{(j)D}, \quad j \in [0, \log_2 n - 1], \\
&\qquad \hat{\boldsymbol{U}}_{(j+1)} \leftarrow \hat{\boldsymbol{U}}_{(j)L} + x_{(j)}\hat{\boldsymbol{U}}_{(j)R}, \quad j \in [\log_2 n, 2\log_2 n].
\end{aligned} \tag{C14}$$

In this process of constructing the matrix series, for the initial $\log_2 n$ rounds, the matrices undergo horizontal folding. Subsequently, for the next $\log_2 n$ rounds, the vectors are folded vertically.

Through mathematical induction, we can establish that the vectors $\mathsf{vec}(\hat{\boldsymbol{a}}_{(j)})$ and $\mathsf{vec}(\hat{\boldsymbol{U}}_{(j)})$ thus constructed is equivalent to the outcomes computed in Line 7 of Algorithm 9 post-transformation. Specifically, for $j \in [1, 2\log_2 n]$, we have:

$$\mathsf{vec}(\hat{\boldsymbol{a}}_{(j)}) \Lleftarrow \#[\text{semiBullet}\,]\vec{\boldsymbol{a}}_{(j)}, \quad j \in [1, 2\log_2 n], \tag{C15}$$

$$\mathsf{vec}(\hat{\boldsymbol{U}}_{(j)}) \Lleftarrow \#[\text{semiBullet}\,]\vec{\boldsymbol{G}}_{(j)}, \quad j \in [1, 2\log_2 n]. \tag{C16}$$

These values also align with those computed in Line 6 of Algorithm 1. Utilizing Eq. (A6) and (A10) and applying mathematical induction, we obtain:

$$\vec{\boldsymbol{A}}_{(j)} = \langle \vec{\boldsymbol{G}} \,|\, \hat{\boldsymbol{a}}_{(j)}^{\top} \,|, \quad \vec{\boldsymbol{v}}_{(j)} = \vec{\boldsymbol{l}}^{\top}\hat{\boldsymbol{a}}_{(j)}^{\top}, \qquad j \in [1, \log_2 n], \tag{C17}$$

$$\left[\vec{\boldsymbol{H}}_{(j)} \otimes \vec{\boldsymbol{G}}\right]^{\top} \oplus \left[(\vec{\boldsymbol{r}}_{(j)}\hat{H}_0) \otimes (\vec{\boldsymbol{l}}\,\hat{G}_0)^{\top}\right] = \hat{\boldsymbol{U}}_{(j)}, \quad j \in [1, \log_2 n]. \tag{C18}$$

For $L_{(j)}$ in line 3 of Algorithm 1, we have:

$$
\begin{aligned}
&L_{(j)} = \langle \vec{A}_{(j)L} \,|\, \vec{H}_{(j)R} \rangle \oplus \langle\, \hat{G}_0 \,|\, \vec{v}_{(j)L} \bullet \vec{r}_{(j)R} \,|\, \hat{H}_0 \,\rangle \\
&= \langle\, (\vec{G}^{\top} \hat{a}_{(j)}{}^{\top})_L \,|\, \vec{H}_{(j)R} \rangle \oplus \langle\, \hat{G}_0 \,|\, (\vec{l}^{\top} \hat{a}_{(j)}{}^{\top})_L \bullet \vec{r}_{(j)R} \,|\, \hat{H}_0 \,\rangle && \text{by Eq.(C17)} \\
&= \langle\, \vec{G}^{\top} (\hat{a}_{(j)U})^{\top} \,|\, \vec{H}_{(j)R} \rangle \oplus \langle\, \hat{G}_0 \,|\, \vec{l}^{\top} (\hat{a}_{(j)U})^{\top} \vec{r}_{(j)R} \,|\, \hat{H}_0 \,\rangle && \text{by Eq.(A5) and(A6)} \\
&= \mathsf{vec}(\hat{a}_{(j)U}) \bullet \mathsf{vec}(\vec{H}_{(j)R} \otimes \vec{G}^{\top}) \\
&\quad \oplus \langle\, \hat{G}_0 \,|\, \mathsf{vec}(\hat{a}_{(j)U}) \bullet \mathsf{vec}(\vec{r}_{(j)R}^{\top} \otimes \vec{l}) \,|\, \hat{H}_0 \,\rangle && \text{by Eq.(A1) and (A2)} \\
&= \mathsf{vec}(\hat{a}_{(j)U}) \bullet \mathsf{vec}([\vec{H}_{(j)R} \otimes \vec{G}^{\top}] \oplus [(\vec{r}_{(j)R} \hat{H}_0) \otimes (\vec{l} \hat{G}_0)^{\top}]) \\
&= \mathsf{vec}(\hat{a}_{(j)})_L \bullet \mathsf{vec}([\vec{H}_{(j)} \otimes \vec{G}^{\top}] \oplus [(\vec{r}_{(j)} \hat{H}_0) \otimes (\vec{l} \hat{G}_0)^{\top}])_R && \text{by Eq.(A4) and(A10)} \\
&= \mathsf{vec}(\hat{a}_{(j)})_L \bullet \mathsf{vec}(\hat{U}_{(j)})_R && \text{by Eq.(C18)} \\
&\Leftarrow \#[\mathsf{semiBullet}\,] \langle \vec{a}_{(j)L}, \vec{G}_{(j)R} \rangle = \#[\mathsf{semiBullet}\,] L_{(j)} && \text{by Eq. (C16).}
\end{aligned}
$$

Similarly, $R_{(j)} \Leftarrow \#[\mathsf{semiBullet}\,] R_{(j)}, j \in [1, 2\log_2 n]$.

Similarly, $R_{(j)} \Leftarrow \#[\mathsf{semiBullet}\,] R_{(j)}, j \in [1, \log_2 n]$.

At Line 8 of Algorithm 1, the values are:

$$\vec{a} = a\vec{\xi}_H^{-1}, \quad H = \vec{\xi}_H \bullet \vec{H}, \quad r = \vec{\xi}_H \bullet \vec{r}. \tag{C19}$$

Following this, $\mathsf{vec}(\hat{a}_{(\log_2 n)})$ and $\mathsf{vec}(\hat{U}_{(\log_2 n)})$, the matrices defined in Eq. (C14), satisfy the following equivalence:

$$
\begin{aligned}
&\mathsf{vec}(\hat{a}_{(\log_2 n)}) = ((\vec{\xi}_H^{-1})^{\top} \hat{a}_{(0)})^{\top} = ((\vec{\xi}_H^{-1})^{\top} a^{\top})^{\top} = a\vec{\xi}_H^{-1} = \vec{a}, \\
&\mathsf{vec}(\hat{U}_{(\log_2 n)}) = (\vec{\xi}_H^{\top} \hat{U}_{(0)})^{\top} = (\vec{\xi}_H^{\top} ([\vec{H} \otimes \vec{G}^{\top}] \oplus [(\vec{r} \hat{H}_0) \otimes (\vec{l} \hat{G}_0)^{\top}]))^{\top} \\
&\qquad = [(\vec{\xi}_H \bullet \vec{H}) \otimes \vec{G}] \oplus [(\vec{\xi}_H \bullet \vec{r}) \hat{H}_0 \otimes (\vec{l} \hat{G}_0)] = [H \otimes \vec{G}] \oplus [(r\hat{H}_0) \otimes (\vec{l} \hat{G}_0)].
\end{aligned}
$$

Here, the RHS are determined by the values in Eq. (C19).

Then, for $j \in [\log_2 n + 1, 2\log_2 n]$, mathematical induction leads to:

$$\mathsf{vec}(\hat{a}_{(j)}) = \vec{a}_{(j)}, \quad \mathsf{vec}(\hat{U}_{(j)}) = [H \otimes \vec{G}_{(j)}] \oplus [(r\hat{H}_0) \otimes (\vec{l}_{(j)} \hat{G}_0)], j \in [\log_2 n + 1, 2\log_2 n].$$

Accordingly, for $L_{(j)}$ in Line 10 of Algorithm 1 and $j \in [\log_2 n + 1, 2\log_2 n]$, we have the following equivalence:

$$
\begin{aligned}
&L_{(j)} = \langle\, \vec{a}_{(j)L} \bullet \vec{G}_{(j)R} \,|\, H \,\rangle \oplus \langle\, \hat{G}_0 \,|\, r\vec{a}_{(j)L} \bullet \vec{l}_{(j)R} \,|\, \hat{H}_0 \,\rangle \\
&= \{\mathsf{vec}(\vec{a}_{(j)L}) \bullet \mathsf{vec}(\vec{G}_{(j)R} \otimes H)\} \oplus \{(\mathsf{vec}(\vec{a}_{(j)L}) \bullet \mathsf{vec}(\vec{l}_{(j)R} \otimes r)) e(\hat{G}_0, \hat{H}_0)\} \\
&= \mathsf{vec}(\vec{a}_{(j)})_L \bullet \mathsf{vec}([H \otimes \vec{G}_{(j)}] \oplus [(r\hat{H}_0) \otimes (\vec{l}_{(j)} \hat{G}_0)])_R \\
&= \mathsf{vec}(\hat{a}_{(j)})_L \bullet \mathsf{vec}(\hat{U}_{(j)})_R && \text{by Eq. (C17) and (C18)} \\
&\Leftarrow \#[\mathsf{semiBullet}\,] (\vec{a}_{(j)L} \bullet \vec{G}_{(j)R}) = \#[\mathsf{semiBullet}\,] L_{(j)} && \text{by Eq. (C15) and (C16).}
\end{aligned}
$$

Utilizing the computation rule of $\vec{\xi} \in \mathbb{Z}_p^{n^2}$ in Eq. (6) and direct mathematical calculations, we establish the following:

$$\mathsf{vec}(\vec{\xi}_H \otimes \vec{\xi}_G{}^{\top}) \Leftarrow \#[\mathsf{semiBullet}\,] \vec{\xi}, \tag{C20}$$

$$\mathsf{vec}(\vec{\xi}_H^{-1} \otimes (\vec{\xi}_G^{-1})^\top) \Leftarrow \#[\mathsf{semiBullet}]\,\vec{\xi}^{-1}, \quad \text{(C21)}$$

In Line 14 of Algorithm 1, we have:

$$\begin{aligned}
&\hat{a} = \vec{\xi}_G^{-1} \bullet \vec{a} = (\vec{\xi}_H^{-1})^\top \boldsymbol{a}^\top \vec{\xi}_G^{-1} = \mathsf{vec}(\boldsymbol{a}^\top) \bullet \mathsf{vec}(\vec{\xi}_H^{-1} \otimes (\vec{\xi}_G^{-1})^\top) && \text{by Eq.(A1)}\\
&\Leftarrow \#[\mathsf{bullet}]\,(\vec{\xi}^{-1} \bullet \vec{a}) = \#[\mathsf{bullet}]\,\hat{a} && \text{by Eq.(C21).}
\end{aligned}$$

We have demonstrated that the transcript elements generated by Algorithm 1 align with those from Algorithm 9 post-transformation.

Equivalence of Verification Checks. The RHS of the verification check in Line 16 of Algorithm 1 is shown to match the RHS in Line 10 of Algorithm 9, as follows:

$$\begin{aligned}
&\langle \hat{G}_0 \,|\, \hat{a}(\vec{\xi}_G \bullet \vec{l})(\vec{\xi}_H \bullet \vec{r}) \,|\, \hat{H}_0 \rangle \oplus \langle \vec{\xi}_G \bullet \vec{G} \,|\, \hat{a} \,|\, \vec{\xi}_H \bullet \vec{H} \rangle\\
&= [(\vec{\xi}_G{}^\top \vec{l}) \otimes (\vec{\xi}_H{}^\top \vec{r})(\hat{a}) e(\hat{G}_0, \hat{H}_0)] \oplus [\hat{a}(\vec{\xi}_G{}^\top \vec{G})(\vec{\xi}_H{}^\top \vec{H})]\\
&= \hat{a}[\vec{\xi}_G{}^\top \otimes \vec{\xi}_H{}^\top)((\vec{l}\hat{G}_0) \otimes (\vec{r}\hat{H}_0)] \oplus [\hat{a}(\vec{\xi}_G^\top \otimes \vec{\xi}_H^\top)(\vec{G} \otimes \vec{H})] && \text{by Eq.(A11)}\\
&= \hat{a}\{\mathsf{vec}(\vec{\xi}_G \otimes \vec{\xi}_H^\top) \bullet \mathsf{vec}((\vec{l}\hat{G}_0) \otimes (\vec{r}\hat{H}_0)^\top)\} \oplus \hat{a}\{\mathsf{vec}(\vec{\xi}_G \otimes \vec{\xi}_H^\top) \bullet \mathsf{vec}(\vec{G} \otimes \vec{H}^\top)\}\\
& && \text{by Eq.(A7)}\\
&= \hat{a}[\{\mathsf{vec}(\vec{\xi}_H \otimes \vec{\xi}_G^\top) \bullet \mathsf{vec}((\vec{r}\hat{H}_0) \otimes (\vec{l}\hat{G}_0)^\top)\} \oplus \{\mathsf{vec}(\vec{\xi}_H \otimes \vec{\xi}_G^\top) \bullet \mathsf{vec}(\vec{H} \otimes \vec{G}^\top)\}]\\
& && \text{by Eq.(A1), (A2) and(A8)}\\
&= \hat{a}\{\mathsf{vec}(\vec{\xi}_H \otimes \vec{\xi}_G^\top) \bullet \mathsf{vec}([\vec{H} \otimes \vec{G}^\top] \oplus [(\vec{r}\hat{H}_0) \otimes (\vec{l}\hat{G}_0)^\top])\}\\
&\Leftarrow \#[\mathsf{semiBullet}]\,\hat{a}(\vec{\xi} \bullet \vec{G}) && \text{by Eq.(C20)and the transformation rule.}
\end{aligned}$$

Thus, the verification check of Algorithm 9 is equivalent to that of Algorithm 1 post-transformation. Given the established argument of knowledge for $\mathcal{R}_{\mathsf{semiBullet}}$, it follows that Algorithm 1 serves as an argument of knowledge for $\mathcal{R}_{\mathsf{scalar}}$. □

C.3 Proof of Proposition 4

Inner-Product Argument in $\mathbb{G}_T$. We have:

$$\begin{aligned}
C &= \langle \hat{G}_0 \,|\, \vec{a} \bullet \vec{b} \,|\, \hat{H}_0 \rangle \oplus \langle \hat{G}_0 \,|\, \vec{a} \bullet \vec{H} \rangle \oplus \langle \vec{G} \bullet \vec{b} \,|\, \hat{H}_0 \rangle\\
&= \{(\vec{a} \bullet \vec{b})(\hat{G}_0 \otimes \hat{H}_0)\} \oplus \{\vec{a} \bullet (\hat{G}_0 \otimes \vec{H})\} \oplus \{(\vec{G} \otimes \hat{H}_0) \bullet \vec{b}\}\\
&\Leftarrow \#[\mathsf{bullet}]\,(\vec{a} \bullet \vec{b})U \oplus (\vec{a} \bullet \vec{G}) \oplus (\vec{b} \bullet \vec{H}) = \#[\mathsf{bullet}]\,C.
\end{aligned}$$

Hence, the relation $\mathcal{R}_{\mathsf{ipGt}}$ is transformed to $\mathcal{R}_{\mathsf{bullet}}$.

Similarly, we can demonstrate that Line 2 of Algorithm 3 is equivalent to Line 2 of Algorithm 8 post transformation. The verification check in Line 8 is equivalent to that of Algorithm 8. As a result, Algorithm 3 inherits the security proof of Algorithm 8. □

C.4 Proof of Proposition 5

Proof

Perfect completeness. If $\mathcal{P}$ *possesses a valid witness* $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$ *for* $\mathcal{R}_{\mathsf{matMul}}$ *such that* $\boldsymbol{c} = \boldsymbol{ab}$*, then for any* $y \in \mathbb{Z}_p$ *and* $x \in \mathbb{Z}_p$ *and Propositions 2-4: (1) the scalar projection argument returns TRUE for input* $\boldsymbol{c}$*; (2) the left projection argument returns TRUE for input* $\boldsymbol{a}$*; (3) the right projection argument returns TRUE for* $\boldsymbol{b}$*; (4) the inner-product argument in* $\mathbb{G}_T$ *returns TRUE for input* $x\boldsymbol{a}_y$ *and* $x^{-1}\boldsymbol{b}_y$*. Consequently, the overarching protocol returns TRUE.*

Computational knowledge soundness. We rewind the matrix-multiplication protocol $3n^2$ times and provide it with n^2 pairwise distinct challenges $y^{(\tau)}, \tau \in [1, n^2]$. For each $y^{(\tau)}$, we provide 3 pairwise-distinct challenges $x^{(\tau\iota)}, \iota = 1, 2, 3$.

Utilizing Propositions 2-4, we employ the extractors of the subprotocols to extract $\boldsymbol{a}^{(\tau\iota)}, \boldsymbol{b}^{(\tau\iota)}, \boldsymbol{c}^{(\tau\iota)}, C_d^{(\tau)}, C_{ay}^{(\tau)}, C_{by}^{(\tau)}$ $\boldsymbol{a}_y^{(\tau\iota)}$, and $\boldsymbol{b}_y^{(\tau\iota)}$ for $\tau \in [1, n^2], \iota = 1, 2, 3$:

$$x^{(\tau\iota)} C_d^{(\tau)} \oplus C_c = \langle \hat{G}_0 \,|\, x^{(\tau\iota)} (\vec{\boldsymbol{l}}^{(\tau)})^\top \boldsymbol{c}^{(\tau\iota)} \vec{\boldsymbol{r}}^{(\tau)} \,|\, \hat{H}_0 \rangle \oplus \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{c}^{(\tau\iota)} \,|\, \vec{\boldsymbol{H}} \rangle, \quad \text{(C22)}$$

$$x^{(\tau\iota)} C_{ay}^{(\tau)} \oplus C_a = \langle \hat{G}_0 \,|\, x^{(\tau\iota)} (\vec{\boldsymbol{l}}^{(\tau)})^\top \boldsymbol{a}^{(\tau\iota)} \,|\, \vec{\boldsymbol{H}} \rangle \oplus \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{a}^{(\tau\iota)} \,|\, \vec{\boldsymbol{H}} \rangle, \quad \text{(C23)}$$

$$x^{(\tau\iota)} C_{by}^{(\tau)} \oplus C_b = \langle \vec{\boldsymbol{G}} \,|\, x^{(\tau\iota)} \boldsymbol{b}^{(\tau\iota)} \vec{\boldsymbol{r}}^{(\tau)} \,|\, \hat{H}_0 \rangle \oplus \langle \vec{\boldsymbol{G}} \,|\, \boldsymbol{b}^{(\tau\iota)} \,|\, \vec{\boldsymbol{H}} \rangle, \quad \text{(C24)}$$

$$C_d^{(\tau)} \oplus x^{(\tau\iota)} C_{ay}^{(\tau)} \oplus (x^{(\tau\iota)})^{-1} C_{by}^{(\tau)} = \langle\, \hat{G}_0 \,|\, \vec{a}_y^{(\tau\iota)} \bullet \vec{b}_y^{(\tau\iota)} \,|\, \hat{H}_0 \,\rangle \oplus \langle\, \hat{G}_0 \,|\, x^{(\tau\iota)} \vec{a}_y^{(\tau\iota)} \bullet \vec{H} \,\rangle \oplus \langle\, (x^{(\tau\iota)})^{-1} \vec{G} \bullet \vec{b}_y^{(\tau\iota)} \,|\, \hat{H}_0 \,\rangle. \tag{C25}$$

By multiplying Eqs. (C22)-(C24) with $x^{(\iota 2)}$ and $x^{(\iota 1)}$ for $\iota = 1, 2$, and subsequently computing the difference, we obtain:

$$C_d^{(\tau)} = \langle\, \hat{G}_0 \,|\, (x^{(\tau 1)} - x^{(\tau 2)})^{-1} (\vec{\boldsymbol{l}}^{(\tau)})^\top (x^{(\tau 1)} \boldsymbol{c}^{(\tau 1)} - x^{(\tau 2)} \boldsymbol{c}^{(\tau 2)}) \vec{\boldsymbol{r}}^{(\tau)} \,|\, \hat{H}_0 \,\rangle \oplus \langle\, \vec{\boldsymbol{G}} \,|\, (x^{(\tau 1)} - x^{(\tau 2)})^{-1} (\boldsymbol{c}^{(\tau 1)} - \boldsymbol{c}^{(\tau 2)}) \,|\, \vec{\boldsymbol{H}} \,\rangle, \tag{C26}$$

$$C_{ay}^{(\tau)} = \langle\, \hat{G}_0 \,|\, (x^{(\tau 1)} - x^{(\tau 2)})^{-1} (\vec{\boldsymbol{l}}^{(\tau)})^\top (x^{(\tau 1)} \boldsymbol{a}^{(\tau 1)} - x^{(\tau 2)} \boldsymbol{a}^{(\tau 2)}) \,|\, \vec{\boldsymbol{H}} \,\rangle \oplus \langle\, \vec{\boldsymbol{G}} \,|\, (x^{(\tau 1)} - x^{(\tau 2)})^{-1} (\boldsymbol{a}^{(\tau 1)} - \boldsymbol{a}^{(\tau 2)}) \,|\, \vec{\boldsymbol{H}} \,\rangle, \tag{C27}$$

$$C_{by}^{(\tau)} = \langle\, \vec{\boldsymbol{G}} \,|\, (x^{(\tau 1)} - x^{(\tau 2)})^{-1} (x^{(\tau 1)} \boldsymbol{b}^{(\tau 1)} - x^{(\tau 2)} \boldsymbol{b}^{(\tau 2)}) \vec{\boldsymbol{r}}^{(\tau)} \,|\, \hat{H}_0 \,\rangle \oplus \langle\, \vec{\boldsymbol{G}} \,|\, (x^{(\tau 1)} - x^{(\tau 2)})^{-1} (\boldsymbol{b}^{(\tau 1)} - \boldsymbol{b}^{(\tau 2)}) \,|\, \vec{\boldsymbol{H}} \,\rangle. \tag{C28}$$

According to Lemma 2, no non-trivial logarithmic relation exists among the following elements of $\mathbb{G}_T$:

$$\{\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}\} \cup \{\vec{\boldsymbol{G}} \otimes \hat{H}_0\} \cup \{\hat{G}_0 \otimes \vec{\boldsymbol{H}}\} \cup \{\hat{G}_0 \otimes \hat{H}_0\} \cup \{\tilde{U}\} \subset \mathbb{G}_T.$$

By substituating Eqs. (C26)-(C28) into Eq. (C25) and comparing the coefficients of $\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}$, we assert:

$$(\boldsymbol{c}^{(\tau 1)} - \boldsymbol{c}^{(\tau 2)}) + x^{(\tau\iota)} (\boldsymbol{a}^{(\tau 1)} - \boldsymbol{a}^{(\tau 2)}) + (x^{(\tau\iota)})^{-1} (\boldsymbol{b}^{(\tau 1)} - \boldsymbol{b}^{(\tau 2)}) = \boldsymbol{0}. \tag{C29}$$

Here, we pad the matrices with zeros to ensure they are of the same size. Given that Eq. (C29) holds for $\iota = 1, 2, 3$, we obtain:

$$\boldsymbol{c}^{(\tau 1)} = \boldsymbol{c}^{(\tau 2)} := \boldsymbol{c}^{(\tau)}, \quad \boldsymbol{a}^{(\tau 1)} = \boldsymbol{a}^{(\tau 2)} := \boldsymbol{c}^{(\tau)}, \quad \boldsymbol{b}^{(\tau 1)} = \boldsymbol{b}^{(\tau 2)} := \boldsymbol{c}^{(\tau)}. \tag{C30}$$

Substituating Eq. (C30) back into Eqs. (C26)-(C28), we know:

$$C_d^{(\tau)} = \langle\, G_0 \,|\, (\vec{\boldsymbol{l}}^{(\tau)})^\top \boldsymbol{c}^{(\tau)} \vec{\boldsymbol{r}}^{(\tau)} \,|\, H_0 \,\rangle, \tag{C31}$$

$$C_{ay}^{(\tau)} = \langle\, \hat{G}_0 \,|\, (\vec{\boldsymbol{l}}^{(\tau)})^\top \boldsymbol{a}^{(\tau)} \,|\, \vec{\boldsymbol{H}} \,\rangle, \tag{C32}$$

$$C_{by}^{(\tau)} = \langle\, \vec{\boldsymbol{G}} \,|\, \boldsymbol{b}^{(\tau)} \vec{\boldsymbol{r}}^{(\tau)} \,|\, \hat{H}_0 \,\rangle. \tag{C33}$$

Substituting Eqs. (C31)-(C33) back to Eqs. (C26)-(C28) and compare the coefficients of $\vec{\boldsymbol{G}} \otimes \vec{\boldsymbol{H}}$ across different τ, we assert:

$$\boldsymbol{c}^{(\tau)} = \text{const} := \boldsymbol{c}, \ \boldsymbol{a}^{(\tau)} = \text{const} := \boldsymbol{a}, \ \boldsymbol{b}^{(\tau)} = \text{const} := \boldsymbol{b}, \ \tau \in [1, n^2]. \tag{C34}$$

Again, substituting Eq. (C34) into Eq. (C25) and comparing the coefficients of $\hat{G}_0 \otimes \hat{H}_0$, $\hat{G}_0 \otimes \vec{\boldsymbol{H}}$, and $\vec{\boldsymbol{G}} \otimes \hat{H}_0$, we deduce that:

$$(\vec{\boldsymbol{l}}^{(\tau)})^\top \boldsymbol{c} \vec{\boldsymbol{r}}^{(\tau)} = \vec{\boldsymbol{a}}_y^{(\tau\iota)} \bullet \vec{\boldsymbol{b}}_y^{(\tau\iota)}, \quad (\vec{\boldsymbol{l}}^{(\tau)})^\top \vec{\boldsymbol{a}} = \vec{\boldsymbol{a}}_y^{(\tau\iota)}, \quad \boldsymbol{b} \vec{\boldsymbol{r}}^{(\tau)} = \vec{\boldsymbol{b}}_y^{(\tau\iota)}.$$

This implies:

$$(\vec{\boldsymbol{l}}^{(\tau)})^\top \boldsymbol{c} \vec{\boldsymbol{r}}^{(\tau)} = (\vec{\boldsymbol{l}}^{(\tau)})^\top (\vec{\boldsymbol{a}} \boldsymbol{b}) \vec{\boldsymbol{r}}^{(\tau)}.$$

Note that $\vec{\boldsymbol{l}}^{\top} \boldsymbol{c} \vec{\boldsymbol{r}}$ is a polynomial in y with degree $n^2 - 1$, as detailed in Cong et al. (2024), where the coefficients are determined by $\boldsymbol{c}$. Similarly, $\vec{\boldsymbol{l}}^{\top} \boldsymbol{a} \boldsymbol{b} \vec{\boldsymbol{r}}$ can be expressed as a polynomial in of y with the same degree, with its coefficients derived from $\boldsymbol{ab}$. Given that the equation above is valid for n^2 distinct values of y, it follows that: $\boldsymbol{c} = \boldsymbol{ab}$. Therefore, the extracted witness $\boldsymbol{c}, \boldsymbol{a}, \boldsymbol{b}$ satisfies $\mathcal{R}_{\mathsf{matMul}}$, and we obtain the extractor $\mathcal{K}_{\mathsf{matMul}}$. □

C.5 Proof of Proposition 6

Proof

Perfect Completeness. Given:

$$C = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}}, A = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{H}}, B = \vec{\boldsymbol{B}} \bullet \vec{\boldsymbol{G}},$$
$$\Delta = \vec{\boldsymbol{G}} \bullet \vec{\boldsymbol{H}}, \Delta^{(j)} = \vec{\boldsymbol{G}}_L \bullet \vec{\boldsymbol{H}}_L,$$
$$C_L = \vec{\boldsymbol{A}}_L \bullet \vec{\boldsymbol{B}}_L, A_L = \vec{\boldsymbol{A}}_L \bullet \vec{\boldsymbol{H}}_L, B_L = \vec{\boldsymbol{B}}_L \bullet \vec{\boldsymbol{G}}_L,$$
$$A_R = \vec{\boldsymbol{A}}_R \bullet \vec{\boldsymbol{H}}_L, B_R = \vec{\boldsymbol{B}}_R \bullet \vec{\boldsymbol{G}}_L,$$

we derive:

$$\begin{aligned}
&\vec{A}' \bullet \vec{B}' \\
=&[\vec{A} \oplus z_{(j)}^2 \vec{G} \oplus z_{(j)}^4 (\vec{A}_L || \vec{0}) \oplus z_{(j)}^{10} (\vec{0} || \vec{G}_L)] \bullet [\vec{B} \oplus z_{(j)} \vec{H} \oplus z_{(j)}^4 (\vec{B}_L || \vec{0}) \oplus z_{(j)}^9 (\vec{0} || \vec{H}_L)] \\
=&\vec{A} \bullet \vec{B} \oplus z_{(j)} \vec{A} \bullet \vec{H} \oplus z_{(j)}^2 \vec{B} \bullet \vec{G} \oplus z_{(j)}^3 \vec{G} \bullet \vec{H} \oplus z_{(j)}^5 \vec{A}_L \bullet \vec{H}_L \oplus z_{(j)}^6 \vec{B}_L \bullet \vec{G}_L \\
&\oplus z_{(j)}^9 \vec{A}_R \bullet \vec{H}_L \oplus z_{(j)}^{10} \vec{B}_R \bullet \vec{G}_L \oplus (2z_{(j)}^4 + z_{(j)}^8) \vec{A}_L \bullet \vec{B}_L \oplus (2z_{(j)}^{11} + z_{(j)}^{19}) \vec{H}_L \bullet \vec{G}_L \\
=&C \oplus z_{(j)} A \oplus z_{(j)}^2 B \oplus z_{(j)}^3 \Delta \oplus z_{(j)}^5 A_L \oplus z_{(j)}^6 B_L \\
&\oplus z_{(j)}^9 A_R \oplus z_{(j)}^{10} B_R \oplus (2z_{(j)}^4 + z_{(j)}^8) C_L \oplus (2z_{(j)}^{11} + z_{(j)}^{19}) \Delta^{(j)},
\end{aligned}$$

$$\begin{aligned}
\vec{A'}_L \bullet \vec{H}_L =&[\vec{A}_L \oplus z_{(j)}^2 \vec{G}_L \oplus z_{(j)}^4 (\vec{A}_L || \vec{0})_L \oplus z_{(j)}^{10} (\vec{0} || \vec{G}_L)_L] \bullet \vec{H}_L \\
=&\vec{A}_L \bullet \vec{H}_L \oplus z_{(j)}^2 \vec{G}_L \bullet \vec{H}_L \oplus z_{(j)}^4 \vec{A}_L \bullet \vec{H}_L \\
=&(1 + z_{(j)}^4) \vec{A}_L \bullet \vec{H}_L \oplus z_{(j)}^2 \vec{G}_L \bullet \vec{H}_L = (1 + z_{(j)}^4) A_L \oplus z_{(j)}^2 \Delta^{(j)}, \\
\vec{A'}_R \bullet \vec{H}_L =&[\vec{A}_R \oplus z_{(j)}^2 \vec{G}_R \oplus z_{(j)}^4 (\vec{A}_L || \vec{0})_R \oplus z_{(j)}^{10} (\vec{0} || \vec{G}_L)_R] \bullet \vec{H}_L \\
=&\vec{A}_R \bullet \vec{H}_L \oplus z_{(j)}^2 \vec{G}_R \bullet \vec{H}_L \oplus z_{(j)}^{10} \vec{G}_L \bullet \vec{H}_L = A_R \oplus z_{(j)}^2 \Delta_{GR}^{(j)} \oplus z_{(j)}^{10} \Delta^{(j)}, \\
\vec{B'}_L \bullet \vec{G}_L =&[\vec{B}_L \oplus z_{(j)} \vec{H}_L \oplus z_{(j)}^4 (\vec{B}_L || \vec{0})_L \oplus z_{(j)}^9 (\vec{0} || \vec{H}_L)_L] \bullet \vec{G}_L \\
=&\vec{B}_L \bullet \vec{G}_L \oplus z_{(j)} \vec{G}_L \bullet \vec{H}_L \oplus z_{(j)}^4 \vec{B}_L \bullet \vec{G}_L \\
=&(1 + z_{(j)}^4) \vec{B}_L \bullet \vec{G}_L \oplus z_{(j)} \vec{G}_L \bullet \vec{H}_L = (1 + z_{(j)}^4) B_L \oplus z_{(j)} \Delta^{(j)}, \\
\vec{B'}_R \bullet \vec{G}_L =&[\vec{B}_R \oplus z_{(j)} \vec{H}_L \oplus z_{(j)}^4 (\vec{B}_L || \vec{0})_R \oplus z_{(j)}^9 (\vec{0} || \vec{H}_L)_R] \bullet \vec{G}_L \\
=&\vec{B}_R \bullet \vec{G}_L \oplus z_{(j)} \vec{H}_R \bullet \vec{G}_L \oplus z_{(j)}^9 \vec{H}_L \bullet \vec{G}_L = B_R \oplus z_{(j)} \Delta_{HR}^{(j)} \oplus z_{(j)}^9 \Delta^{(j)}.
\end{aligned}$$

Consequently, at Line 6 of Algorithm 5, the computed values A'_L, A'_R, B'_L, B'_R satisfy:

$$C' = \vec{A}' \bullet \vec{B}', A'_L = \vec{A'}_L \bullet \vec{H}_L, A'_R = \vec{A'}_R \bullet \vec{H}_L, B'_L = \vec{B'}_L \bullet \vec{G}_L, B'_R = \vec{B'}_R \bullet \vec{G}_L.$$

Given:

$$L = \vec{A'}_L \bullet \vec{B'}_R, \quad R = \vec{A'}_R \bullet \vec{B'}_L,$$

we further obtain:

$$\begin{aligned}
\vec{A''} \bullet \vec{B''} =&[\vec{A}'_L \oplus x_{(j)}^{-1} \vec{A}'_R] \bullet [\vec{B}'_L \oplus x_{(j)} \vec{B}'_R] \\
=&\vec{A}'_L \bullet \vec{B}'_L \oplus \vec{A}'_R \bullet \vec{B}'_R \oplus x_{(j)} \vec{A}'_L \bullet \vec{B}'_R \oplus x_{(j)}^{-1} \vec{A}'_R \bullet \vec{B}'_L \\
=&\vec{A}' \bullet \vec{B}' \oplus x_{(j)} \vec{A}'_L \bullet \vec{B}'_R \oplus x_{(j)}^{-1} \vec{A}'_R \bullet \vec{B}'_L = C' \oplus x_{(j)} L \oplus x_{(j)}^{-1} R, \\
\vec{A''} \bullet \vec{H}_L =&[\vec{A}'_L \oplus x_{(j)}^{-1} \vec{A}'_R] \bullet \vec{H}_L \\
=&\vec{A}'_L \bullet \vec{H}_L \oplus x_{(j)}^{-1} \vec{A}'_R \bullet \vec{H}_L = A'_L \oplus x_{(j)}^{-1} A'_R, \\
\vec{B''} \bullet \vec{G}_L =&[\vec{B}'_L \oplus x_{(j)}^{-1} \vec{B}'_R] \bullet \vec{G}_L \\
=&\vec{B}'_L \bullet \vec{G}_L \oplus x_{(j)}^{-1} \vec{B}'_R \bullet \vec{G}_L = B'_L \oplus x_{(j)}^{-1} B'_R,
\end{aligned}$$

Thus, the values A'', B'', and C'' computed at Line 11 satisfy:

$$C'' = \vec{A''} \bullet \vec{B''}, \quad A'' = \vec{A''} \bullet \vec{H}_L, \quad B'' = \vec{B''} \bullet \vec{G}_L.$$

Finally, based on the update rule at Line 13, the inputs for the next iteration satisfy:

$$C = \vec{A} \bullet \vec{B}, \quad A = \vec{A} \bullet \vec{H}, \quad B = \vec{B} \bullet \vec{G}.$$

Applying mathematical induction establishes the perfect completeness of our protocol.

Knowledge Soundness. At the final iteration, an extractor $\mathcal{K}$ can extract $\vec{A}'' = \hat{A}$ and $\vec{B}'' = \hat{B}$, satisfying:

$$C'' = \vec{A''} \bullet \vec{B''}, \quad A'' = \vec{A''} \bullet \vec{H}_L, \quad B'' = \vec{B''} \bullet \vec{G}_L.$$

We demonstrate that: by rewinding the protocol and providing distinct random challenges, the extractor can extract vectors $\vec{A}$ and $\vec{B}$ from $\vec{A}''$ and $\vec{B}''$ such that:

$$C = \vec{A} \bullet \vec{B}, A = \vec{A} \bullet \vec{H}, B = \vec{B} \bullet \vec{G}.$$

Rewinding the protocol to Line 9 with four distinct challenges $x^{(\tau)}, \tau \in [1,4]$, the extractor obtains four sets:

$$C''^{(\tau)}, A''^{(\tau)}, B''^{(\tau)}, \vec{A}''^{(\tau)}, \vec{B}''^{(\tau)}, \tau \in [1,4],$$

satisfying:

$$\begin{aligned} C' \oplus x^{(\tau)} L \oplus (x^{(\tau)})^{-1} R = C''^{(\tau)} = \vec{A}''^{(\tau)} \bullet \vec{B}''^{(\tau)}, \\ A'_L \oplus (x^{(\tau)})^{-1} A'_R = A''^{(\tau)} = \vec{A}''^{(\tau)} \bullet \vec{H}_L, \\ B'_L \oplus x^{(\tau)} B'_R = B''^{(\tau)} = \vec{B}''^{(\tau)} \bullet \vec{G}_L. \end{aligned} \tag{C35}$$

Given these equations, $\mathcal{K}$ can extract vectors $\vec{A}'^{(\tau)}_L, \vec{A}'^{(\tau)}_R, \vec{B}'^{(\tau)}_L, \vec{B}'^{(\tau)}_R$ from from pairs of values indexed by τ and $(\tau+1 \mod 4)$, satisfying:

$$A'_L = \vec{A}'^{(\tau)}_L \bullet \vec{H}_L, A'_R = \vec{A}'^{(\tau)}_R \bullet \vec{H}_L, B'_L = \vec{B}'^{(\tau)}_L \bullet \vec{G}_L, B'_R = \vec{B}'^{(\tau)}_R \bullet \vec{G}_L.$$

According to Lemma 1, the vectors $\vec{A}'^{(\tau)}_L, \vec{A}'^{(\tau)}_R, \vec{B}'^{(\tau)}_L, \vec{B}'^{(\tau)}_R$ are independent of the choice of τ. Thus, we remove the superscript (τ) and denote them as $\vec{A}'_L, \vec{A}'_R, \vec{B}'_L, \vec{B}'_R$, such that:

$$A'_L = \vec{A}'_L \bullet \vec{H}_L, A'_R = \vec{A}'_R \bullet \vec{H}_L, B'_L = \vec{B}'_L \bullet \vec{G}_L, B'_R = \vec{B}'_R \bullet \vec{G}_L.$$

Substituting these vectors back into Eq. (C35), we derive:

$$\begin{aligned} \vec{A}''^{(\tau)} &= \vec{A}'_L \oplus (x^{(\tau)})^{-1} \vec{A}'_R, \\ \vec{B}''^{(\tau)} &= \vec{B}'_L \oplus x^{(\tau)} \vec{B}'_R, \\ C' \oplus x^{(\tau)} L \oplus (x^{(\tau)})^{-1} R &= [\vec{A}'_L \oplus (x^{(\tau)})^{-1} \vec{A}'_R] \bullet [\vec{B}'_L \oplus x^{(\tau)} \vec{B}'_R]. \end{aligned}$$

Comparing constant terms on both sides, we conclude:

$$C' = \vec{A}'_L \bullet \vec{B}'_L \oplus \vec{A}'_R \bullet \vec{B}'_R = (\vec{A}'_L || \vec{A}'_R) \bullet (\vec{B}'_L || \vec{B}'_R).$$

Next, rewinding again to Line 4 with 11 distinct challenges $z^{(\kappa)}, \kappa \in [1,11]$, yields 11 sets of values:

$$C'^{(\tau)}, A'^{(\tau)}, B'^{(\tau)}, \vec{A}'^{(\kappa)}_L, \vec{A}'^{(\kappa)}_R, \vec{B}'^{(\kappa)}_L, \vec{B}'^{(\kappa)}_R, \quad \kappa \in [1,11],$$

satisfying:

$$\begin{aligned} & C \oplus z^{(\kappa)} A \oplus (z^{(\kappa)})^2 B \oplus (z^{(\kappa)})^3 \Delta \oplus (z^{(\kappa)})^5 \\ & A_L \oplus (z^{(\kappa)})^6 B_L \oplus (z^{(\kappa)})^9 A_R \oplus (z^{(\kappa)})^{10} B_R \\ & \oplus (2(z^{(\kappa)})^4 + (z^{(\kappa)})^8) C_L \oplus (2(z^{(\kappa)})^{11} + (z^{(\kappa)})^{19}) \Delta^{(j)} \end{aligned} \tag{C36}$$

$$\begin{aligned} & = C'^{(\tau)} = (\vec{A}'^{(\kappa)}_L || \vec{A}'^{(\kappa)}_R) \bullet (\vec{B}'^{(\kappa)}_L || \vec{B}'^{(\kappa)}_R) \\ & (1 + (z^{(\kappa)})^4) A_L \oplus (z^{(\kappa)})^2 \Delta^{(j)} = A'^{(\tau)}_L = \vec{A}'^{(\kappa)}_L \bullet \vec{H}_L, \\ & A_R \oplus (z^{(\kappa)})^2 \Delta^{(j)}_{GR} \oplus (z^{(\kappa)})^{10} \Delta^{(j)} = A'^{(\tau)}_R = \vec{A}'^{(\kappa)}_R \bullet \vec{H}_L, \\ & (1 + (z^{(\kappa)})^4) B_L \oplus z^{(\kappa)} \Delta^{(j)} = B'^{(\tau)}_L = \vec{B}'^{(\kappa)}_L \bullet \vec{G}_L, \\ & B_R \oplus z^{(\kappa)} \Delta^{(j)}_{HR} \oplus (z^{(\kappa)})^9 \Delta^{(j)} = B'^{(\tau)}_R = \vec{B}'^{(\kappa)}_R \bullet \vec{G}_L. \end{aligned} \tag{C37}$$

Using any 10 of these 11 sets of values, the extractor $\mathcal{K}$ can uniquely determine the values $\vec{A}_L, \vec{A}_R, \vec{B}_L, \vec{B}_R$ such that:

$$A_L = \vec{A}_L \bullet \vec{H}_L, A_R = \vec{A}_R \bullet \vec{H}_L, B_L = \vec{B}_L \bullet \vec{G}_L, B_R = \vec{B}_R \bullet \vec{G}_L.$$

Substituting these results into Eq. (C37), we derive:

$$\begin{aligned} (1 + (z^{(\kappa)})^4) \vec{A}_L \oplus (z^{(\kappa)})^2 \vec{G}_L &= \vec{A}'^{(\kappa)}_L, \\ \vec{A}_R \oplus (z^{(\kappa)})^2 \vec{G}_R \oplus (z^{(\kappa)})^{10} \vec{G}_L &= \vec{A}'^{(\kappa)}_R, \\ (1 + (z^{(\kappa)})^4) \vec{B}_L \oplus z^{(\kappa)} \vec{H}_L &= \vec{B}'^{(\kappa)}_L, \\ \vec{B}_R \oplus z^{(\kappa)} \vec{H}_R \oplus (z^{(\kappa)})^9 \vec{H}_L &= \vec{B}'^{(\kappa)}_R. \end{aligned}$$

Then, substituting these expressions to Eq. (C36), we obtain:

$$\begin{aligned} & C \oplus z^{(\kappa)} A \oplus (z^{(\kappa)})^2 B \oplus (z^{(\kappa)})^3 \Delta \oplus (z^{(\kappa)})^5 A_L \oplus (z^{(\kappa)})^6 B_L \oplus (z^{(\kappa)})^9 A_R \oplus (z^{(\kappa)})^{10} B_R \\ & \oplus (2(z^{(\kappa)})^4 + (z^{(\kappa)})^8) C_L \oplus (2(z^{(\kappa)})^{11} + (z^{(\kappa)})^{19}) \Delta^{(j)} \\ = & (\vec{A}'^{(\kappa)}_L || \vec{A}'^{(\kappa)}_R) \bullet (\vec{B}'^{(\kappa)}_L || \vec{B}'^{(\kappa)}_R) \\ = & [\vec{A}^* \oplus (z^{(\kappa)})^2 \vec{G} \oplus (z^{(\kappa)})^4 (\vec{A}_L || \vec{0}) \oplus (z^{(\kappa)})^{10} (\vec{0} || \vec{G}_L)] \\ & \bullet [\vec{B}^* \oplus z^{(\kappa)} \vec{H} \oplus (z^{(\kappa)})^4 (\vec{B}_L || \vec{0}) \oplus (z^{(\kappa)})^9 (\vec{0} || \vec{H}_L)], \end{aligned}$$

where $\vec{A}^* = (\vec{A}_L || \vec{A}_R)$ and $\vec{B}^* = (\vec{B}_L || \vec{B}_R)$.

By comparing coefficients of the constant, $z^{(\kappa)}$, and $(z^{(\kappa)})^2$ terms, we conclude:

$$C = \vec{A}^* \bullet \vec{B}^*, \quad A = \vec{A}^* \bullet \vec{H}, \quad B = \vec{B}^* \bullet \vec{G}. \tag{C38}$$

Thus, the extractor $\mathcal{K}$ successfully extracts vectors $\vec{A}^*$ and $\vec{B}^*$. From this extraction procedure, we also observe that each $\vec{A}^{''(\kappa,\tau)}$ and $\vec{B}^{''(\kappa,\tau)}$ is uniquely determined by $\vec{A}$, $\vec{B}$, and the challenges $x^{(\tau)}, z^{(\kappa)}$.

By mathematical induction, the extractor can uniquely extract $\vec{A}^*$ and $\vec{B}^*$ that satisfy Eq. (C38) for $j = 1$, where the C, A, and B at Line 11 of the first iteration correspond to the public input of the protocol. This establishes the knowledge soundness of Algorithm 5. Moreover, the final transcript elements $\hat{A}$ and $\hat{B}$ sent in the protocol are uniquely determined by the extracted vectors $\vec{A}$, $\vec{B}$, and the random challenges.

Computational Soundness. The computational soundness of Algorithm 5 does not rely on Lemma 5. Therefore, for computational soundness alone, the base vectors do not need to be linearly independent. Indeed, we may even employ the zero vectors $\vec{O} \in \mathbb{G}_1^n$ and $\vec{O} \in \mathbb{G}_2^n$ as the base vectors.

Let $C_L^{(j)}, A_L^{(j)}, B_L^{(j)}, A_R^{(j)}, B_R^{(j)}$ denote the group elements sent by the prover in round j at Line 3. Define the following vectors and values iteratively for each round j:

$$\begin{aligned}
&\vec{G}^{(j)} = \vec{G}_{[0,n/2^{j-1}]}, \vec{H}^{(j)} = \vec{H}_{[0,n/2^{j-1}]},\\
&C^{(0)} \leftarrow C, A^{(0)} \leftarrow A, B^{(0)} \leftarrow B, \vec{A}^{(0)} \leftarrow \vec{A}, \vec{B}^{(0)} \leftarrow \vec{B},\\
&C^{(j+1)} \leftarrow C^{''(j)}, A^{(j+1)} \leftarrow A^{''(j)}, B^{(j+1)} \leftarrow B^{''(j)}, \vec{A}^{(j)} \leftarrow \vec{A}^{''(j)}, \vec{B}^{(j)} \leftarrow \vec{B}^{''(j)},\\
&C^{'(j)} \leftarrow C^{(j)} \oplus z^{(j)} A^{(j)} \oplus (z^{(j)})^2 B^{(j)} \oplus (z^{(j)})^3 \Delta \oplus (z^{(j)})^5 A_L^{(j)} \oplus (z^{(j)})^6 B_L^{(j)}\\
&\quad \oplus (z^{(j)})^9 A_R^{(j)} \oplus (z^{(j)})^{10} B_R^{(j)} \oplus (2(z^{(j)})^4 + (z^{(j)})^8) C_L^{(j)} \oplus (2(z^{(j)})^{11} + (z^{(j)})^{19}) \Delta^{(j)}\\
&A_L^{'(j)} \leftarrow (1 + (z^{(j)})^4) A_L^{(j)} \oplus (z^{(j)})^2 \Delta^{(j)}, A_R^{'(j)} \leftarrow A_R^{(j)} \oplus (z^{(j)})^2 \Delta_{GR}^{(j)} \oplus (z^{(j)})^{10} \Delta^{(j)},\\
&B_L^{'(j)} \leftarrow (1 + (z^{(j)})^4) B_L^{(j)} \oplus (z^{(j)}) \Delta^{(j)}, B_R^{'(j)} \leftarrow B_R^{(j)} \oplus (z^{(j)}) \Delta_{HR}^{(j)} \oplus (z^{(j)})^9 \Delta^{(j)},\\
&\vec{A}'^{(j)} \leftarrow \vec{A}^{(j)} \oplus (z^{(j)})^2 \vec{G}^{(j)} \oplus (z^{(j)})^4 (\vec{A}_L^{(j)} || \vec{0}) \oplus (z^{(j)})^{10} (\vec{0} || \vec{G}_L^{(j)}),\\
&\vec{B}'^{(j)} \leftarrow \vec{B}^{(j)} \oplus (z^{(j)}) \vec{H}^{(j)} \oplus (z^{(j)})^4 (\vec{B}_L^{(j)} || \vec{0}) \oplus (z^{(j)})^9 (\vec{0} || \vec{H}_L^{(j)}),\\
&C^{''(j)} \leftarrow C^{'(j)} \oplus x^{(j)} L^{(j)} \oplus (x^{(j)})^{-1} R^{(j)},\\
&A^{''(j)} \leftarrow A_L^{'(j)} \oplus (x^{(j)})^{-1} A_R^{'(j)}\\
&B^{''(j)} \leftarrow B_L^{'(j)} \oplus x^{(j)} B_R^{'(j)}\\
&\vec{A}''^{(j)} \leftarrow \vec{A}_L^{'(j)} \oplus (x^{(j)})^{-1} \vec{A}_R',\\
&\vec{B}''^{(j)} \leftarrow \vec{B}_L^{'(j)} \oplus x^{(j)} \vec{B}_R^{'(j)},\\
&C \leftarrow C^{(n)}, A \leftarrow A^{(n)}, B \leftarrow B^{(n)}, \hat{A} \leftarrow \vec{A}''^{(n)}, \hat{B} \leftarrow \vec{B}''^{(n)}.
\end{aligned} \tag{C39}$$

Using the Schwartz-Zippel lemma, we have: If the following condition ① does not hold:

$$\begin{aligned}
①C^{(j)} &= \vec{A}^{(j)} \bullet \vec{B}^{(j)} \wedge A^{(j)} = \vec{A}^{(j)} \bullet \vec{H}^{(j)} \wedge B^{(j)}\\
&= \vec{A}^{(j)} \bullet \vec{G}^{(j)} \wedge C_L^{(j)} = \vec{A}_L^{(j)} \bullet \vec{B}_L^{(j)},
\end{aligned}$$

then with high probability, condition ② also does not hold:

$$②C^{'(j)} = \vec{A}^{'(j)} \bullet \vec{B}^{'(j)} \wedge A_L^{'(j)} = \vec{A}_L^{'(j)} \bullet \vec{H}_L^{(j)} \wedge A_R^{'(j)} = \vec{A}_R^{'(j)} \bullet \vec{H}_L^{(j)} \wedge B_L^{'(j)} = \vec{B}_L^{(j)} \bullet \vec{G}_L^{(j)} \wedge B_R^{'(j)} = \vec{B}_R^{(j)} \bullet \vec{G}_L^{(j)}.$$

This failure occurs independently of the values of $C_L^{(j)}, A_L^{(j)}, A_R^{(j)}, B_L^{(j)}, B_R^{(j)}$ sent by the prover, provided that $z^{(j)}$ is randomly chosen. The failure of condition ② implies with high probability the failure of the following condition ③:

$$③C^{''(j)} = \vec{A}^{''(j)} \bullet \vec{B}^{''(j)} \wedge A^{''(j)} = \vec{A}^{''(j)} \bullet \vec{H}_L^{(j)} \wedge B^{''(j)} = \vec{A}^{''(j)} \bullet \vec{G}_L^{(j)}.$$

Similarly, this failure is independent of the choice of $L^{(j)}$ and $R^{(j)}$. The failure of condition ③ further implies the failure of condition ④ with high probability:

$$\begin{aligned}
④C^{(j+1)} &= \vec{A}^{(j+1)} \bullet \vec{B}^{(j+1)} \wedge A^{(j+1)} = \vec{A}^{(j+1)} \bullet \vec{H}^{(j+1)}\\
\wedge B^{(j+1)} &= \vec{A}^{(j+1)} \bullet \vec{G}^{(j+1)} \wedge C_L^{(j+1)} = \vec{A}_L^{(j+1)} \bullet \vec{B}_L^{(j+1)}
\end{aligned}$$

In summary, we have the chain of implications:

$$\neg ① \implies \neg ② \implies \neg ③ \implies \neg ④,$$

where condition ④ is condition ① with the index j incremented by 1.

Thus, by mathematical induction, if initially:

$$\neg\{C = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{B}} \wedge A = \vec{\boldsymbol{A}} \bullet \vec{\boldsymbol{H}} \wedge B = \vec{\boldsymbol{B}} \bullet \vec{\boldsymbol{G}}\},$$

and if all computations in Eq. (C39) are correctly executed, the verifier returns FALSE with high probability. □

C.6 Proof of Proposition 7

Proof

In Algorithm 5, the values $\hat{A}$ and $\hat{B}$ obtained at Line 14 satisfy the following homomorphic property:

1. $\hat{A}$ is determined solely by $\vec{\boldsymbol{A}}$ and $\vec{\boldsymbol{G}}$, independent of $\vec{\boldsymbol{B}}$ and $\vec{\boldsymbol{H}}$; similarly, $\hat{B}$ is determined solely by $\vec{\boldsymbol{B}}$ and $\vec{\boldsymbol{H}}$, independent of $\vec{\boldsymbol{A}}$ and $\vec{\boldsymbol{G}}$.
2. If $\vec{\boldsymbol{A}} = \vec{\boldsymbol{A}}_1 \oplus \vec{\boldsymbol{A}}_1$ and $\vec{\boldsymbol{G}} = \vec{\boldsymbol{G}}_1 \oplus \vec{\boldsymbol{G}}_2$, then $\hat{A} = \hat{A}_1 \oplus \hat{A}_2$, where each $\hat{A}_i, i \in [1,2]$ is computed from corresponding pairs $\vec{\boldsymbol{A}}_i$ and $\vec{\boldsymbol{G}}_i$. Analogous results hold for $\hat{B}$.

For inner products ①,②,③, and ④, $\hat{A}$ and $\hat{B}$ are computed from the following group vectors:

$$\begin{aligned}
&\hat{A}_{①}: \ (\vec{\boldsymbol{G}}, \vec{\boldsymbol{G}}), \qquad \hat{B}_{①}: \ (\vec{\boldsymbol{H}}, \vec{\boldsymbol{H}}),\\
&\hat{A}_{②}: \ (\vec{\boldsymbol{O}}, \vec{\boldsymbol{G}}), \qquad \hat{B}_{②}: \ (\vec{\boldsymbol{O}}, \vec{\boldsymbol{H}}),\\
&\hat{A}_{③}: \ (\vec{\boldsymbol{G}}, \vec{\boldsymbol{O}}), \qquad \hat{B}_{③}: \ (\vec{\boldsymbol{a}}\hat{H}_0, \vec{\boldsymbol{H}}),\\
&\hat{A}_{④}: \ (\vec{\boldsymbol{b}}\hat{G}_0, \vec{\boldsymbol{G}}), \qquad \hat{B}_{④}: \ (\vec{\boldsymbol{H}}, \vec{\boldsymbol{O}}).
\end{aligned}$$

For an additional inner product ⑤: $(\vec{\boldsymbol{b}}\hat{G}_0) \bullet (\vec{\boldsymbol{a}}\hat{H}_0)$, with bases $\vec{\boldsymbol{O}} \in \mathbb{G}_1^n$ and $\vec{\boldsymbol{O}} \in \mathbb{G}_2^n$, the corresponding $\hat{A}_{⑤}$ and $\hat{B}_{⑤}$ are determined by:

$$\hat{A}_{⑤}: \ (\vec{\boldsymbol{b}}\hat{G}_0, \vec{\boldsymbol{O}}), \quad \hat{B}_{⑤}: \ (\vec{\boldsymbol{a}}\hat{H}_0, \vec{\boldsymbol{O}}).$$

By the homomorphic property, we derive:

$$\begin{aligned}
&\hat{A}_{③} = \hat{A}_{①} \ominus \hat{A}_{②}, \quad \hat{B}_{③} = \hat{B}_{②} \oplus \hat{B}_{⑤},\\
&\hat{A}_{④} = \hat{A}_{②} \oplus \hat{A}_{⑤}, \quad \hat{B}_{④} = \hat{B}_{①} \ominus \hat{B}_{②}.
\end{aligned}$$

The knowledge soundness of Algorithm 5 ensures reliable computation of $\hat{A}_{①}, \hat{B}_{①}$, and $\hat{A}_{②}, \hat{B}_{②}$, while $\hat{A}_{⑤}, \hat{B}_{⑤}$ can be efficiently computed by the verifier when $\vec{\boldsymbol{a}}$ and $\vec{\boldsymbol{b}}$ have tensor structures.

Specifically, when $\vec{\boldsymbol{a}} = (1, x_{a(1)}) \otimes \cdots \otimes (1, x_{a(\log_2 n)}) \in \mathbb{Z}_p^n$ for some $x_{a(1)}, \ldots, x_{a(\log_2 n)} \in \mathbb{Z}_p$, we have:

$$\vec{\boldsymbol{B}}_R = x_{a(1)} \vec{\boldsymbol{B}}_L.$$

Let $\vec{\boldsymbol{B}} = \vec{\boldsymbol{a}}\hat{H}_0$ and $\vec{\boldsymbol{H}} = \vec{\boldsymbol{O}}$, the vector $\vec{\boldsymbol{B}}''$ computed in the first round of Algorithm 5 becomes:

$$\begin{aligned}
\vec{\boldsymbol{B}}'' &= \vec{\boldsymbol{B}}_L' \oplus x\vec{\boldsymbol{B}}_R' = (\vec{\boldsymbol{B}}_L + z^4 \vec{\boldsymbol{B}}_L) \oplus x\vec{\boldsymbol{B}}_R = (1 + z^4)\vec{\boldsymbol{B}}_L \oplus (x \cdot x_{a(1)})\vec{\boldsymbol{B}}_L\\
&= (1 + z^4 + x \cdot x_{a(1)})\vec{\boldsymbol{B}}_L.
\end{aligned}$$

Iteratively applying this result yields:

$$\hat{B}_{⑤} = [\prod_{j=1}^{\log_2 n} (1 + z_{(j)}^4 + x_{(j)} \cdot x_{a(j)})]\hat{H}_0,$$

where $z_{(j)}, x_{(j)}$, $j \in [1, \log_2 n]$, are the random challenges generated in Algorithm 5 at the j-th round.

Similarly, when $\vec{\boldsymbol{b}} = (1, x_{b(1)}) \otimes \cdots \otimes (1, x_{b(\log_2 n)}) \in \mathbb{Z}_p^n$ for some $x_{b(1)}, \ldots, x_{b(\log_2 n)} \in \mathbb{Z}_p$, the verifier can efficiently compute:

$$\hat{A}_{⑤} = [\prod_{j=1}^{\log_2 n} (1 + z_{(j)}^4 + (x_{(j)})^{-1} \cdot x_{b(j)})]\hat{G}_0.$$

Consequently, $\hat{A}_{①}, \hat{B}_{①}$, $\hat{A}_{②}, \hat{B}_{②}$, and $\hat{A}_{⑤}, \hat{B}_{⑤}$ computed in Algorithm 6 are reliable. The homomorphic property implies that $\hat{A}_{③}, \hat{B}_{③}$, and $\hat{A}_{④}, \hat{B}_{④}$ are also reliable. Thus, the computational soundness of Algorithm 5 ensures the validity of the following inner products:

$$③: \ e(\hat{G}_0, V_a) = \vec{\boldsymbol{G}} \bullet (\vec{\boldsymbol{a}}\hat{H}_0), \quad ④: \ e(V_b, \hat{H}_0) = (\vec{\boldsymbol{a}}\hat{G}_0) \bullet \vec{\boldsymbol{H}}.$$

This further implies:

$$V_a = \vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}}, \quad V_b = \vec{\boldsymbol{b}} \bullet \vec{\boldsymbol{H}}.$$

Hence, Algorithm 6 is a proof for $\mathcal{L}_{\mathsf{pip}}$. □

C.7 Proof of Proposition 8

Proof

Perfect Completeness. From the perfect completeness of subprotocols in Lines 7-11 of Algorithm 7, we know these subprotocols return TRUE.

If $\mathcal{P}$ holds $\vec{\boldsymbol{a}}$, $\vec{\boldsymbol{b}}$, and $\tilde{d}$ that satisfies $\mathcal{R}_{\mathsf{zklpGt}}$, then it also holds a witness for $\mathcal{R}_{\mathsf{zklp}}$ where $C := \tilde{C} \ominus (\tilde{c}\tilde{U}) = (\vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{b}})U \oplus (\vec{\boldsymbol{a}} \bullet \vec{\boldsymbol{G}}) \oplus (\vec{\boldsymbol{b}} \bullet \vec{\boldsymbol{H}})$. From the perfect completeness of Algorithm 3, we know the verification check in Line 8 of Algorithm 3 returns TRUE.

Hence:

$$\begin{aligned} F =& \{\tilde{C} \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} \tilde{L}_{(j)} \oplus x_{(j)}^{-1} \tilde{R}_{(j)})\} \ominus \{\tilde{C}_U \oplus \tilde{A}_V \oplus \tilde{B}_V\} \\ =& \{C \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} L_{(j)} \oplus x_{(j)}^{-1} R_{(j)})\} \ominus \{(\hat{a}\hat{b})U \oplus \hat{a} C_H \oplus \hat{b} C_G\} \\ & \ominus \{\tilde{c} + \sum_{j=1}^{\log_2 n} (x_{(j)} \tilde{l}_{(j)} + x_{(j)}^{-1} \tilde{r}_{(j)}) - (\tilde{c}_U + \tilde{a}_V + \tilde{b}_V)\} \tilde{U} \\ =& \tilde{f} \tilde{U}. \end{aligned} \tag{C40}$$

Because there is no non-trivial of discrete logarithm relation between U and $\tilde{U}$:

$$\hat{a}^{\mathsf{mul}} = \hat{a}^{\mathsf{semi}} := \hat{a}, \quad \hat{b}^{\mathsf{mul}} = \hat{b}^{\mathsf{semi}} := \hat{b},$$
$$\tilde{a}^{\mathsf{mul}} = \tilde{a}^{\mathsf{semi}} := \tilde{a}, \quad \tilde{b}^{\mathsf{mul}} = \tilde{b}^{\mathsf{semi}} := \tilde{b}.$$

Hence,

$$\tilde{C}_U = \hat{a}\hat{b}U \oplus \tilde{c}_U \tilde{U}, \quad \tilde{A}_V = \hat{a} C_H \oplus \tilde{a}_V \tilde{U}, \quad \tilde{B}_V = \hat{b} C_G \oplus \tilde{b}_V \tilde{U}.$$

Computational knowledge soundness of $\mathbb{G}_1$-public-inner-product protocol and the $\mathbb{G}_2$-public-inner-product protocol ensure that:

$$V_H = \vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{H}}, \quad V_G = \vec{\boldsymbol{\xi}}^{-1} \bullet \vec{\boldsymbol{G}}.$$

Hence,

$$C_H = e(\hat{G}_0, \vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{H}}), \quad C_G = e(\vec{\boldsymbol{\xi}}^{-1} \bullet \vec{\boldsymbol{G}}, \hat{H}_0).$$

Additionally, applying the extractor of Schnorr's protocol yields $\tilde{f}$ such that $\tilde{F} = \tilde{f}\tilde{U}$. Substituting this into the computation of F in Line 12 yields:

$$\begin{aligned} \tilde{f}\tilde{U} = \tilde{F} =& \{\tilde{C} \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} \tilde{L}_{(j)} \oplus x_{(j)}^{-1} \tilde{R}_{(j)})\} \ominus \{\tilde{C}_U \oplus \tilde{A}_V \oplus \tilde{B}_V\} \\ =& \{\tilde{C} \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} \tilde{L}_{(j)} \oplus x_{(j)}^{-1} \tilde{R}_{(j)})\} \\ & \ominus \{\langle \hat{G}_0 \mid \hat{a}\hat{b} \mid \hat{H}_0 \rangle \oplus \langle \hat{G}_0 \mid \hat{a} \mid \vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{H}} \rangle \oplus \langle \vec{\boldsymbol{\xi}}^{-1} \bullet \vec{\boldsymbol{G}} \mid \hat{b} \mid \hat{H}_0 \rangle\} \ominus (\tilde{c}_U + \tilde{a}_V + \tilde{b}_V)\tilde{U}. \end{aligned}$$

Consequently, the Schnorr's protocol also returns TRUE.

Knowledge Soundness. Suppose an adversary $\mathcal{A}$ can successfully generated a transcript in probabilistic polynomial time. By using the extractors of the subprotocols in Lines 9-11 of Algorithm 7, $\mathcal{A}$ can extract:

$$\hat{a}^{\mathsf{mul}}, \hat{b}^{\mathsf{mul}}, \tilde{c}_U, \tilde{a}^{\mathsf{mul}}, \tilde{b}^{\mathsf{mul}}, \hat{a}^{\mathsf{semi}}, \tilde{a}^{\mathsf{semi}}, \tilde{a}_V, \hat{b}^{\mathsf{semi}}, \tilde{b}^{\mathsf{semi}}, \tilde{b}_V \in \mathbb{Z}_p,$$

satisfying:

$$\tilde{C}_U = \hat{a}^{\mathsf{mul}} \hat{b}^{\mathsf{mul}} U \oplus \tilde{c}_U \tilde{U}, \ \tilde{A} = \hat{a}^{\mathsf{mul}} U \oplus \tilde{a}^{\mathsf{mul}} \tilde{U}, \quad \tilde{B} = \hat{b}^{\mathsf{mul}} U \oplus \tilde{b}^{\mathsf{mul}} \tilde{U}, \tag{C41}$$

$$\tilde{A}_V = \hat{a}^{\mathsf{semi}} C_H \oplus \tilde{a}_V \tilde{U}, \quad \tilde{A} = \hat{a}^{\mathsf{semi}} U \oplus \tilde{a}^{\mathsf{semi}} \tilde{U}, \tag{C42}$$

$$\tilde{B}_V = \hat{b}^{\mathsf{semi}} C_G \oplus \tilde{b}_V \tilde{U}, \quad \tilde{B} = \hat{b}^{\mathsf{semi}} U \oplus \tilde{b}^{\mathsf{semi}} \tilde{U}. \tag{C43}$$

This implies, $\mathcal{A}$ can extract $f := \tilde{f} + (\tilde{c}_U + \tilde{a}_V + \tilde{b}_V)$ such that:

$$\begin{aligned} & \tilde{C} \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} \tilde{L}_{(j)} \oplus x_{(j)}^{-1} \tilde{R}_{(j)}) \\ =& f\tilde{U} \oplus \langle \hat{G}_0 \mid \hat{a}\hat{b} \mid \hat{H}_0 \rangle \oplus \langle \hat{G}_0 \mid \hat{a} \mid \vec{\boldsymbol{\xi}} \bullet \vec{\boldsymbol{H}} \rangle \oplus \langle \vec{\boldsymbol{\xi}}^{-1} \bullet \vec{\boldsymbol{G}} \mid \hat{b} \mid \hat{H}_0 \rangle. \end{aligned}$$

By using 2 distinct values of each $x_{(j)}$, $\mathcal{A}$ can solve the linear equation and express $\tilde{C}$ as a linear combination of elements from:

$$\{\vec{\boldsymbol{G}} \otimes \hat{H}_0\} \cup \{\hat{G}_0 \otimes \vec{\boldsymbol{H}}\} \cup \{\hat{G}_0 \otimes \hat{H}_0\} \cup \{\tilde{U}\} \subset \mathbb{G}_T.$$

The coefficient of $\tilde{U}$, denoted by $\tilde{c}$, is unique. Otherwise the assumption of absence of non-trivial logarithm relation is violated.

Define $C = \tilde{C} \ominus \tilde{c}\tilde{U}$. By rewinding the overarching protocol again and provide it with new randomness, we must have:

$$C \oplus \bigoplus_{j=1}^{\log_2 n} (x_{(j)} \tilde{L}_{(j)} \oplus x_{(j)}^{-1} \tilde{R}_{(j)})$$
$$= \langle \hat{G}_0 \mid \hat{a}\hat{b} \mid \hat{H}_0 \rangle \oplus \langle \hat{G}_0 \mid \hat{a} \mid \vec{\xi} \bullet \vec{H} \rangle \oplus \langle \vec{\xi}^{-1} \bullet \vec{G} \mid \hat{b} \mid \hat{H}_0 \rangle.$$

That is, the verification checks of Algorithm 3 are passed for the subsequent rewindings for the public input C.

Hence, $\mathcal{A}$ can apply the extractor of Algorithm 3 to extract a valid witness $\vec{a}$ and $\vec{b}$ for $\mathcal{R}_{\mathsf{ipGt}}$. It can be seen that $\vec{a}, \vec{b}$, and $\tilde{c}$ form a valid witness for $\mathcal{R}_{\mathsf{zkIpGt}}$

SHVZK. All transcript elements generated by the hiding commitments are uniformly distributed in $\mathbb{G}_T$. The atomic subprotocols in Lines 9-11 are all SHVZK, while the information for the subprotocols in Lines 8 and 7 is naturally public. Consequently, the overarching protocol exhibits the SHVZK property. □

Acknowledgements
We would like to thank the anonymous reviewers for detailed comments and useful feedback.

Author Contributions
The first author designed the proposed scheme and drafted the manuscript. All authors participated in discussions, verified the correctness of the scheme, and critically reviewed and approved the final manuscript.

Funding
Not applicable.

Data Availability
Our code is available at https://github.com/mirandaprivate/dualmatrix.

Declarations

Conflict of interest
The authors declare that they have no Conflict of interest.

Received: 6 May 2025 Accepted: 29 July 2025
Published online: 22 February 2026

References

Abe M, Fuchsbauer G, Groth J, Haralambiev K, Ohkubo M (2010) Structure-preserving signatures and commitments to group elements. CRYPTO 209–236
Ames S, Hazay C, Ishai Y, Venkitasubramaniam M (2017) Ligero: Lightweight sublinear arguments without a trusted setup. In: CCS, pp 2087—2104
Attema T, Cramer R, Kohl L (2021) A compressed σ-protocol theory for lattices. In: CRYPTO, pp 549–579
Attema T, Fehr S, Klooß M (2022) Fiat-Shamir transformation of multi-round interactive proofs. In: TCC, pp 113–142
Ben-Sasson E et al (2019) Aurora: Transparent succinct arguments for r1cs. In: EUROCRYPT, pp 103–128
Bootle J, Cerulli A, Chaidos P, Groth J, Petit C (2016) Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. IN: EUROCRYPT, pp 327–357
Bootle J, Chiesa A, Hu Y Orrú M (2022) Gemini: Elastic SNARKs for diverse environments. In: EUROCRYPT, pp 427–457
Bünz B et al (2018) Bulletproofs: Short proofs for confidential transactions and more. S &P 315–334
Bünz B, Fisch B, Szepieniec A (2020) Transparent SNARKs from DARK compilers. IN: EUROCRYPT, pp 677–706
Bünz B, Maller M, Mishra P, Tyagi N, Vesely P (2021) Proofs for inner pairing products and applications. In: ASIACRYPT, pp 65–97
Campanelli M, Fiore D, Querol A (2019) LegoSnark: Modular design and composition of succinct zero-knowledge proofs. In: CCS, pp 2075–2092
Canetti R, et al (2018) Fiat-shamir from simpler assumptions. IACR Cryptol. ePrint Arch. 2018/1004
Chen B, Bünz B, Boneh D, Zhang Z (2023) Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In: EUROCRYPT, pp 499–530
Chiesa A et al (2020) Marlin: preprocessing zkSNARKs with universal and updatable SRS. In: EUROCRYPT, pp 738–768
Chiesa A, Ojha D, Spooner N (2020) Fractal: post-quantum and transparent recursive proofs from holography. In: EUROCRYPT, pp 769–793
Cong M, Yuen TH, Yiu SM (2024) zkMatrix: Batched short proof for committed matrix multiplication. In: ASIACCS, pp 289–305
Gabizon A, Williamson ZJ, Ciobotaru O (2019) Plonk: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. IACR Cryptol. ePrint Arch. 2019/953
Gennaro R, Gentry C, Parno B, Raykova M (2013) Quadratic span programs and succinct NIZKs without PCPs. In: EUROCRYPT, pp 626–645
Gentry C, Halevi S, Lyubashevsky V (2022) Practical non-interactive publicly verifiable secret sharing with thousands of parties. In: EUROCRYPT, pp 458–487
Ghodsi Z, Gu T, Garg, S (2017) Verifiable execution of deep neural networks on an untrusted cloud. In: NIPS, SafetyNets
Golovnev A, Lee J, Setty S, Thaler J, Wahby RS (2023) Brakedown: linear-time and field-agnostic SNARKs for R1CS. In: CRYPTO, pp 193–226
Groth J (2004) Honest verifier zero-knowledge arguments applied. Ph.D. thesis, BRICS
Groth J (2009) Linear algebra with sub-linear zero-knowledge arguments. In: CRYPTO, pp 192–208
Groth J (2011) Efficient zero-knowledge arguments from two-tiered homomorphic commitments. In: ASIACRYPT, pp 431–448
Groth J (2016) On the size of pairing-based non-interactive arguments. In: EUROCRYPT, pp 305–326
Hoffmann M, Klooß M, Rupp A (2019) Efficient zero-knowledge arguments in the discrete log setting, revisited. In: CCS, pp 2093-2110
Lee J (2021) Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In: TCC, pp 1–34
Liu T, Xie X, Zhang Y (2021) zkCNN: zero knowledge proofs for convolutional neural network predictions and accuracy. In: CCS, pp 2968—2985
Maller M, Bowe S, Kohlweiss M, Meiklejohn S (2019) Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings. In: CCS, pp 2111–2128
Parno B, Howell J, Gentry C, Raykova M (2016) Pinocchio: Nearly practical verifiable computation. Commun ACM 59:103–112
Setty S (2020) Spartan: Efficient and general-purpose zksnarks without trusted setup. In: CRYPTO, pp 704–737
Setty S, Lee J (2020) Quarks: Quadruple-efficient transparent zksnarks. IACR Cryptol. ePrint Arch. 2020/1275
Thaler J (2013) Time-optimal interactive proofs for circuit evaluation. In: CRYPTO, pp 71–89
Touvron H et al (2023) Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288
Weng C, Yang K, Xie X, Katz J, Wang X (2021) Mystique: efficient conversions for Zero-Knowledge proofs with applications to machine learning. In: USENIX, pp 501–518
Williams VV (2012) Multiplying matrices faster than coppersmith-winograd. STOC 887–898
Xie T, Zhang J, Zhang Y, Papamanthou C, Song D (2019) Libra: Succinct zero-knowledge proofs with optimal prover computation. CRYPTO, pp 733–764
Xu P et al (2024) Retrieval meets long context large language models. ICLR
Xu G, Li H, Liu S, Yang K, Lin X (2019) Verifynet: secure and verifiable federated learning. IEEE Trans Inf Forensics Secur 15:911–926
Yang K, Sarkar P, Weng C Wang X (2021) Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In: CCS, pp 2986–3001

Yuen TH, Esgin MF, Liu JK, Au MH, Ding Z (2021) DualRing: generic construction of ring signatures with efficient instantiations. In: CRYPTO, pp 251–281
Yuster R, Zwick U (2005) Fast sparse matrix multiplication. ACM Trans Algorithm 1:2–13
Zhang J, Xie T, Zhang Y, Song D (2020) Transparent polynomial delegation and its applications to zero knowledge proof. In: S &P, pp 859–876

Publisher's Note